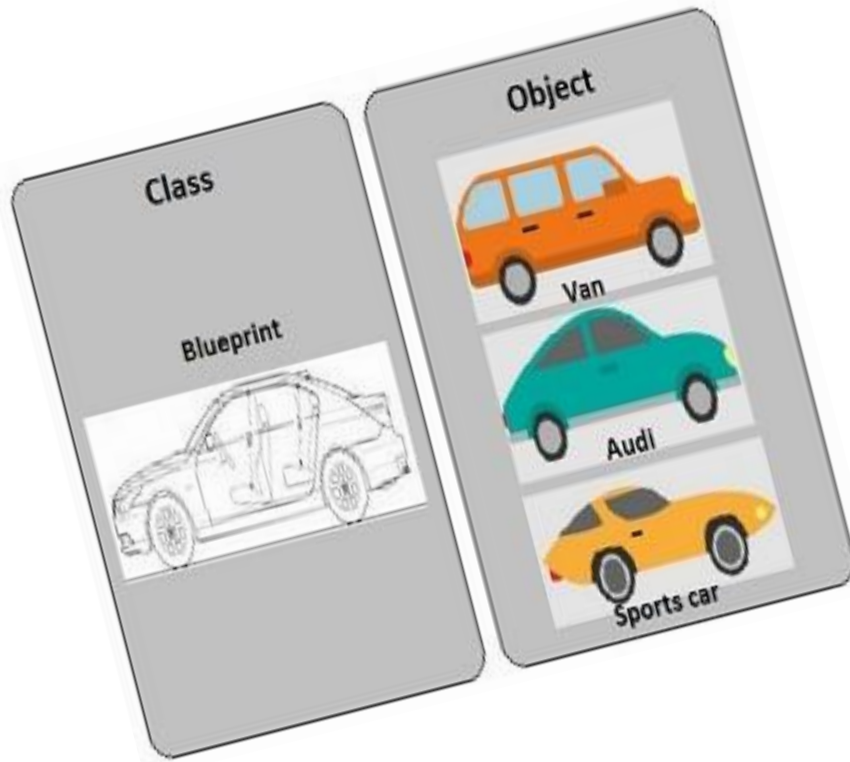




Writing our own classes
to build
custom data types

Computer Science OOD
Boston University

Christine Papadakis-Kanaris

Why the need for classes?

- Let's say I want to keep track of data on my students:
 - **Name {first, last, middle}**
 - **Date of birth {month, day, year}**
 - **Student ID**
- Let's assume all data is stored as strings.
- How can I maintain all this data for each student?
- We wouldn't create a variable for each piece of data multiplied by all students?
- **We could use arrays. So let's start there!**

Why the need for classes?

```
String [] studentName = new String[N];  
String [] dob = new String[N];  
String [] sid = new String[N];
```

studentName



chris	pam	nick	john	molly					
-------	-----	------	------	-------	--	--	--	--	--

dob



2/21	3/14	12/9	5/3	8/9					
------	------	------	-----	-----	--	--	--	--	--

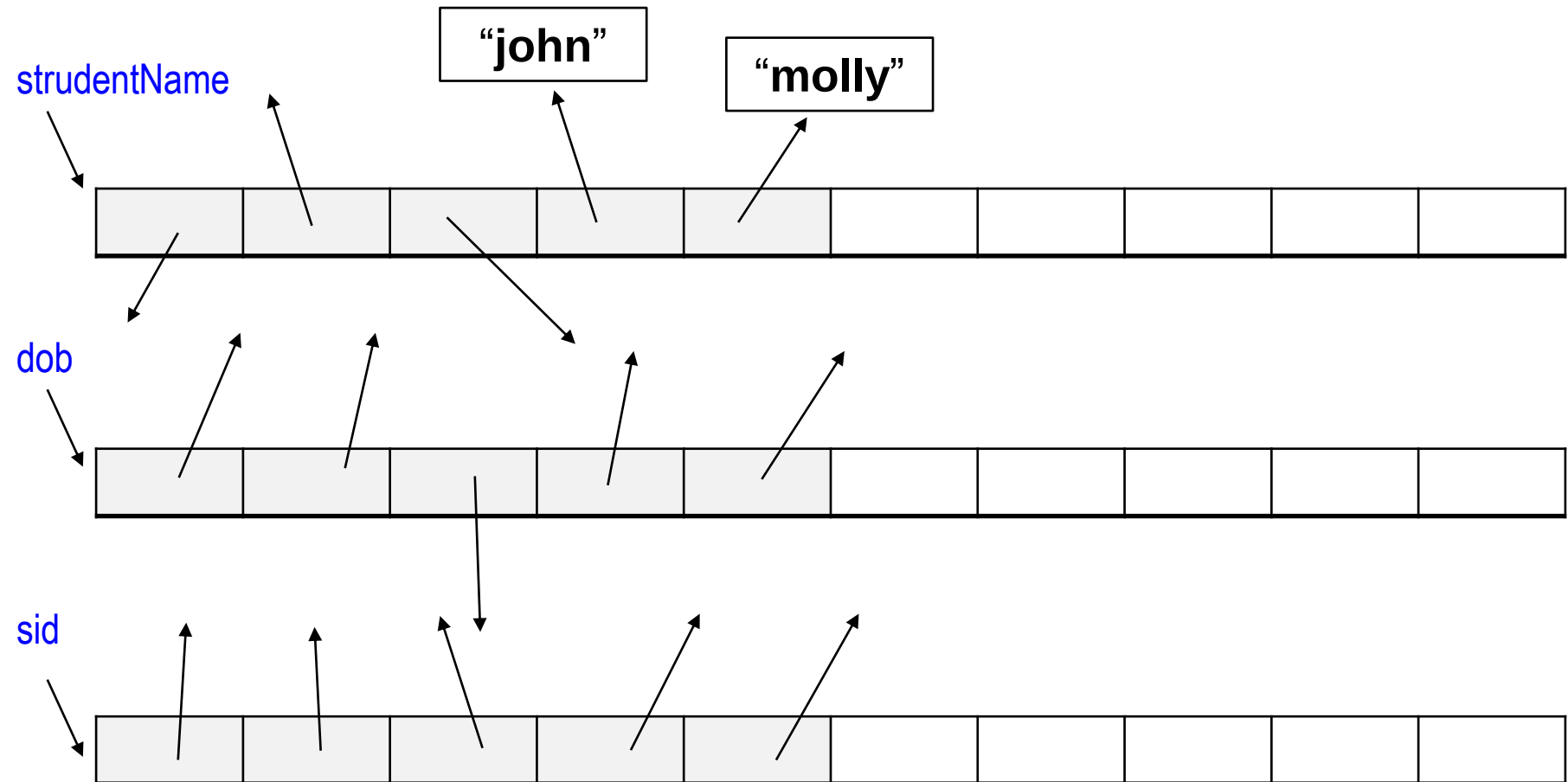
sid



a00	a11	a22	a33	a44					
-----	-----	-----	-----	-----	--	--	--	--	--

Why the need for classes?

```
String [] studentName = new String[N];  
String [] dob = new String[N];  
String [] sid = new String[N];
```



Why the need for classes?

```
String [] studentName = new String[N];  
int [] month = new int[N];  
int [] day = new int[N];  
int [] year = new int[N];
```

... three *integer* arrays
to properly represent the
date of birth!

studentName



chris	pam	nick	john	molly					
-------	-----	------	------	-------	--	--	--	--	--

month



2	3	12	5	8					
---	---	----	---	---	--	--	--	--	--

day



21	14	9	3	9					
----	----	---	---	---	--	--	--	--	--

year



1989	2001	1996	2012	1999					
------	------	------	------	------	--	--	--	--	--

sid



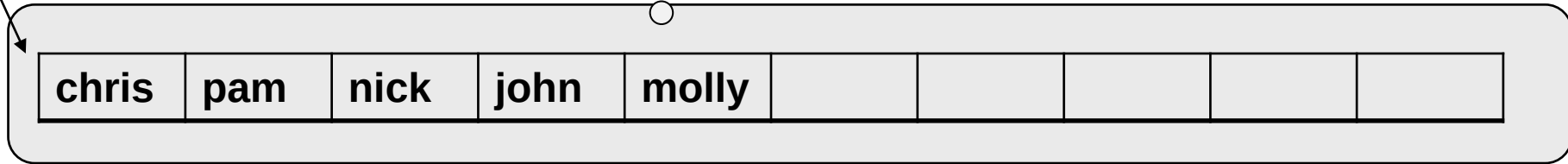
a00	a11	a22	a33	a44					
-----	-----	-----	-----	-----	--	--	--	--	--

Why the need for classes

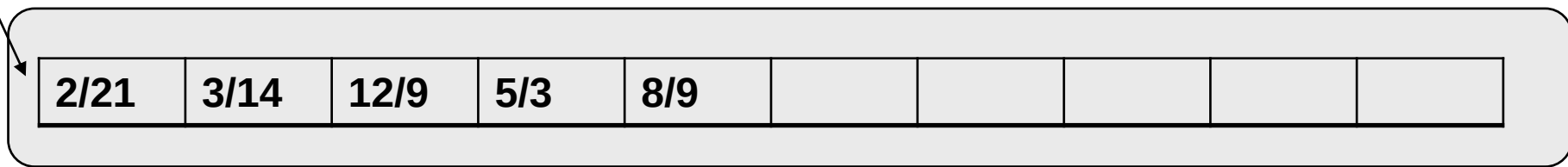
```
String [] studentName = new String[10];  
String [] dob = new String[10];  
String [] sid = new String[10];
```

Recall the physical
memory layout of
an array is contiguous.

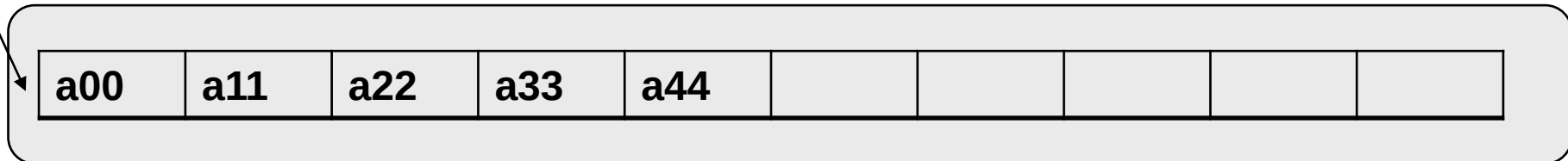
studentName



dob



sid



Why the need for class

```
String [] studentName = new String[N];  
String [] dob = new String[N];  
String [] sid = new String[N];
```

We would usually
be interested in
all the data for a
specific student

studentName



chris	pam	nick	john	molly					
-------	-----	------	------	-------	--	--	--	--	--

dob



2/21	3/14	12/9	5/3	8/9					
------	------	------	-----	-----	--	--	--	--	--

sid



a00	a11	a22	a33	a44					
-----	-----	-----	-----	-----	--	--	--	--	--

Why the need for class

```
String [] studentName = new String[N];  
String [] dob = new String[N];  
String [] sid = new String[N];
```

Therefore we rely
on the index:
studentName[4]
dob[4]
sid[4]

studentName

chris	pam	nick	john	molly					
-------	-----	------	------	-------	--	--	--	--	--

dob

2/21	3/14	12/9	5/3	8/9					
------	------	------	-----	-----	--	--	--	--	--

sid

a00	a11	a22	a33	a44					
-----	-----	-----	-----	-----	--	--	--	--	--

Why the need for classes

```
String [] studentName = new String[10];  
String [] dob = new String[10];  
String [] sid = new String[10];
```

Consider the
inefficiency
of the memory
use in this scenario?

studentName

chris	pam	nick	john	joy	molly				
-------	-----	------	------	-----	-------	--	--	--	--

dob

2/21	3/14	12/9	5/3	9/1	8/9				
------	------	------	-----	-----	-----	--	--	--	--

sid

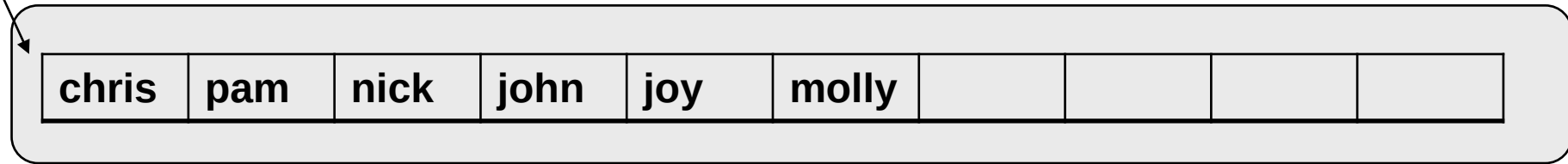
a00	a11	a22	a33	a34	a44				
-----	-----	-----	-----	-----	-----	--	--	--	--

Why the need for class?

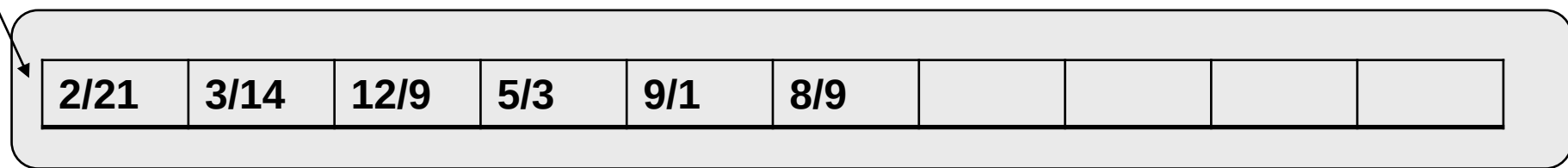
```
String [] studentName = new String[10];  
String [] dob = new String[10];  
String [] sid = new String[10];
```

The problem is that our **logical** view does not match the physical memory *layout* of our data!

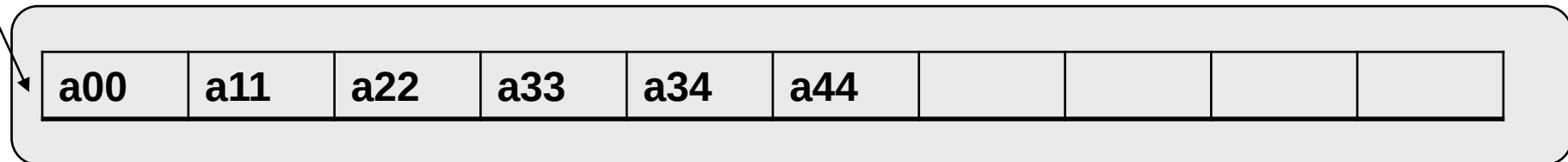
studentName



dob



sid



Why the need for class?

```
String [] studentName = new String[10];  
String [] dob = new String[10];  
String [] sid = new String[10];
```

We need a physical
representation
that matches
our logical view!

studentName

chris	pam	nick	john	joy	molly				
-------	-----	------	------	-----	-------	--	--	--	--

dob

2/21	3/14	12/9	5/3	9/1	8/9				
------	------	------	-----	-----	-----	--	--	--	--

sid

a00	a11	a22	a33	a34	a44				
-----	-----	-----	-----	-----	-----	--	--	--	--

Why the need for class?

```
String [] studentName = new String[10];  
String [] dob = new String[10];  
String [] sid = new String[10];
```

This is what
class definitions
allow us to do!

studentName

chris	pam	nick	john	joy	molly				
-------	-----	------	------	-----	-------	--	--	--	--

dob

2/21	3/14	12/9	5/3	9/1	8/9				
------	------	------	-----	-----	-----	--	--	--	--

sid

a00	a11	a22	a33	a34	a44				
-----	-----	-----	-----	-----	-----	--	--	--	--

Why the need for classes?

```
class Student {  
    String name;  
    String dob;  
    String sid;  
}
```

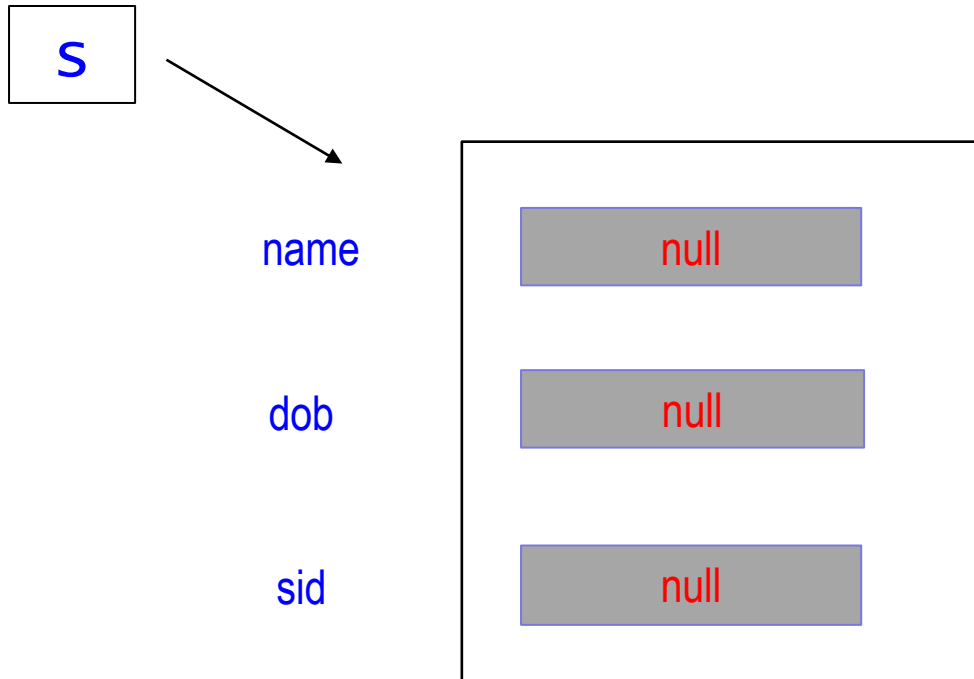
Student is a custom datatype that we created and can use to create an *instance* of the Student class: the physical object created from this class definition.

Why the need for classes?

```
class Student {  
    String name;  
    String dob;  
    String sid;  
}
```

Create an instance of the Student class:

```
Student s = new Student();
```



Why the need for classes?

```
class Student {  
    Name name;  
    Date dob;  
    String sid;  
}
```

Create an instance of the Student class!

We can even define **new** classes, that logically represent a person's name as *first, middle, last* and date of birth as *month, day, year*.

S

name

dob

sid

instance of class Name

instance of class Date

"A00"

Why the need for classes?

```
class Student {  
    Name name;  
    Date dob;  
    String sid;  
}
```

Create an instance of the Student class!

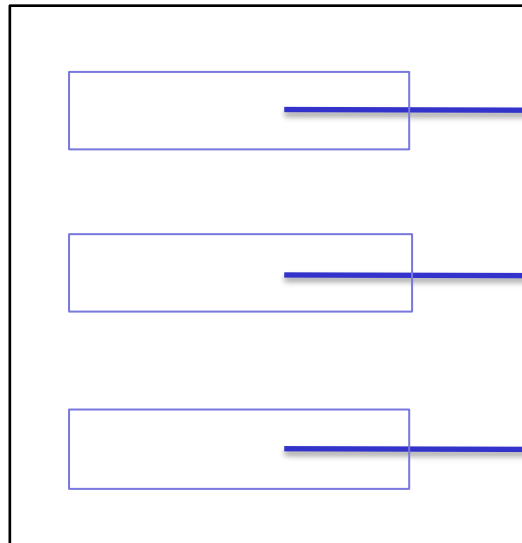
This is referred to as *object composition*. When one class is composed of objects of other classes!

S

name

dob

sid



instance of class Name

instance of class Date

"A00"

From Arrays to C-Structures...

```
struct Student {  
    char name[20];  
    char dob[6];  
    char sid[10];  
};
```

student.h

```
#include "student.h"
```

```
int main() {  
    Student s;
```

```
}
```

4001		name
4002		
4003		
4004		dob
4005		
4006		sid
5001		
...		
6001		

In the world of C/C++
you can declare the
physical structure to be
part of the Stack frame,

From Arrays to C-Structures...

```
struct Student {  
    char name[20];  
    char dob[6];  
    char sid[10];  
};
```

student.h

```
#include "student.h"
```

```
int main() {  
    Student s;
```

```
    printf("Enter a name: " );  
    scanf("s", s.name );
```

```
}
```



4001		<i>name</i>	{
4002			
4003			
4004			
4005		<i>dob</i>	{
4006			
		<i>sid</i>	{
5001			
...			
6001			

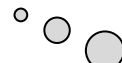
From Arrays to C-Structures...

```
struct Student {  
    char name[20];  
    char dob[6];  
    char sid[10];  
};
```

student.h

```
#include "student.h"
```

```
int main() {  
    Student s;  
  
    initialize(s);  
    output(s);  
}
```



What is being
passed?

4001		name
4002		
4003		
4004		dob
4005		
4006		sid
5001		
...		
6001		

Functions to work on objects

```
#include "student.h"
```

```
void initialize(struct Student s) {  
    printf("Enter a name: "); scanf("%s", s.name );  
    printf("Enter a dob: "); scanf("%s", s.dob );  
    printf("Enter a student id: "); scanf("%d", &s.sid );  
}
```

Declaring the parameter to be a Student structure.

```
void output(struct Student s) {  
    printf(("Name: %s, Dob: %s, sid: %s\n",  
           s.name, s.dob, s.sid ));  
}
```

```
int main() {  
    struct Student s1, s2;  
  
    initialize(s1); // call the function and pass the object to initialize  
  
    output(s1); // call the function and pass the object to output  
}
```

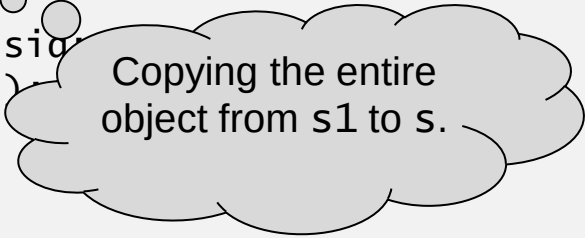
Functions to work on objects

```
#include "student.h"
```

```
void initialize(struct Student s) {  
    printf("Enter a name: "); scanf("%s", s.name );  
    printf("Enter a dob: "); scanf("%s", s.dob );  
    printf("Enter a student id: "); scanf("%s", s.id );  
}
```

```
void output(struct Student s) {  
    printf(("Name: %s, Dob: %s, sid: %s",  
           s.name, s.dob, s.sid ));  
}
```

```
int main() {  
    struct Student s1, s2;  
  
    initialize(s1); // call the function and pass the object to initialize  
  
    output(s1); // call the function and pass the object to output  
}
```



Copying the entire object from s1 to s.

Functions to work on objects

```
#include "student.h"

void initialize(struct Student s) {
    printf("Enter a name: " ); scanf("%s", s.name );
    printf("Enter a dob: " ); scanf("%s", s.dob );
    printf("Enter a student id: " ); scanf("%s", s.id );
}
```

```
void output(struct Student s) {
    printf(("Name: %s, Dob: %s, sid: %s",
           s.name, s.dob, s.sid ));
}
```

Will not output the
values that were
entered.

```
int main() {
    struct Student s1, s2;

    initialize(s1); // call the function and pass the object to initialize
    output(s1); // call the function and pass the object to output
}
```

Functions to work on objects

```
#include "student.h"
```

```
void initialize(struct Student *s) {
```

```
    printf("Enter a name: "); scanf("%s", s.name );
```

```
    printf("Enter a dob: "); scanf("%d", &s.dob );
```

```
    printf("Enter a student id: "); scanf("%d", &s.sid );
```

```
}
```

```
void output(struct Student s)
```

```
    printf(("Name: %s, Dob: %s, Sid: %d",  
           s.name, s.dob, s.sid ));
```

```
}
```

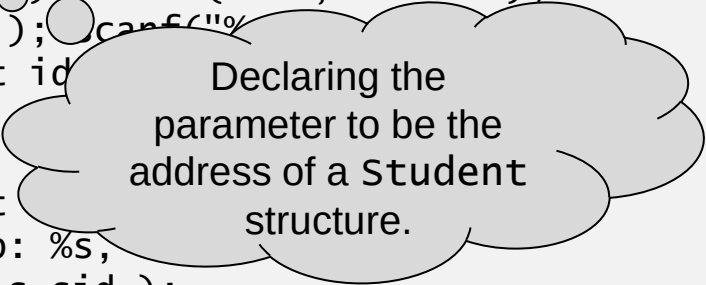
```
int main() {
```

```
    struct Student s1, s2;
```

```
    initialize(&s1); // call the function and pass the address of the  
                    // object to initialize
```

```
    output(s1); // call the function and pass the object to output
```

```
}
```



Declaring the
parameter to be the
address of a Student
structure.

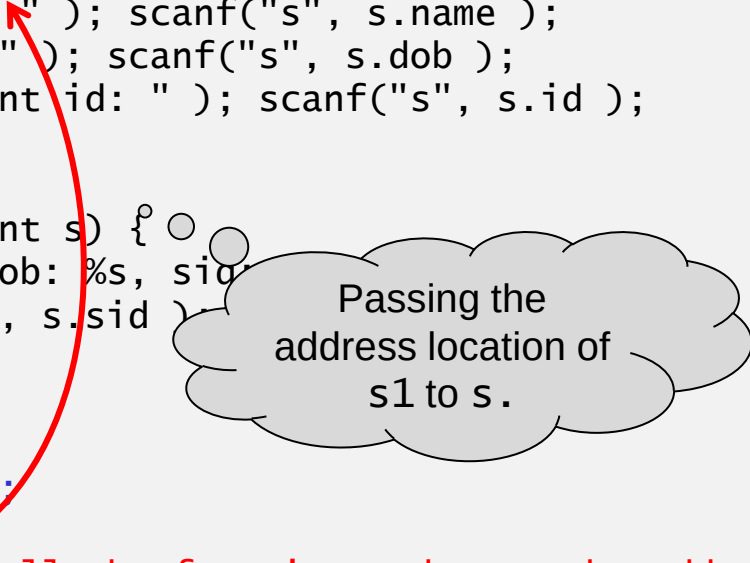
Functions to work on objects

```
#include "student.h"
```

```
void initialize(struct Student *s) {  
    printf("Enter a name: "); scanf("s", s.name );  
    printf("Enter a dob: "); scanf("s", s.dob );  
    printf("Enter a student id: "); scanf("s", s.id );  
}
```

```
void output(struct Student s) {  
    printf(("Name: %s, Dob: %s, sid: %s",  
           s.name, s.dob, s.sid ));  
}
```

```
int main() {  
    struct Student s1, s2;  
  
    initialize(&s1); // call the function and pass the address of the  
                    // object to initialize  
  
    output(s1); // call the function and pass the object to output  
}
```



Passing the
address location of
s1 to s.

Functions to work on objects

```
#include "student.h"

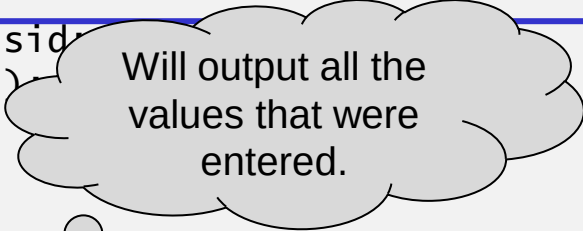
void initialize(struct Student *s) {
    printf("Enter a name: " ); scanf("s", s.name );
    printf("Enter a dob: " ); scanf("s", s.dob );
    printf("Enter a student id: " ); scanf("s", s.id );
}

void output(struct Student s) {
    printf(("Name: %s, Dob: %s, sid: %s\n",
           s.name, s.dob, s.sid ));
}

int main() {
    struct Student s1, s2;

    initialize(&s1); // call the function and pass the address of the
                    // object to initialize

    output(s1); // call the function and pass the object to output
}
```



Will output all the values that were entered.

From Arrays to C-Structures...

```
struct Student {  
    char name[20];  
    char dob[6];  
    char sid[10];  
};
```

student.h

```
#include "student.h"
```

```
int main() {  
    Student *s;  
}
```

4001	
4002	
4003	
4004	
4005	
4006	
5001	
...	
6001	

s

This declares a variable that will *point* to a student structure which will dynamically allocated at run-time.

From Arrays to C-Structures...

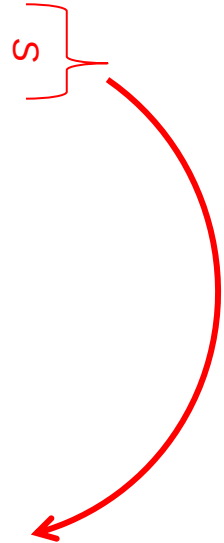
```
struct Student {  
    char name[20];  
    char dob[6];  
    char sid[10];  
};
```

student.h

```
#include "student.h"
```

```
int main() {  
    Student *s;  
    s = (Student *)  
        malloc(sizeof(Student));  
}
```

4001	6001
4002	
4003	
4004	
4005	
4006	
5001	
...	
6001	



Had to explicitly allocate the structure and assign its address to the reference variable s.

From Arrays to C-Structures...

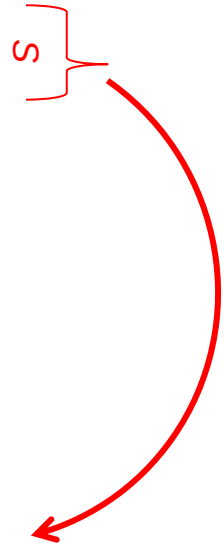
```
struct Student {  
    char name[20];  
    char dob[6];  
    char sid[10];  
};
```

student.h

```
#include "student.h"
```

```
int main() {  
    Student *s;  
    s = (Student *)  
        malloc(sizeof(Student));  
  
    inititalize(s); // pass the reference  
}
```

4001	6001
4002	
4003	
4004	
4005	
4006	
5001	
...	
6001	



From Arrays to C-Structures...

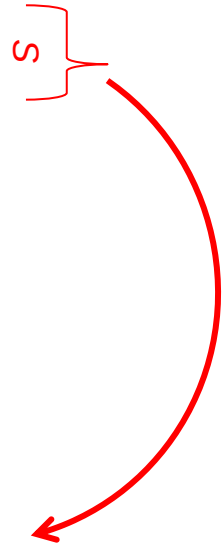
```
struct Student {  
    char name[20];  
    char dob[6];  
    char sid[10];  
};
```

student.h

```
#include "student.h"
```

```
int main() {  
    Student *s;  
    s = (Student *)  
        malloc(sizeof(Student));  
  
    initializs(*s); // pass the object  
}
```

4001	6001
4002	
4003	
4004	
4005	
4006	
5001	
...	
6001	



Functions to work on objects

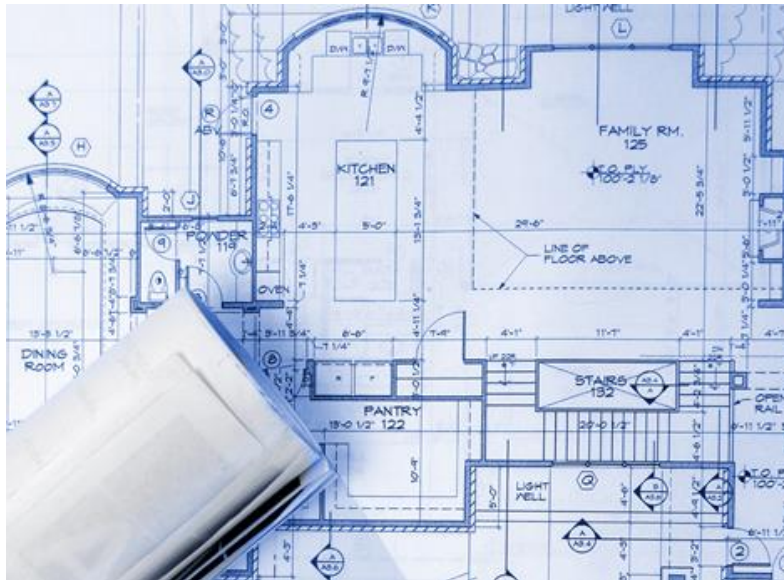
```
#include "student.h"

void initialize(struct Student *s) {
    printf("Enter a name: " ); scanf("s", s.name );
    printf("Enter a dob: " ); scanf("s", s.dob );
    printf("Enter a student id: " ); scanf("s", s.id );
}

void output(struct Student s) {
    printf(("Name: %s, Dob: %s, sid: %s\n"
           s.name, s.dob, s.sid );
}

int main() {
    struct Student s[100]; // An array of 100 student structures
    int i;

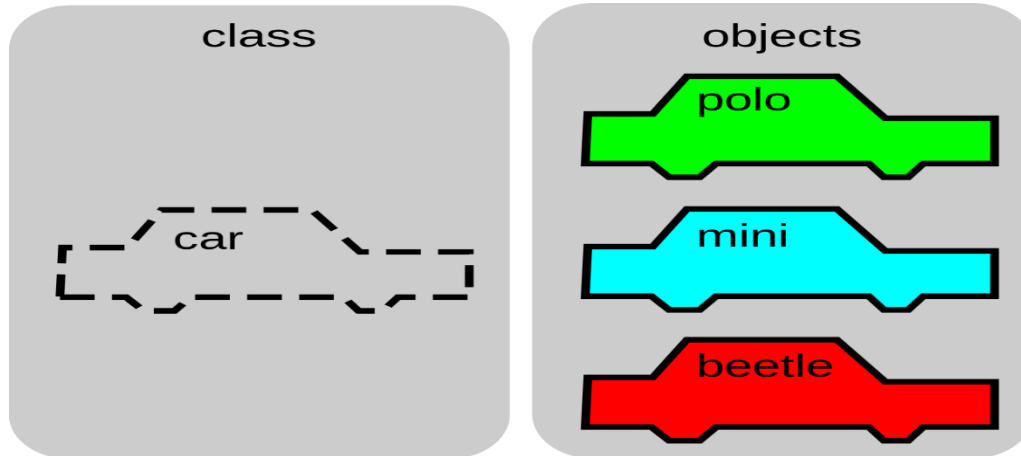
    for (i = 0; i < 100; i++ ) {
        initialize( s[i] );
        output( s[i] );
    }
}
```



From Structures to Objects:

Understanding Classes
and the
Object-Oriented Programming Paradigm

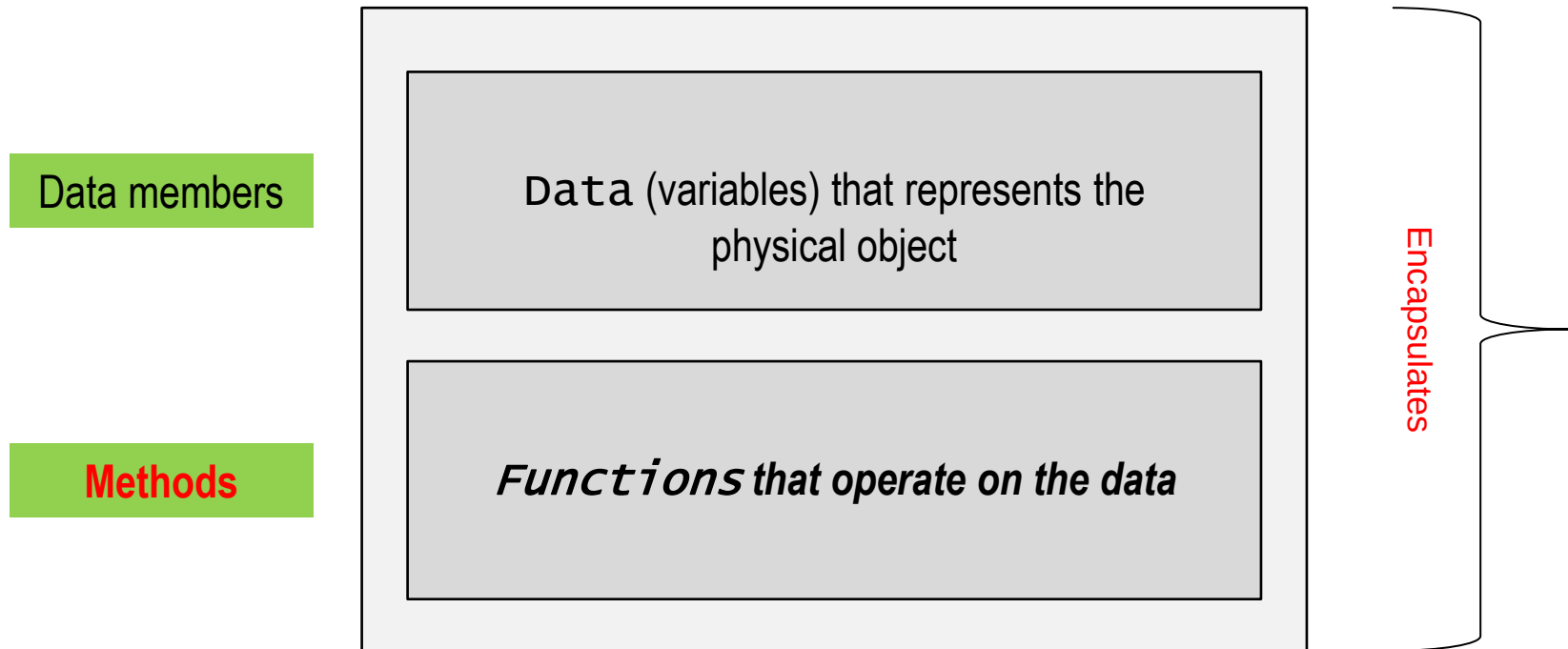
Why classes?



Classes allow us to specify a blueprint
that can be used to create a
physical structure of data
that *models* the logical entity it is
representing!

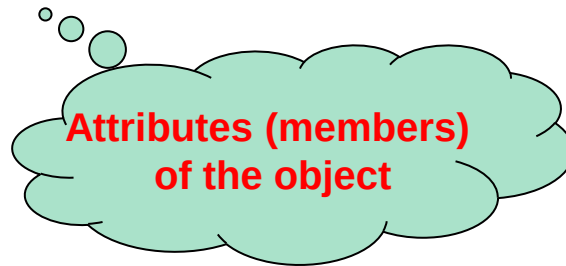
Each physical structure (or object)
created is an ***instance*** of the class!

From Structures to Classes

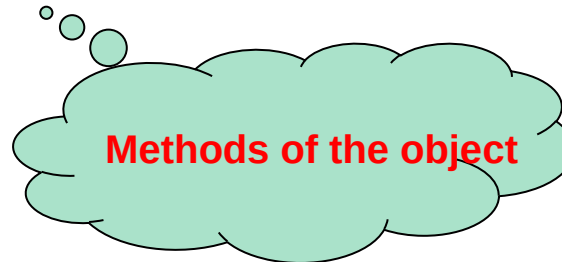


What Is An Object?

- An object is a construct that groups together:
 - one or more data values that describe an object



- one or more functions that operate on those data values



Why methods?

The purpose of adding functions or *methods* to a class is to provide a class with all the functionality that its' objects need to perform.

Java Methods

There are two types of methods
we can define for a class:

```
graph TD; A[There are two types of methods we can define for a class:] --> B[Static]; A --> C[non Static];
```

Static

non Static

Java Methods

There are two types of methods
we can define for a class:

```
graph TD; A[There are two types of methods we can define for a class:] --> B[A static method can be called on the class, but NOT on an instance of the class. Math.pow(8, 3)]; A --> C[non Static];
```

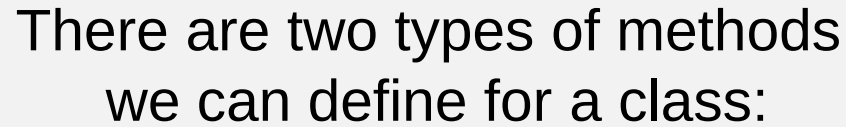
A **static** method can be called
on the class, but **NOT** on
an instance of the class.

`Math.pow(8, 3)`

non Static

Java Methods

There are two types of methods
we can define for a class:



```
graph TD; A[There are two types of methods we can define for a class:] --> B[A static method can be called on the class, but NOT on an instance of the class.]; A --> C[non Static];
```

A **static** method can be called on the class, but **NOT** on an instance of the class.

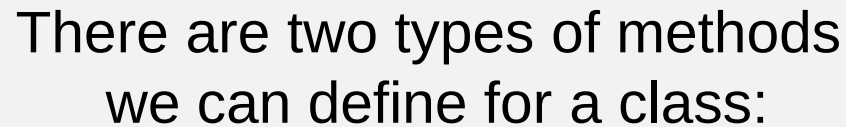
```
int[] arr = {1,2,3};  
Arrays.toString(arr)
```

If the static method needs access to a specific object, then the object must be explicitly passed to the method as an input parameter.

non Static

Java Methods

There are two types of methods
we can define for a class:



```
graph TD; A[There are two types of methods we can define for a class:] --> B[A static method can be called on the class, but NOT on an instance of the class.]; A --> C[A non static method must be called on an instance of the class.];
```

A static method can be called
on the class, but NOT on
an instance of the class.

`className.method(object)`

If the static method needs access
to a specific object, then the object
must be explicitly passed to the
method as an input parameter.

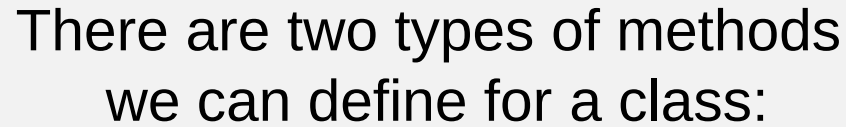
A **non static** method must be
called on an *instance* of the class.

`object.method()`

As the method is being called on
an object, the method has
direct access to all the data and
methods of the class.

Java Methods

There are two types of methods
we can define for a class:



```
graph TD; A[There are two types of methods we can define for a class:] --> B[A static method can be called on the class, but NOT on an instance of the class.]; A --> C[A non static method must be called on an instance of the class.];
```

A static method can be called
on the class, but NOT on
an instance of the class.

`className.method(object)`

If the static method needs access
to a specific object, then the object
must be explicitly passed to the
method as an input parameter.

A **non static** method must be
called on an *instance* of the class.

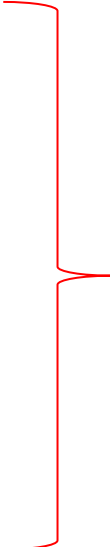
```
String str = "Hello!";  
int len = str.length()
```

As the method is being called on
an object, the method has
direct access to all the data and
methods of the class.

Designing a Custom Class

- What's in a Name?
- Example:
 - Ms. Gremelda Lyons
 - Mr. John Reynolds III
 - Dr. Michael Carepenter

- **First Name** // String
- **Last Name** // String
- **Middle Initial** // String *or char*
- **Prefix** // String
- **Suffix** // String
- **Nick Name** // String



Attributes
of a *name*

Designing a Custom Class

- What's in a Date?
- Example:
 - 02/25/1962
 - 03/02/1996
 - 10/04/1999

- **month** // int
- **day** // int
- **year** // int

Attributes
of a date

Class Definition:

a user defined custom datatype

```
public class Date {
```

```
    int month;  
    int day;  
    int year;
```

Attributes
of the class

```
}
```

```
public class testDate {
```

```
    public static void main( String[] args ) {  
        Date bday = new Date();  
        bday.month = 2;  
        bday.day = 25;  
        bday.year = 1962;  
    }
```

```
}
```

Class Definition:

a user defined custom datatype

```
public class Date {
```

```
    int month;  
    int day;  
    int year;
```

```
}
```

Attributes
of class

How can we ensure that
applications or *clients* who
create instances of our
class use them correctly?

```
public class testDate {
```

```
    public static void main( String[] args ) {  
        Date bday = new Date();  
        bday.month = 13;           // Invalid Data  
        bday.day = 25;  
        bday.year = 1962;
```

```
    }
```

```
}
```

Class Definition:

a user defined custom datatype

```
public class Date {
```

```
    int month;  
    int day;  
    int year;
```

Attributes
of the class

```
}
```

```
public class testDate {
```

```
    public static void main()  
    {  
        Date bday = new Date();  
        bday.month = 13;  
        bday.day = 25;  
        bday.year = 1962;  
    }
```

Have the class determine
exactly what information
client programs can
access!

```
}
```

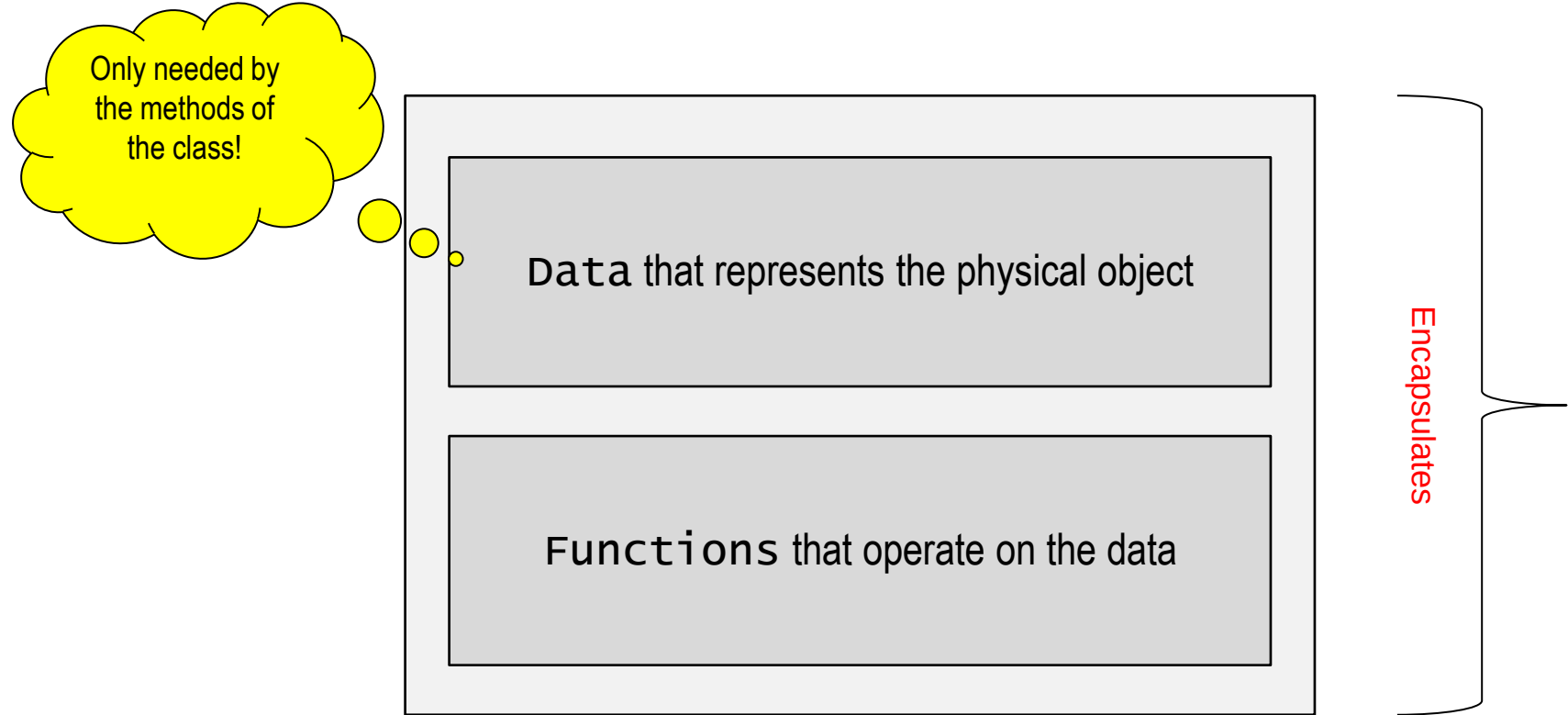
Classes and Data and Function Encapsulation

Only needed by
the methods of
the class!

Data that represents the physical object

Functions that operate on the data

Encapsulates



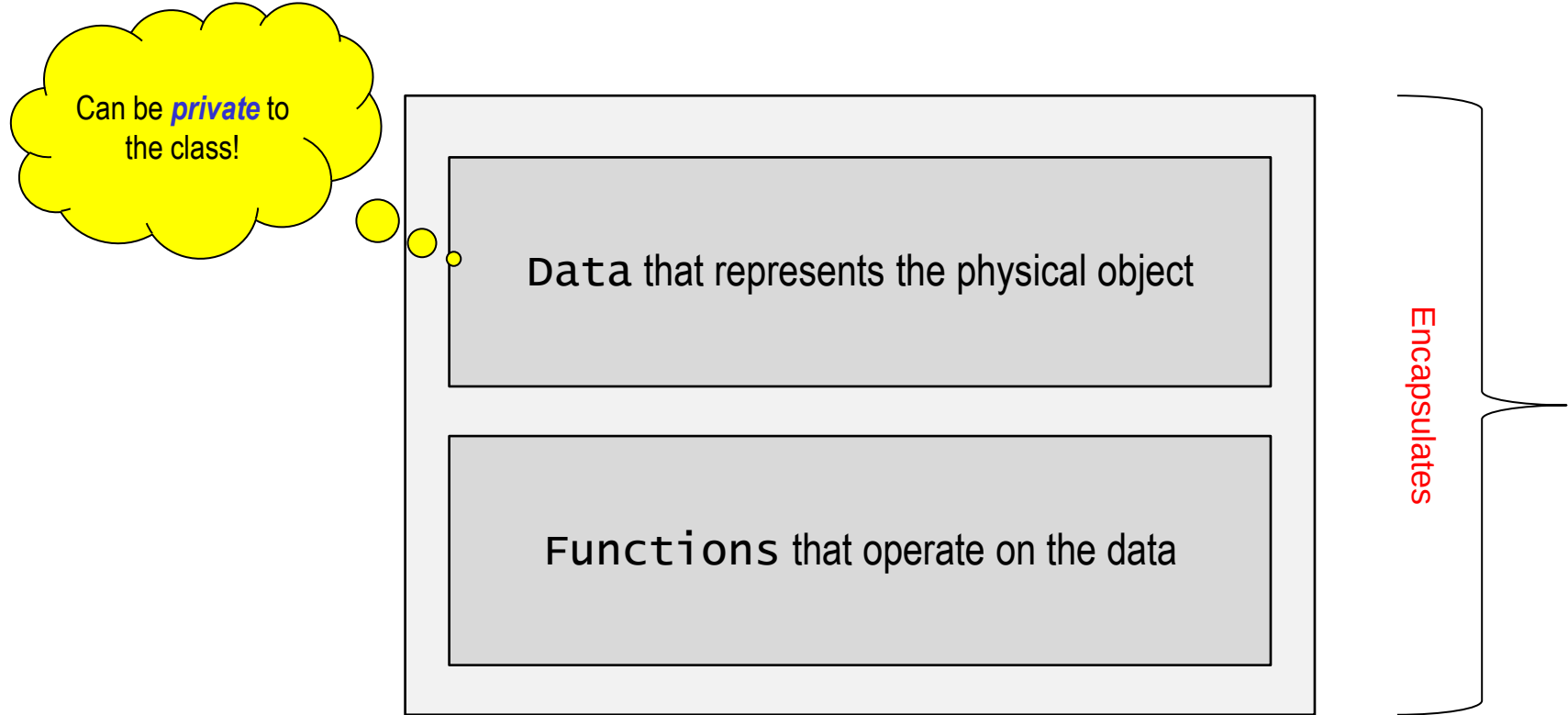
Classes and Data and Function Encapsulation

Can be *private* to the class!

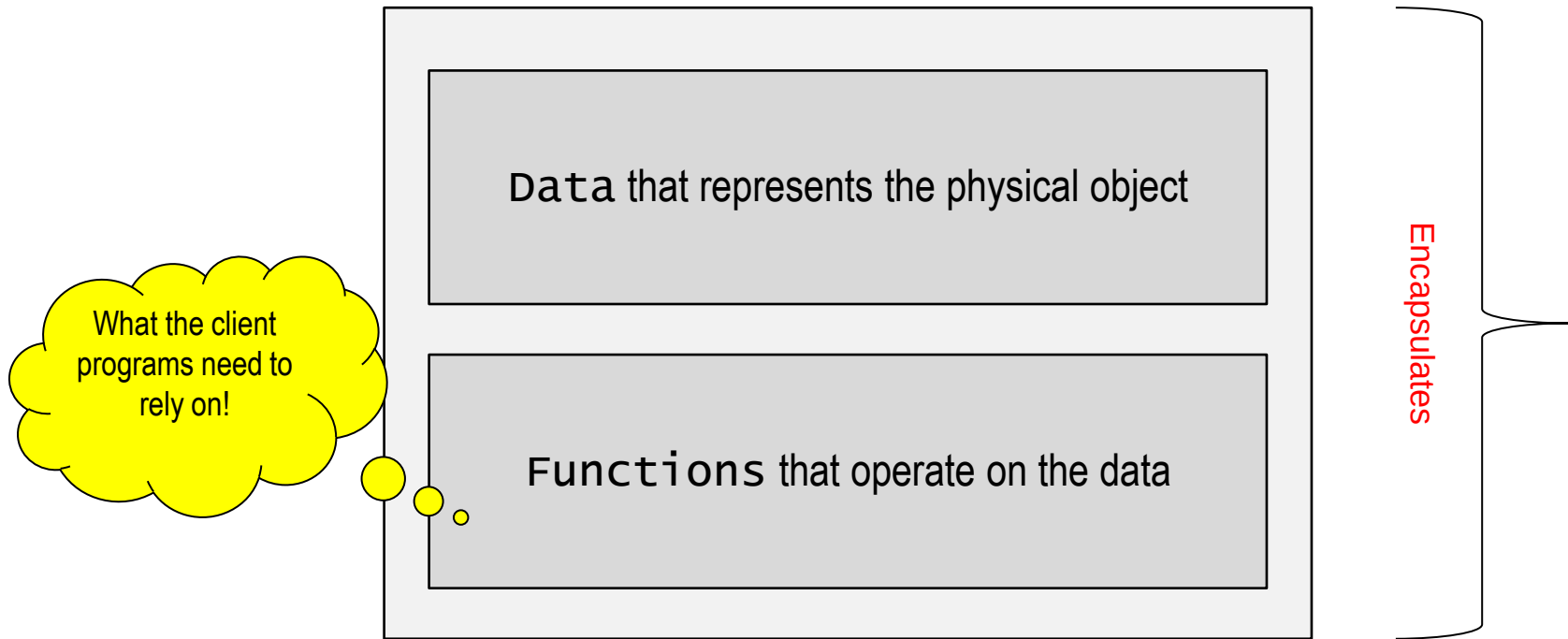
Data that represents the physical object

Functions that operate on the data

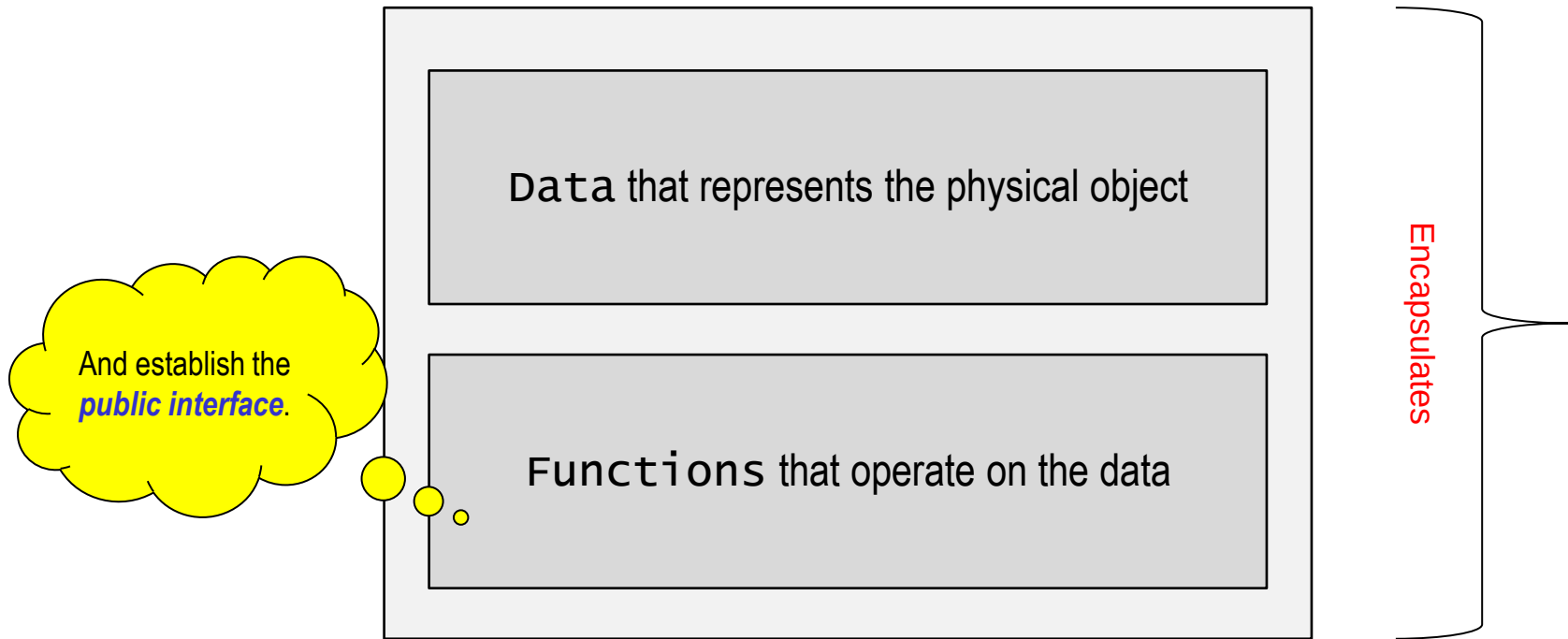
Encapsulates



Classes and Data and Function Encapsulation



Classes and Data and Function Encapsulation



Classes with a **Domain** Specific Interface

C++ and Java

```
class className
```

```
  private:
```



```
  public:
```

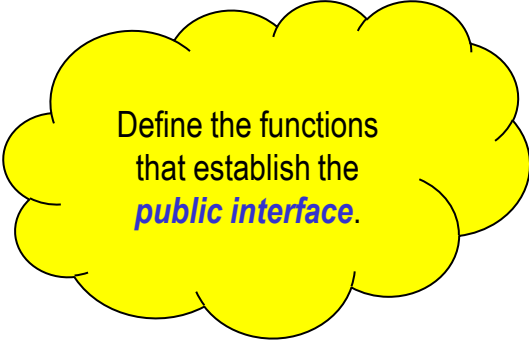
Declare the data
members that define
the **physical object**.

Classes with a **Domain** Specific Interface

C++ and Java

```
class className  
private
```

public:



Define the functions
that establish the
public interface.

Class Definition:

a user defined custom datatype

```
public class Date {
```

```
    private int month;  
    private int day;  
    private int year;
```

Attributes
of the class

```
}
```

```
public class testDate {
```

```
    public static void main()  
    {  
        Date bday = new Date();  
        bday.month = 13;  
        bday.day = 25;  
        bday.year = 1962;  
    }
```

```
}
```

Have the class determine
exactly what information
client programs can
access!

Class Definition:

a user defined custom datatype

```
public class Date {
```

```
    private int month;  
    private int day;  
    private int year;
```

Attributes
of the class

```
}
```

```
public class testDate {
```

```
    public static void main( String[] args ) {  
        Date bday = new Date()  
        bday.month = 2;  
        bday.day = 25;  
        bday.year = 1962;  
    }
```



```
}
```

Class Definition:

a user defined custom datatype

```
public class Date {
```

```
private int month;  
private int day;  
private int year;
```

The class defines the
public interface!

```
public boolean isHoliday() { ... }
```

```
public int calculateAge() { ... }
```

```
public int daysUntil( Date someDate ) { ... }
```

```
public String formatDate() { ... }
```

```
public boolean equals( Date someDate ) { .. }
```

```
}
```

Behaviors
of the class

*Functions
that each
object can
perform!*

Class Definition:

a user defined custom datatype

```
public class Date {
```

```
    private int month;  
    private int day;  
    private int year;
```

```
    public boolean isHoliday() { ... }
```

```
    ...
```

```
}
```

```
public class testDate {
```

```
    public static void main( String[] args ) {  
        Date bday = new Date();  
        // can only access public items  
        // need to assign the date by calling  
        // a public method of the class!  
    }
```

```
}
```

Class Definition:

a user defined custom datatype

```
public class Date {
```

```
    private int month;  
    private int day;  
    private int year;
```

```
    public boolean isHoliday() { ... }  
    public boolean setDate(int m, int d, int y) {...}  
    ...
```

```
}
```

```
public class testDate {
```

```
    public static void main( String[] args ) {  
        Date bday = new Date();  
        if ( !bday.setDate( 02, 25, 1962 ) )  
            // throw an exception...
```

```
    }
```

```
}
```


Class Definition:

a user defined custom datatype

```
public class Date {
```

```
    private int month;  
    private int day;  
    private int year;
```

```
    public boolean isHoliday() { ... }  
    public boolean setDate(int m, int d, int y) {...}  
    ...
```

```
}
```

```
public class testDate {
```

```
    public static void main( String[] args ) {  
        Date bday = new Date();  
        if ( !bday.setDate( 02, 30, 1962 ) )  
            // throw an exception...
```

```
    }
```

```
}
```

Class Definition:

a user defined custom datatype

```
public class Date {
```

```
    private int month;  
    private int day;  
    private int year;
```

```
    public boolean isHoliday() {  
    public boolean setDate(  
    ...  
}
```

```
public class test {
```

```
    public static void main(  
        Date bday = new Date(  
        if ( !bday.setDate(  
            // throw an exception...  
    }  
}
```

*Making the data attributes private **encapsulates** the data within the object and we can prevent the data from being accessed by any external code.*

Class Definition:

a user defined custom datatype

```
public class Date {
```

```
    private int month;  
    private int day;  
    private int year;
```

```
    public boolean isLeapYear()  
    public boolean isDateValid()  
    ...  
}
```

*But what if I want to create
an instance of Date with
specific initial values,
should I always have to call
the setDate method?*

```
public class testDate {
```

```
    public static void main( String[] args ) {  
        Date bday = new Date();  
        if ( !bday.setDate( 02, 25, 1962 ) )  
            // throw an exception...  
    }  
}
```

Class Definition:

a user defined custom datatype

```
public class Date
```

```
private int month;  
private int day;  
private int year;
```

```
public boolean isLeapYear()  
public boolean setDate(int month, int day, int year)  
...
```

```
}
```

*Class can provide special methods called **constructors** which can assign initial values at object creation!*

```
public class testDate {
```

```
public static void main( String[] args ) {  
    Date bday = new Date(2, 25, 1962);  
}
```

```
}
```

Class Definition:

a user defined custom datatype

```
public class Date {
```

```
    private int month;  
    private int day;  
    private int year;
```

```
    public boolean is  
    public boolean setDa  
    ...
```

```
}
```

*And constructors can also
ensure that our objects are
only initialized with valid
data!*

```
public class testDate {
```

```
    public static void main( String[] args ) {  
        Date bday = new Date(2, 30, 1962);
```

```
    }
```

```
}
```

Class Definition

```
public class className {
```

```
    private instance members    // object scope
```

Attributes of
the object

Instance Methods of the class

```
    public className() { ... };
```

```
    public datatype setMethod() { ... };
```

```
    public datatype getMethod() { ... };
```

```
    // ... methods to print, compare, etc.
```

Behaviors of
the object

*Functions that
each object
can perform!*

```
}
```

Class Definition

```
public class className {
```

```
    private instance members    // object scope
```

Instance Methods of the class

```
    public className() { ... };
```

```
    public datatype setMethod() { ... };
```

```
    public datatype getMethod() { ... };
```

```
    // ... methods to print, compare, etc.
```

```
}
```

**Access
modifiers**

*Allows the
class to
control
access of
data
members and
methods.*

Class Definition

```
public class className {
```

```
    public static variables    // class scope
```

Attributes of
the class.

```
    private instance members  // object scope
```

Instance Methods of the class

```
    public className() { ... };
```

```
    public datatype setMethod() { ... };
```

```
    public datatype getMethod() { ... };
```

```
    // ... methods to print, compare, etc.
```

Static Methods of the class

```
}    // methods called at the class level
```

Methods of
the class.

Class Definition:

a user defined custom datatype

```
public class Date {
```

```
    private int month;  
    private int day;  
    private int year;
```

```
    public String getMonth() {  
        String monthOfYear; // variable for month as string  
        if ( month == 1 )  
            monthOfYear = "January";  
        else  
            if ( monthOfYear == 2 )  
                monthOfYear = "February";  
        else  
            ...  
  
        return( monthOfYear );  
    }
```

```
}
```

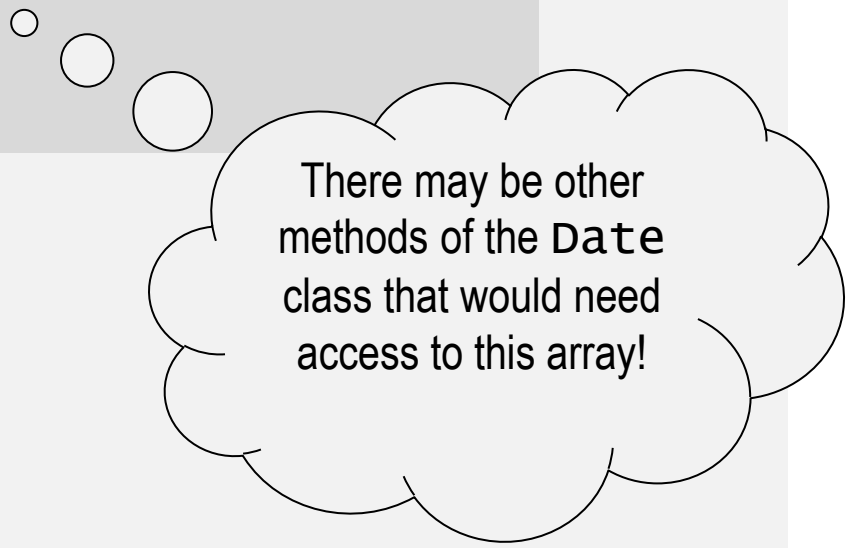
Class Definition:

a user defined custom datatype

```
public class Date {
```

```
    private int month;  
    private int day;  
    private int year;
```

```
    public String getMonth() {  
        String[] months = { "January", "February" ... };  
  
        return( months[month-1] );  
    }
```



There may be other methods of the **Date** class that would need access to this array!

```
}
```

Class Definition:

a user defined custom datatype

```
public class Date {
```

```
    private static final String[] months =  
        {"January", "February", ..., "November", "December"};
```

```
    private int month;  
    private int day;  
    private int year;
```

```
    public String formatDate() {  
        String formattedDate =  
            months[month-1] + " " + day + "," + " " + year;  
  
        return( formattedDate );  
    }
```

```
    public String getMonth() {  
  
        return( months[month-1] );  
    }
```

```
}
```

Class Definition:

a user defined custom datatype

```
public class Date {
```

```
    private static final String[] months =  
        {"January", "February", "March", "April", "May", "June", "July", "August", "September", "October", "November", "December"};
```

```
    private int month;  
    private int day;  
    private int year;
```

```
    public String formatMonth()  
    {  
        String format = "%d %s %d";  
        return( format.  
            format( day, months[month-1], year ) );  
    }
```

```
public class clientDate {  
    public static void main(String[] s) {  
  
        Date d = new Date();  
  
        System.out.println(d.getMonth(2));  
    }  
}
```

```
    public String getMonth(int m) {  
  
        return( months[m-1] );  
    }  
}
```

Class Definition:

a user defined custom datatype

```
public class Date {
```

```
    private static final String[] months =  
        {"January", "February", "March", "April", "May", "June", "July", "August", "September", "October", "November", "December"};
```

```
    private int month;  
    private int day;  
    private int year;
```

```
    public String format()  
    {  
        String format = "%d/%d/%d";  
        return( format.  
            format( day, month, year ) );  
    }
```

```
public class clientDate {  
    public static void main(String[] s) {  
  
        System.out.println(Date.getMonth(2));  
    }  
}
```



```
    public static String getMonth(int m) {  
  
        return( months[m-1] );  
    }  
}
```

Class Definition:

a user defined custom datatype

```
public class Date {
```

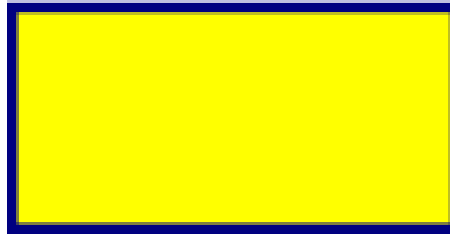
```
    private static final String[] months =  
        {"January", "February", "March", "April", "May", "June", "July", "August", "September", "October", "November", "December"};
```

```
    private int month;  
    private int day;  
    private int year;
```

```
    public String format()  
    {  
        String format = "%d/%d/%d";  
        return( format );  
    }
```

```
public class clientDate {  
    public static void main(String[] s) {  
  
        System.out.println(Date.getMonth(22));  
    }  
}
```

```
    public static String getMonth(int m) {  
        // error check 1 <= m <= 12  
        return( months[m-1] );  
    }  
}
```



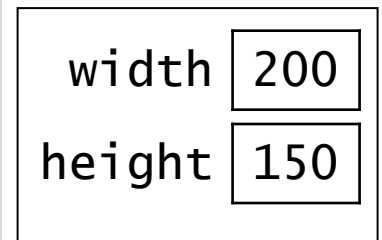
Rectangle:
a custom data types

Computer Science OOD
Boston University

Christine Papadakis-Kanaris

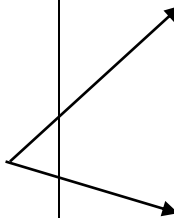
Example: A Rectangle Class

- Let's say that we want to create a data type for objects that represent rectangles.
- Every Rectangle object should have two variables inside it (*width* and *height*) for the rectangle's dimensions.
 - these variables are referred to as *fields*
 - also known as: attributes, instance variables
- We'll also put functions/methods inside the object.



Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
    public Rectangle(int dim) {  
        width = height = dim;  
    }  
  
    .  
    .  
    .  
    .  
  
}
```



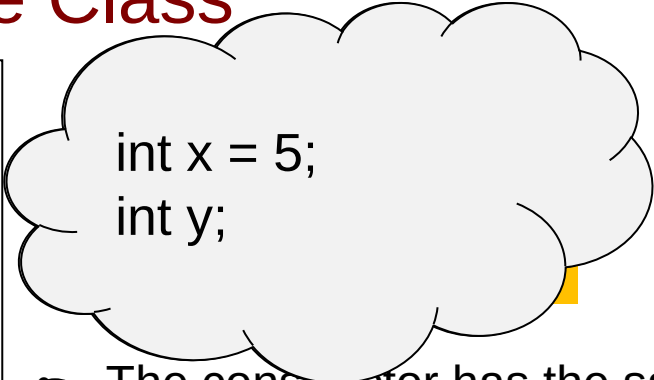
Constructor

- The constructor has the same name as the class.
 - it is non-static
 - it has no return type
- The purpose of the constructor is to initialize the members.
- Constructors can be **overloaded**.
- A constructor that defines no parameters is referred to as the no-arg constructor.
- If a class does not define **any** constructors, Java will provide a default no-arg constructor for the class.

Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
    public Rectangle(int dim) {  
        width = height = dim;  
    }  
    public Rectangle() {  
        width = height = 0;  
    }  
}
```

•
•
•
•



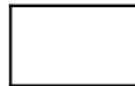
int x = 5;
int y;

- The constructor has the same name as the class.
 - it is non-static
 - it has no return type
- The purpose of the constructor is to initialize the members.
- Constructors can be overloaded.
- A constructor that defines no parameters is referred to as the a no-arg constructor.
- If a class does not define **any** constructors, Java will provide a **default** no-arg constructor for the class.

Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
  
    public Rectangle(int dim) {  
        width = height = dim;  
    }  
  
    public Rectangle() {  
        width = height = 0;  
    }  
}
```

r1 

r2 

width	
height	

```
public static void main( String [] args ) {
```

```
    Rectangle r1 = new Rectangle();  
    Rectangle r2 = new Rectangle(5, 10);
```

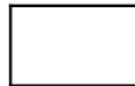
```
}
```

```
}
```

Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
  
    public Rectangle(int dim) {  
        width = height = dim;  
    }  
  
    public Rectangle() {  
        width = height = 0;  
    }  
    .  
    .  
    .  
}
```

r1 

r2 

width	
height	

```
public static void main( String [] args ) {
```

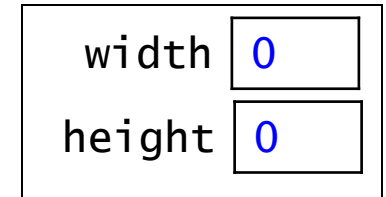
```
    Rectangle r1 = new Rectangle();  
    Rectangle r2 = new Rectangle(5, 10);
```

```
}
```

```
}
```

Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
    public Rectangle(int dim) {  
        width = height = dim;  
    }  
    public Rectangle() {  
        width = height = 0;  
    }  
    .  
    .  
    .  
}
```



How do we know that width and height are the members of the object we want initialized?

```
public static void main( String [] args ) {  
  
    Rectangle r1 = new Rectangle();  
    Rectangle r2 = new Rectangle(5, 10);  
}
```

```
}
```

Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
  
    public Rectangle(int dim) {  
        width = height = dim;  
    }  
  
    public Rectangle() {  
        width = height = 0;  
    }  
    .  
    .  
    .  
}
```

r1 

width	0
height	0

Implicit to every
instance (non-static) method
is the **this** parameter!

```
public static void main( String [] args ) {  
  
    Rectangle r1 = new Rectangle();  
    Rectangle r2 = new Rectangle(5, 10);  
}
```

}

Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
  
    public Rectangle(int dim) {  
        width = height = dim;  
    }  
  
    public Rectangle() {  
        width = height = 0;  
    }  
    .  
    .  
    .  
}
```

r1 

width	0
height	0

r2

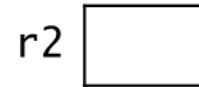
*The **this** parameter contains the address location of the object the method was called on.*

```
public static void main( String [] args ) {  
  
    Rectangle r1 = new Rectangle();  
    Rectangle r2 = new Rectangle(5, 10);  
}
```

}

Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
  
    public Rectangle(int dim) {  
        width = height = dim;  
    }  
  
    public Rectangle() {  
        this.width = this.height = 0;  
    }  
    .  
    .  
    .  
}
```



width	0
height	0

```
public static void main( String [] args ) {
```

```
    Rectangle r1 = new Rectangle();  
    Rectangle r2 = new Rectangle(5, 10);
```

```
}
```

```
}
```


Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
  
    public Rectangle(int dim) {  
        width = height = dim;  
    }  
  
    public Rectangle() {  
        this.width = this.height = 0;  
    }  
    .  
    .  
    .  
}
```

r1 

width	0
height	0

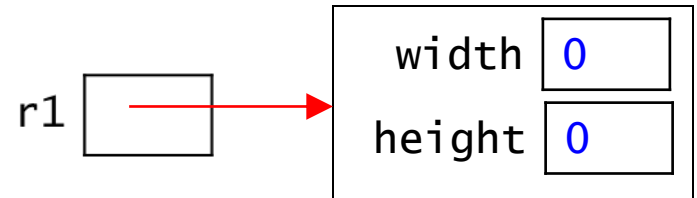
*Note that this call is
part of an assignment
statement.*

```
public static void main( String [] args ) {  
  
    Rectangle r1 = new Rectangle();  
    Rectangle r2 = new Rectangle(5, 10);  
}
```

}

Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
  
    public Rectangle(int dim) {  
        width = height = dim;  
    }  
  
    public Rectangle() {  
        this.width = this.height = 0;  
    }  
    .  
    .  
    .  
}
```



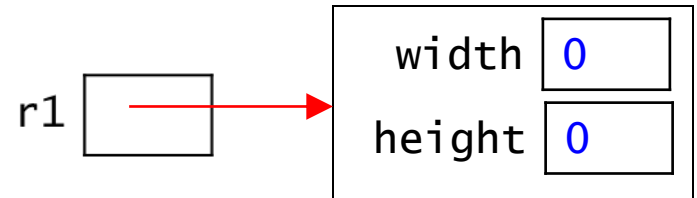
Constructors return the address location of the object constructed via the *this* parameter!

```
public static void main( String [] args ) {  
  
    Rectangle r1 = new Rectangle();  
    Rectangle r2 = new Rectangle(5, 10);  
}
```

```
}
```

Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
  
    public Rectangle(int dim) {  
        width = height = dim;  
    }  
  
    public Rectangle() {  
        this.width = this.height = 0;  
    }  
    .  
    .  
    .  
}
```



This is why constructors cannot be declared to be void methods!

```
public static void main( String [] args ) {  
  
    Rectangle r1 = new Rectangle();  
    Rectangle r2 = new Rectangle(5, 10);  
}
```

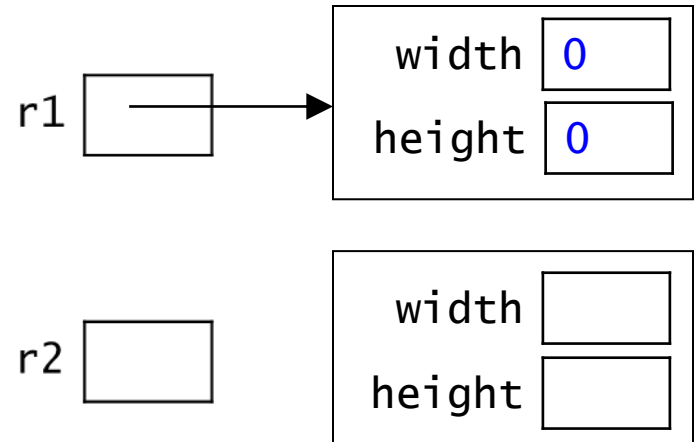
```
}
```

Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
  
    public Rectangle(int dim) {  
        width = height = dim;  
    }  
  
    public Rectangle() {  
        width = height = 0;  
    }  
    .  
    .  
    .  
}
```

```
public static void main( String [] args ) {
```

```
    Rectangle r1 = new Rectangle();  
    Rectangle r2 = new Rectangle(5, 10);  
}
```

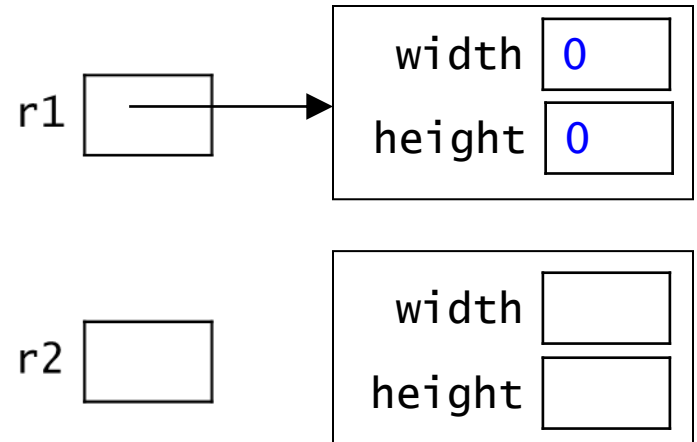


```
}
```

Sample Rectangle Class

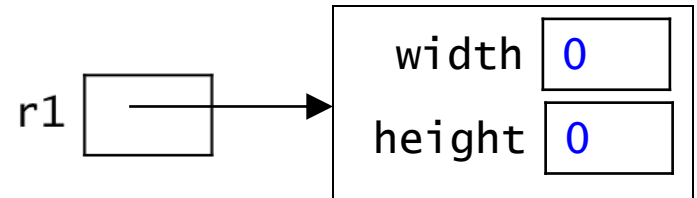
```
public class Rectangle {  
    private int width;  
    private int height;  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
    public Rectangle(int dim) {  
        width = height = dim;  
    }  
    public Rectangle() {  
        width = height = 0;  
    }  
    .  
    .  
    .
```

```
public static void main( String [] args ) {  
    Rectangle r1 = new Rectangle();  
    Rectangle r2 = new Rectangle(5, 10);  
}
```



Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
    public Rectangle(int dim) {  
        width = height = dim;  
    }  
    public Rectangle() {  
        width = height = 0;  
    }  
    .  
    .  
    .  
}
```



Do we need to use
the *this* reference to
access the data members?

```
public static void main( String [] args ) {  
    Rectangle r1 = new Rectangle();  
    Rectangle r2 = new Rectangle(5, 10);  
}
```

}

Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;
```

```
    public Rectangle(int width, int height) {  
        width = width;  
        height = height;  
    }
```

```
    public Rectangle(int dim) {  
        width = height = dim;  
    }
```

```
    public Rectangle() {  
        width = height = 0;  
    }
```

```
    .  
    .  
    .
```

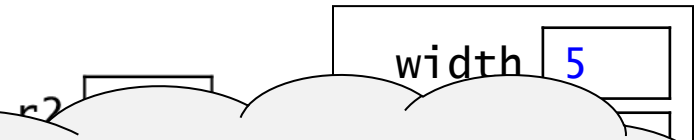
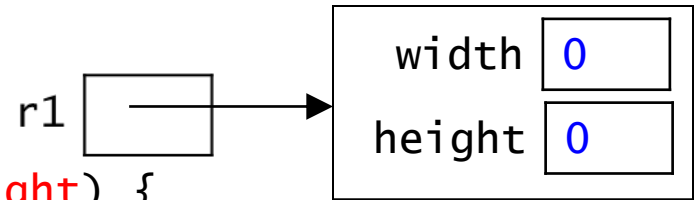
```
    public static void main( String [] args ) {
```

```
        Rectangle r1 = new Rectangle();
```

```
        Rectangle r2 = new Rectangle(5, 10);
```

```
    }
```

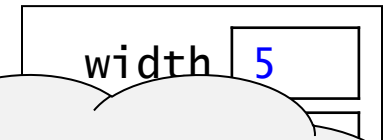
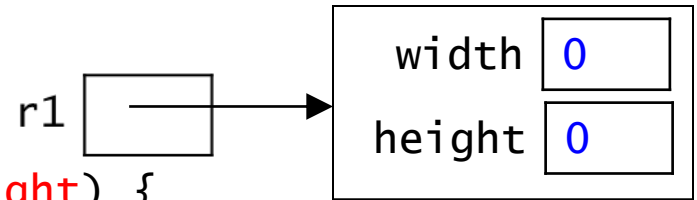
```
}
```



What if we give the parameters the same identifier name as the data members?

Sample Rectangle Class

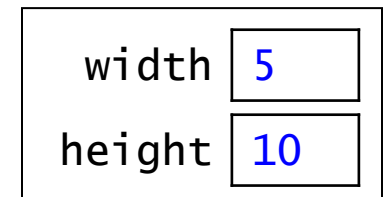
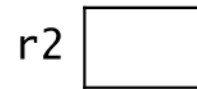
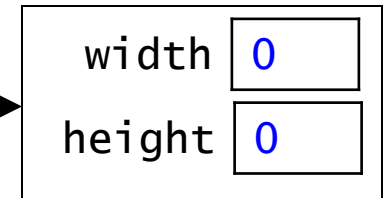
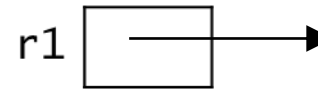
```
public class Rectangle {  
    private int width;  
    private int height;  
    public Rectangle(int width, int height) {  
        width = width;  
        height = height;  
    }  
    public Rectangle(int dim) {  
        width = height = dim;  
    }  
    public Rectangle() {  
        width = height = 0;  
    }  
    .  
    .  
    .  
    public static void main( String [] args ) {  
        Rectangle r1 = new Rectangle();  
        Rectangle r2 = new Rectangle(5, 10);  
    }  
}
```



Now we have a scope issue!

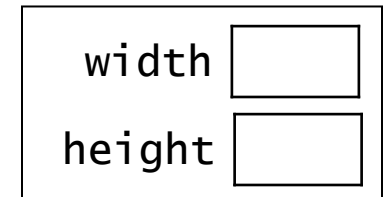
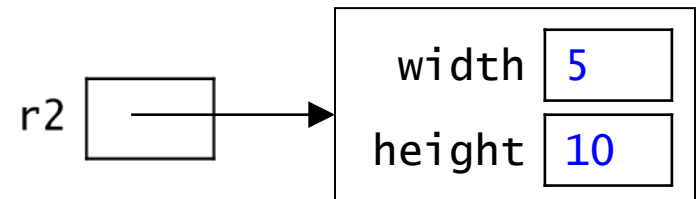
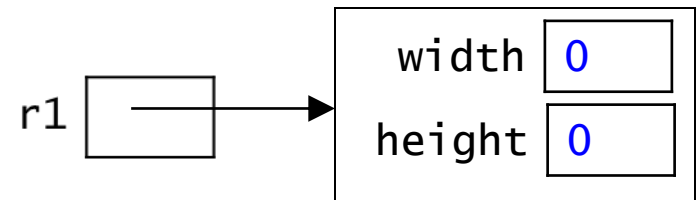
Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
    public Rectangle(int width, int height) {  
        this.width = width;  
        this.height = height;  
    }  
    public Rectangle(int dim) {  
        width = height = dim;  
    }  
    public Rectangle() {  
        width = height = 0;  
    }  
    .  
    .  
    .  
    public static void main( String [] args ) {  
        Rectangle r1 = new Rectangle();  
        Rectangle r2 = new Rectangle(5, 10);  
    }  
}
```



Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
    public Rectangle(int dim) {  
        width = height = dim;  
    }  
    public Rectangle() {  
        width = height = 0;  
    }  
    .  
    .  
    .  
}
```

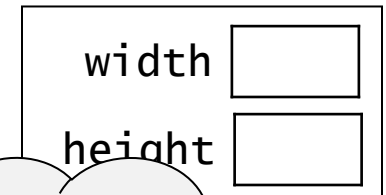
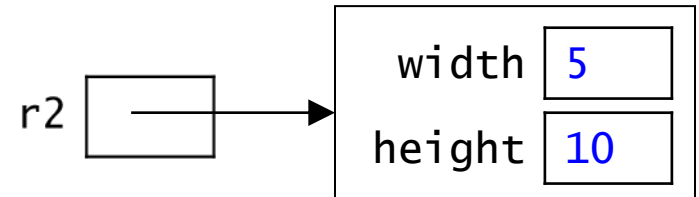
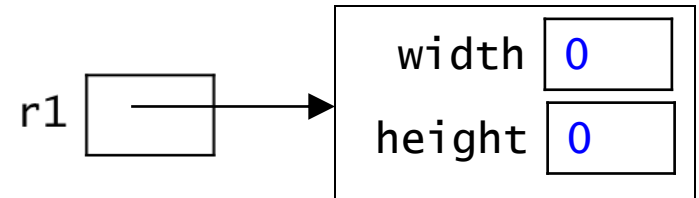


```
public static void main( String [] args ) {  
  
    Rectangle r1 = new Rectangle();  
    Rectangle r2 = new Rectangle(5, 10);  
    Rectangle r3 = new Rectangle(7);  
}
```

```
}
```

Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
    public Rectangle(int dim) {  
        width = height = dim;  
    }  
    public Rectangle() {  
        width = height = 0;  
    }  
    .  
    .  
    .  
    public static void main( String  
        Rectangle r1 = new Rectangle(  
        Rectangle r2 = new Rectangle(5, 10,  
        Rectangle r3 = new Rectangle(7);  
    }  
}
```



Note that both
constructors are
doing the
same thing.

Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;
```

```
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }
```

```
    public Rectangle(int dim) {  
        this(dim, dim);
```

```
    }  
    public Rectangle() {  
        width = height = 0;  
    }
```

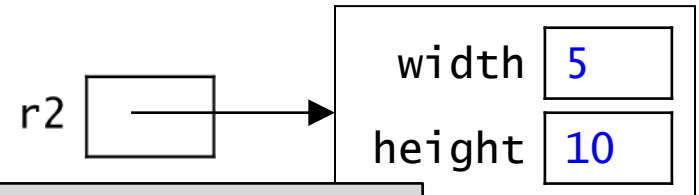
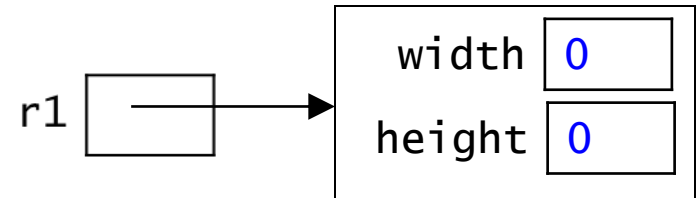
```
    .  
    .  
    .
```

```
    public static void main( String [] args ) {
```

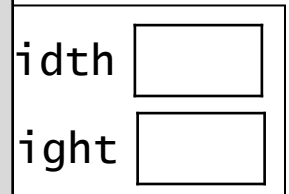
```
        Rectangle r1 = new Rectangle();  
        Rectangle r2 = new Rectangle(5, 10);  
        Rectangle r3 = new Rectangle(7);
```

```
    }
```

```
}
```



Constructors can call
other constructors of the
same class by using
the *this* as the call.



Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;
```

```
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }
```

```
    public Rectangle(int dim) {  
        this(dim, dim);
```

```
    }  
    public Rectangle() {  
        width = height = 0;  
    }
```

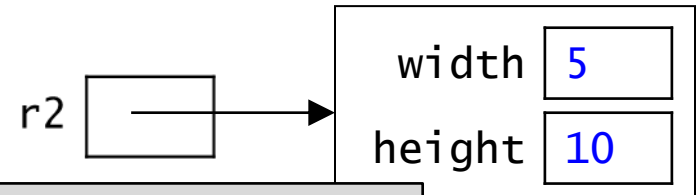
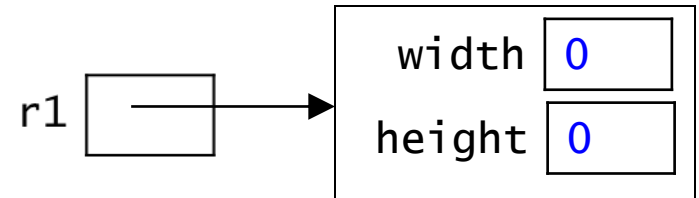
```
    .  
    .  
    .
```

```
    public static void main( String [] args ) {
```

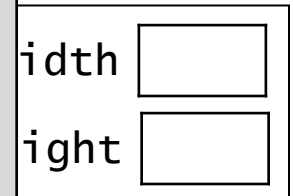
```
        Rectangle r1 = new Rectangle();  
        Rectangle r2 = new Rectangle(5, 10);  
        Rectangle r3 = new Rectangle(7);
```

```
    }
```

```
}
```



The *this* call to another constructor must be the first call in the method.



Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;
```

```
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }
```

```
    public Rectangle(int dim) {  
        this(dim, dim);  
    }
```

```
    public Rectangle() {  
        width = height = 0;  
    }
```

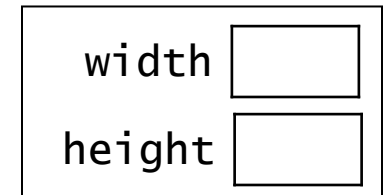
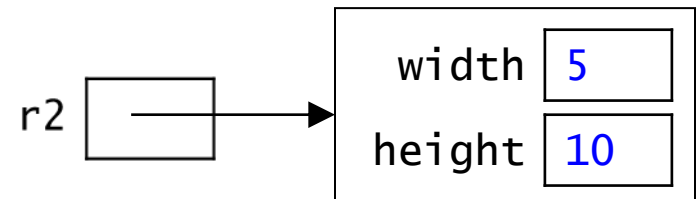
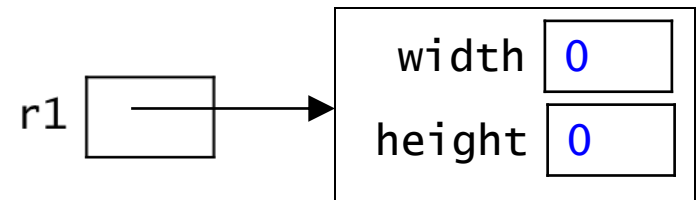
```
    .  
    .  
    .
```

```
    public static void main( String [] args ) {
```

```
        Rectangle r1 = new Rectangle();  
        Rectangle r2 = new Rectangle(5, 10);  
        Rectangle r3 = new Rectangle(7);
```

```
    }
```

```
}
```



Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;
```

```
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }
```

```
    public Rectangle(int dim) {  
        this(dim, dim);  
    }
```

```
    public Rectangle() {  
        width = height = 0;  
    }
```

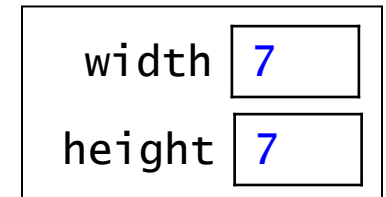
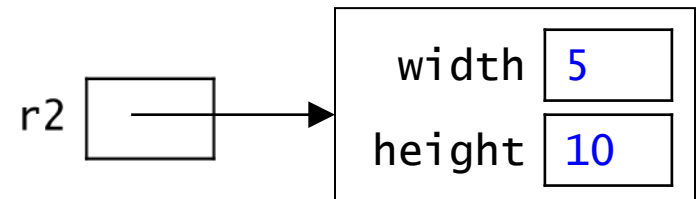
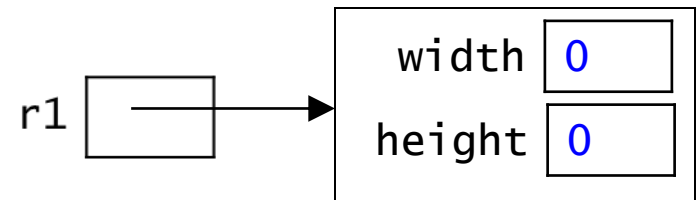
```
    .  
    .  
    .
```

```
    public static void main( String [] args ) {
```

```
        Rectangle r1 = new Rectangle();  
        Rectangle r2 = new Rectangle(5, 10);  
        Rectangle r3 = new Rectangle(7);
```

```
    }
```

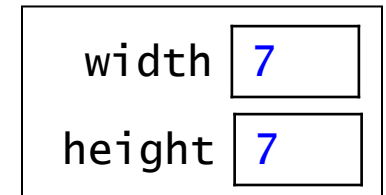
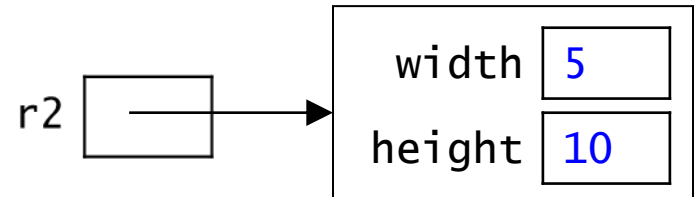
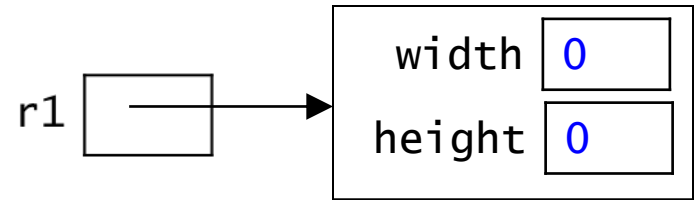
```
}
```



Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
  
    public Rectangle(int dim) {  
        this(dim, dim);  
    }  
  
    public Rectangle() {  
        width = height = 0;  
    }  
    .  
    .  
    .  
    public static void main
```

```
        Rectangle r1 = new  
        Rectangle r2 = new Rectangle(  
        Rectangle r3 = new Rectangle(  
    }  
}
```

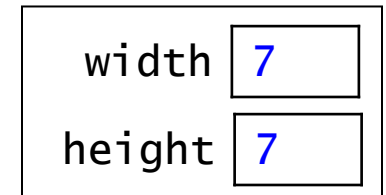
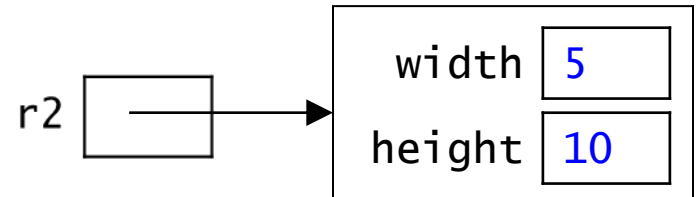
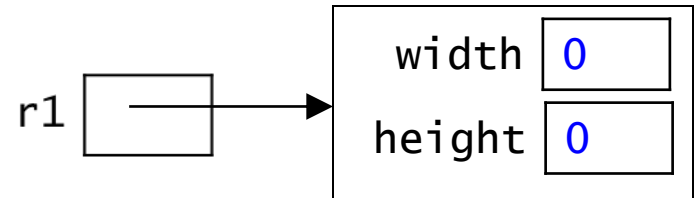


This constructor is also doing the same as the other two constructors.

Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
  
    public Rectangle(int dim) {  
        this(dim, dim);  
    }  
  
    public Rectangle() {  
        this(0, 0);  
    }  
    .  
    .  
    .  
    public static void main
```

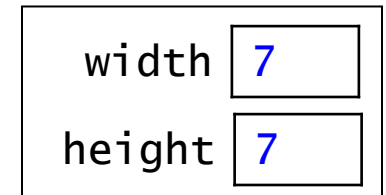
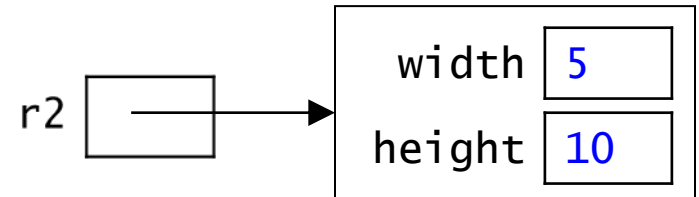
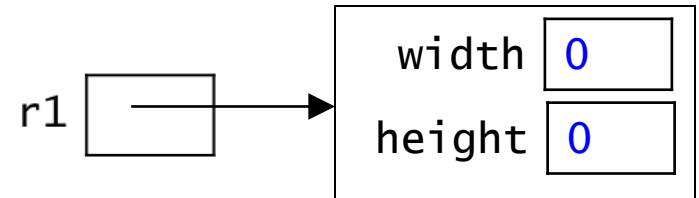
```
        Rectangle r1 = new  
        Rectangle r2 = new Rect  
        Rectangle r3 = new Rectangrct.  
    }  
}
```



Can use *this* to
call one of the other
constructors.

Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
    public Rectangle(int dim) {  
        this(dim, dim);  
    }  
    public Rectangle() {  
        this(0);  
    }  
    .  
    .  
    .  
    public static void main  
  
        Rectangle r1 = new  
        Rectangle r2 = new Rect  
        Rectangle r3 = new Rectangrect.  
  
}
```



Can use *this* to
call one of the other
constructors.

Blueprint Class vs. Client Program:

understanding the principle of Encapsulation

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w,  
        this.width = w;  
        this.height = h;  
}
```

We use proper
class encapsulation and
establish the *private* domain!

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
  
        int area1 = r1.width * r1.height;  
        System.out.println("r1's area = " + area1);  
  
        int area2 = r2.width * r2.height;  
        System.out.println("r2's area = " + area2);  
  
        // grow both rectangles  
        r1.width += 50;  r1.height += 10;  
        r2.width += 5;   r2.height += 30;  
  
        System.out.println("r1: " + r1.width + " x " + r1.height);  
        System.out.println("r2: " + r2.width + " x " + r2.height);  
    }  
}
```

Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
    ...  
}
```

Accessor Methods

Mutator Methods

Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
    public int getWidth() {  
        return width;  
    }  
    public int getHeight() {  
        return height;  
    }  
    public void grow(int dw, int dh) {  
        width += dw;  
        height += dh;  
    }  
    public int area() {  
        return( width*height );  
    }  
  
    ...  
}
```

Accessor Methods

- Allow *applications* or *client methods* to gain access to the data stored in private data members!

Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
    public int getWidth() {  
        return width;  
    }  
    public int getHeight() {  
        return height;  
    }  
    public void grow(int dw, int dh) {  
        width += dw;  
        height += dh;  
    }  
    public int area() {  
        return( width*height );  
    }  
    ...  
}
```

Accessor Methods

- Allow *applications* or *client methods* to gain access to the data stored in private data members!
- Or perform a necessary operation of the class without altering the values of the data members.

Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(  
        width = w;  
        height =  
    }  
    public int get  
        return wi  
    }  
    public int getHeight()  
        return height;  
    }  
    public void grow(int dw, int dh) {  
        width += dw;  
        height += dh;  
    }  
    public long area() {  
        return( width*height );  
    }  
  
    ...  
}
```

If the width and height of the rectangle object are very large integers, we can change the return type of the area method to the Long datatype!

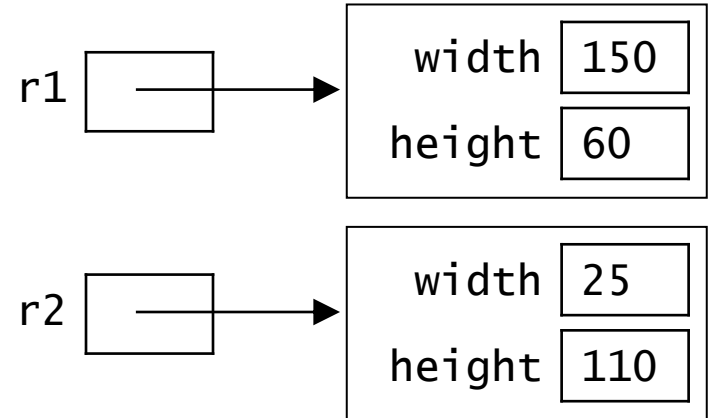
applications or *client* methods to gain access to the data stored in private data members!

- Or perform a necessary operation of the class without altering the values of the data members.

Blueprint Class vs. Client Program:

understanding the principle of Encapsulation

```
public class Rectangle {  
    int width;  
    int height;  
  
    public Rectangle(int w, int h) {  
        this.width = w;  
        this.height = h;  
    }  
}
```

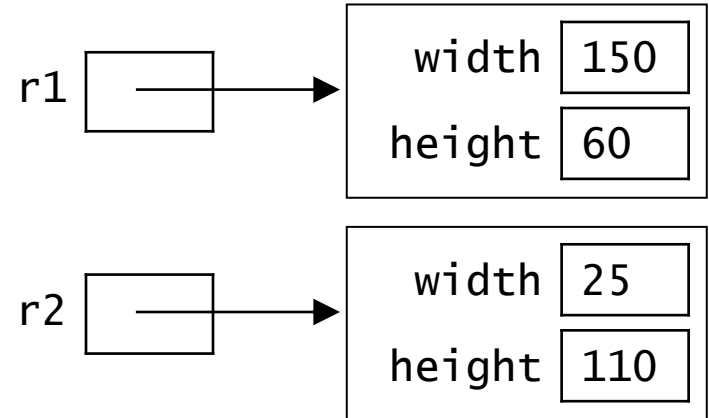


```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
  
        int area1 = r1.area();  
        System.out.println("r1's area = " + area1);  
  
        int area2 = r2.area();  
        System.out.println("r2's area = " + area2);  
  
        // grow both rectangles  
        r1.width += 50;  r1.height += 10;  
        r2.width += 5;   r2.height += 30;  
  
        System.out.println("r1: " + r1.width + " x " + r1.height);  
        System.out.println("r2: " + r2.width + " x " + r2.height);  
    }  
}
```


Blueprint Class vs. Client Program:

understanding the principle of Encapsulation

```
public class Rectangle {  
    int width;  
    int height;  
  
    public Rectangle(int w, int h) {  
        this.width = w;  
        this.height = h;  
    }  
}
```



```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
  
        int area1 = r1.area();  
        System.out.println("r1's area = " + area1);  
  
        int area2 = r2.area();  
        System.out.println("r2's area = " + area2);  
  
        // grow both rectangles  
        r1.width += 50;  r1.height += 10;  
        r2.width += 5;   r2.height += 30;  
  
        System.out.println("r1: " + r1.width + " x " + r1.height);  
        System.out.println("r2: " + r2.width + " x " + r2.height);  
    }  
}
```

Sample Rectangle Class

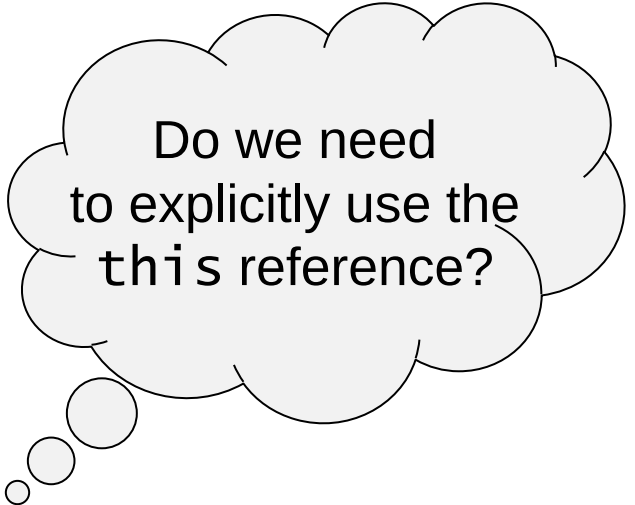
```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
    public int getWidth() {  
        return width;  
    }  
    public int getHeight() {  
        return height;  
    }  
    public void grow(int dw, int dh) {  
        width += dw;  
        height += dh;  
    }  
    public int area() {  
        return( width*height );  
    }  
  
    ...  
}
```

Mutator Methods

- Alter the values of the data members.

Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
    .  
    .  
    .  
}
```

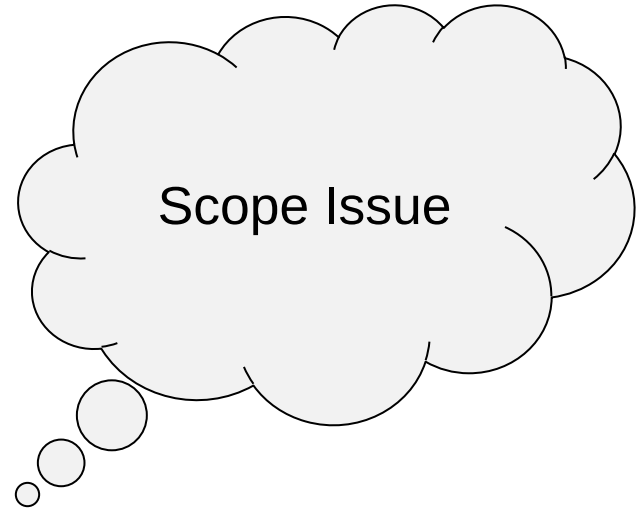


Do we need
to explicitly use the
this reference?

```
    public void grow(int dw, int dh) {  
        width += dw;  
        height += dh;  
    }  
    .  
    .  
    .  
}
```

Sample Rectangle Class

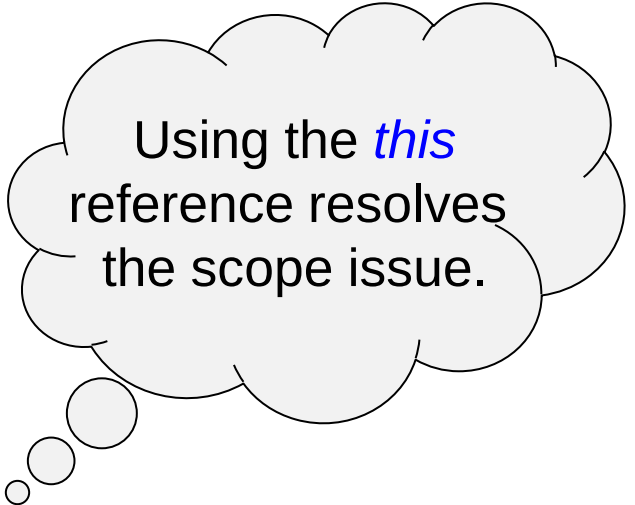
```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
    .  
    .  
    .  
}
```



```
    public void grow(int width, int height) {  
        width += width;  
        height += height;  
    }  
  
    .  
    .  
    .  
}
```

Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
    .  
    .  
    .  
}
```

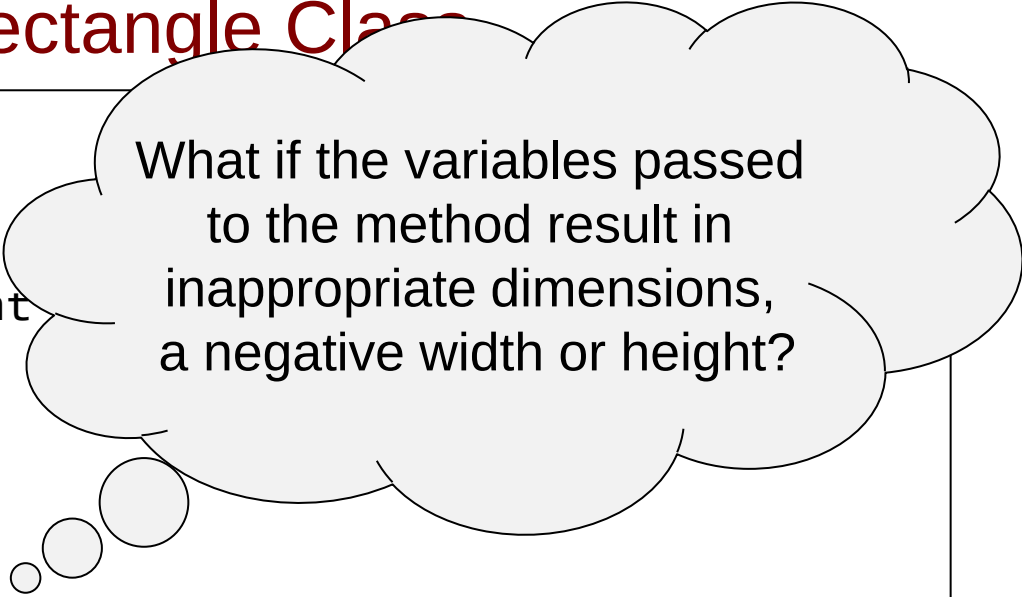


Using the *this* reference resolves the scope issue.

```
    public void grow(int width, int height) {  
        this.width += width;  
        this.height += height;  
    }  
    .  
    .  
    .  
}
```

Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
    .  
    .  
    .  
}
```



What if the variables passed to the method result in inappropriate dimensions, a negative width or height?

```
    public void grow(int dw, int dh) {  
        width += dw;  
        height += dh;  
    }  
    .  
    .  
    .  
}
```

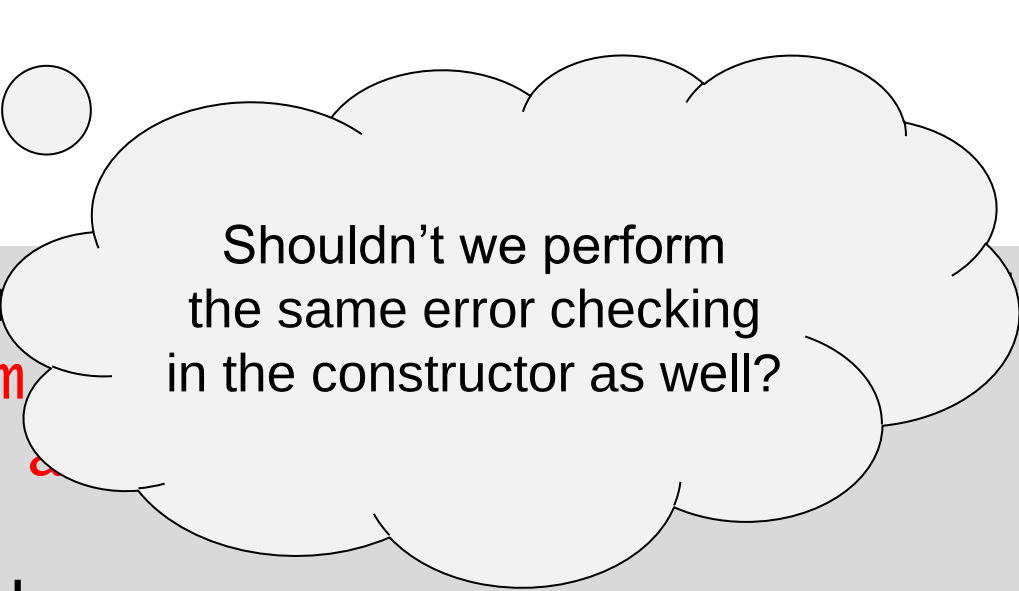
Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
    .  
    .  
    .  
  
    public void grow(int dw, int dh) {  
        // perform error checking before  
        // making assignments?  
  
        width += dw;  
        height += dh;  
    }  
}
```

Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
    .  
    .  
    .  
}
```

```
public void grow(int dw, int dh) {  
    // perform error checking  
    // making sure dw and dh are positive  
  
    width += dw;  
    height += dh;  
}
```



Shouldn't we perform
the same error checking
in the constructor as well?

Allowing Appropriate Changes

- To allow for appropriate changes to an object, we add whatever mutator methods make sense.
- These (*setter*) methods can prevent inappropriate changes:

```
public void setwidth(int w) {  
    if (w <= 0) {  
        throw new IllegalArgumentException();  
    }  
    this.width = w;  
}
```

Throwing an exception
ends the method call.

```
public void setHeight(int h) {  
    if (h <= 0) {  
        throw new IllegalArgumentException();  
    }  
    this.height = h;  
}
```

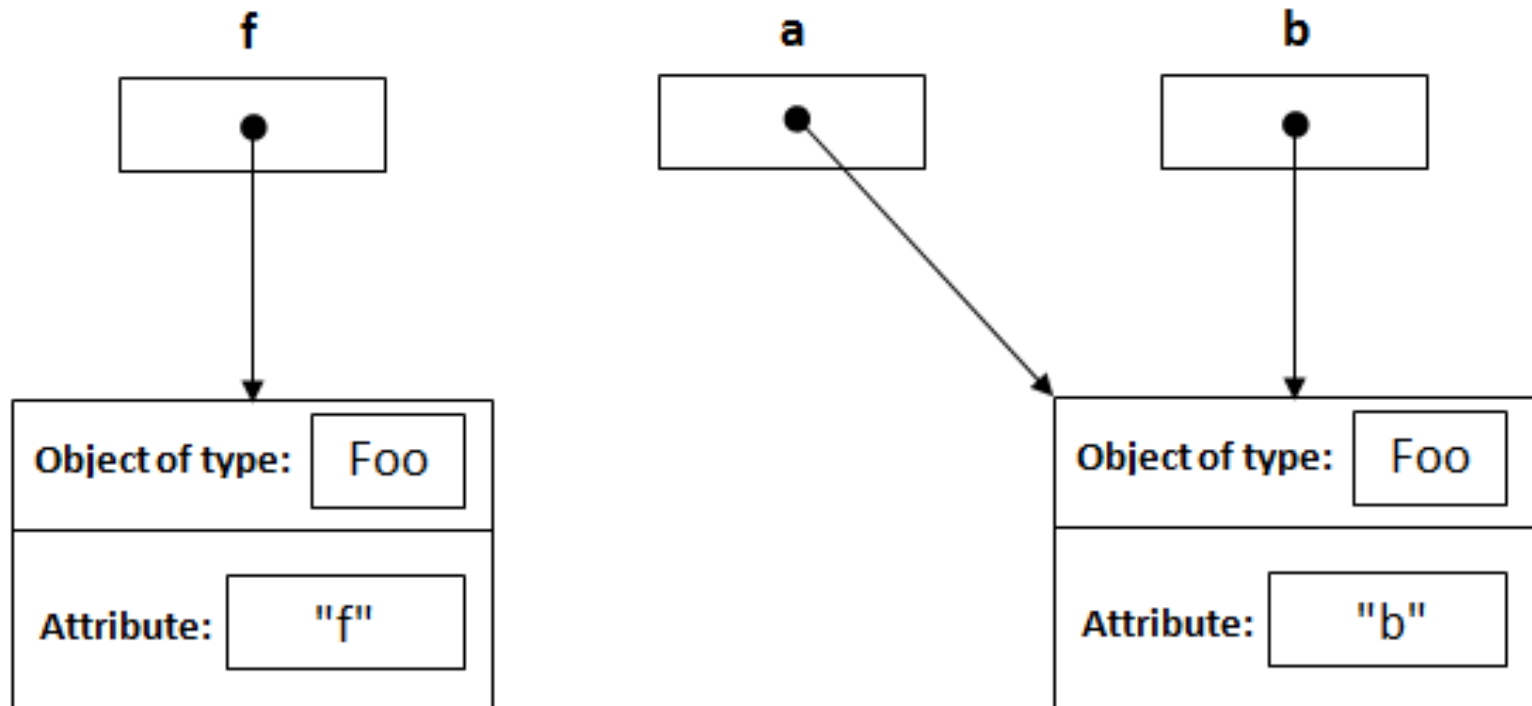
Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
    .  
    .  
    .  
  
    public void grow(int dw, int dh) {  
        setWidth(width+dw);  
        setHeight(height+dh);  
    }  
  
    .  
    .  
    .  
}
```

Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        this.setWidth(w);  
        this.setHeight(h);  
    }  
    .  
    .  
    .  
  
    public void grow(int dw, int dh) {  
        this.setWidth(width+dw);  
        this.setHeight(height+dh);  
    }  
  
    .  
    .  
    .  
}
```

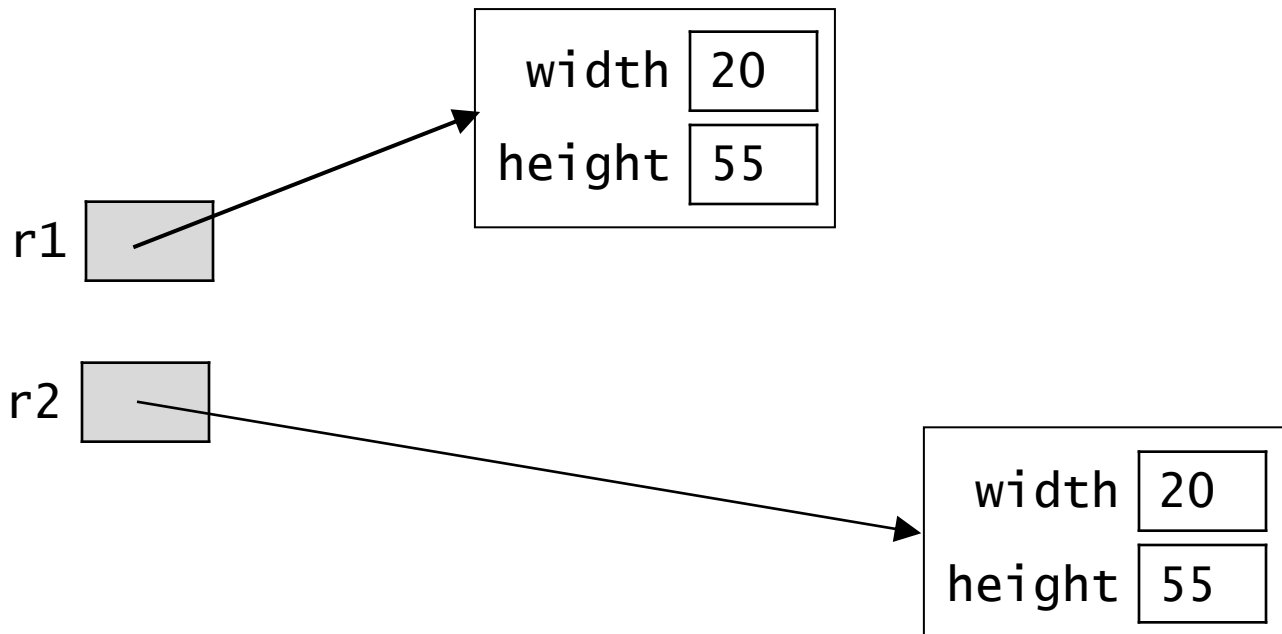
Objects are *Reference* types



Testing for Equivalent Objects

- Let's say that we have two different Rectangle objects, both of which represent the same rectangle:

```
Rectangle r1 = new Rectangle(20, 55);  
Rectangle r2 = new Rectangle(20, 55);
```

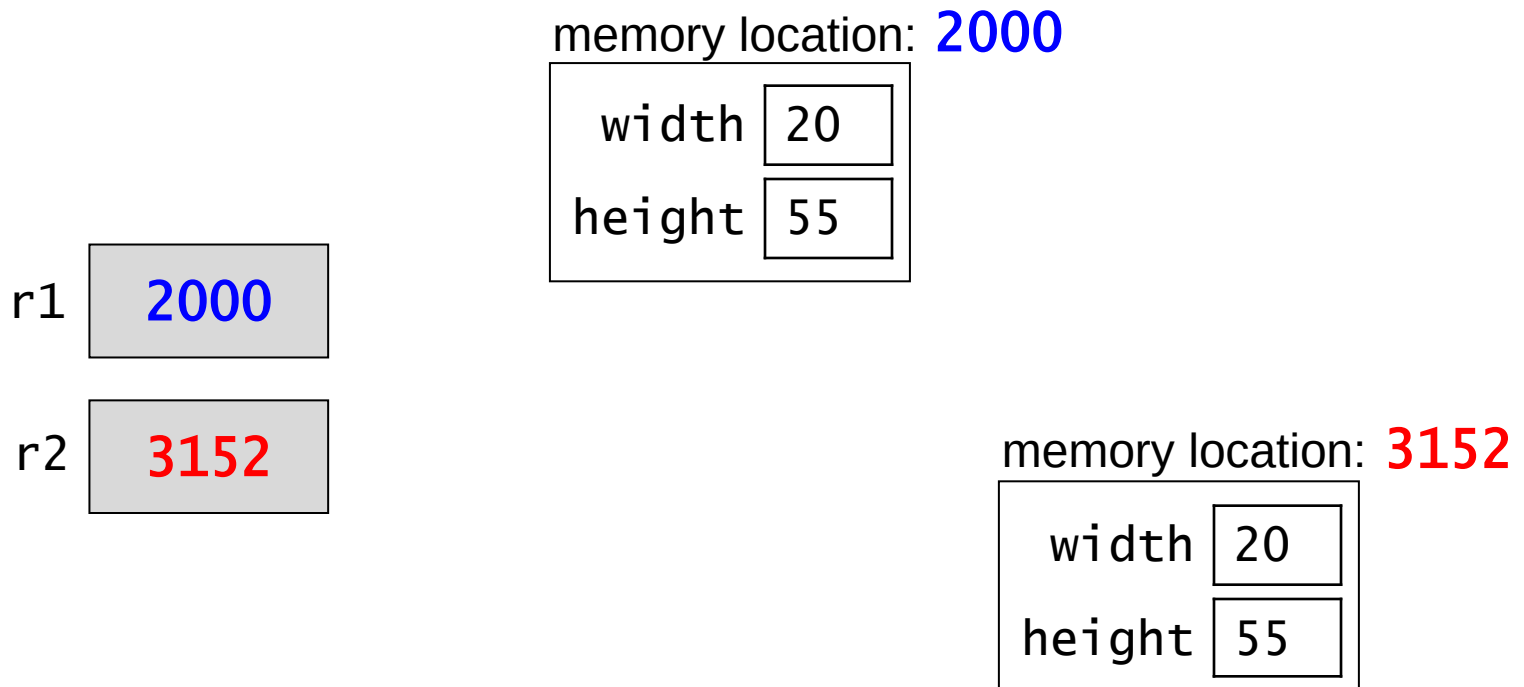


- What is the value of the following condition?

```
r1 == r2
```

Testing for Equivalent Objects (cont.)

- The condition
 $r1 == r2$
compares the *references* stored in $r1$ and $r2$.



- It doesn't compare the objects themselves.

Testing for Equivalent Objects (cont.)

- To test for equivalent objects, we need to use the `equals` method:

```
r1.equals(r2)
```

- Java's built-in classes have an *equals* methods that:
 - returns `true` if the two objects are equivalent to each other
 - returns `false` otherwise

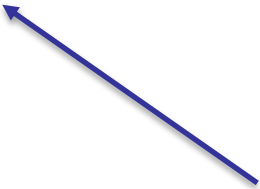
```
String s1 = "CS112";  
String s2 = "CS611";  
if ( s1.equals(s2) )  
    System.out.println("I am not doing my job!");
```

Default equals() Method

- If we don't write an equals() method for a class, objects of that class get a default version of this method.
- The default equals() just tests if the memory addresses of the two objects are the same.
 - the same as what == does!
- To ensure that we're able to test for equivalent objects, we need to write our own equals() method.

equals() Method for Our Rectangle Class

```
public boolean equals(Rectangle other) {  
    boolean isEqual = true;  
  
    if (other == null) {  
        isEqual = false;  
    } else  
        if (this.width != other.width) {  
            isEqual = false;  
        } else  
            if (this.height != other.height) {  
                isEqual = false;  
            }  
    return( isEqual );  
}
```



- **Note:** The method is able to access the fields in other directly (without using accessor methods).
- Instance methods can access the private data members of *any* object from the same class.

equals() Method for Our Rectangle Class (cont.)

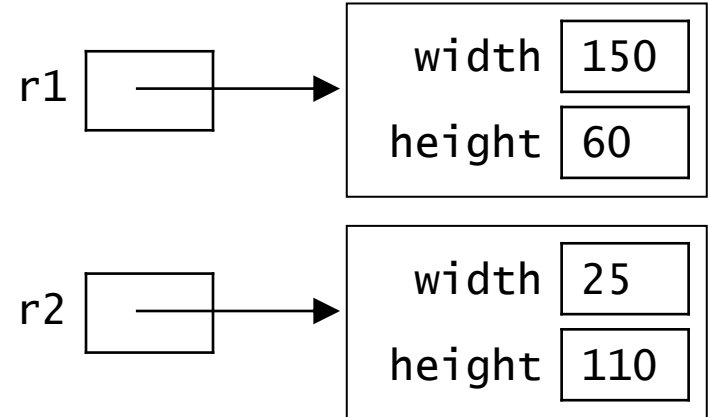
- Here's an alternative version:

```
public boolean equals(Rectangle other) {  
    return (other != null  
            && this.width == other.width  
            && this.height == other.height);  
}
```

Blueprint Class vs. Client Program:

understanding the principle of Encapsulation

```
public class Rectangle {  
    int width;  
    int height;  
  
    public Rectangle(int w, int h) {  
        this.width = w;  
        this.height = h;  
    }  
}
```

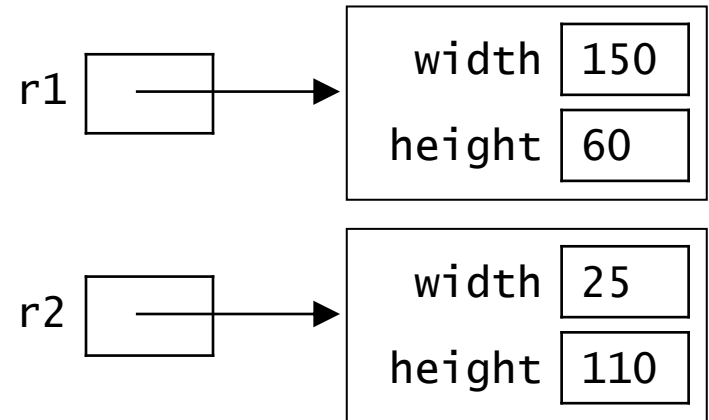


```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
  
        int area1 = r1.area();  
        System.out.println("r1's area = " + area1);  
  
        int area2 = r2.area();  
        System.out.println("r2's area = " + area2);  
  
        // grow both rectangles  
        r1.grow(50,10);  
        r2.grow(5,30);  
  
        System.out.println("r1: " + r1.width + " x " + r1.height);  
        System.out.println("r2: " + r2.width + " x " + r2.height);  
    }  
}
```

Blueprint Class vs. Client Program:

understanding the principle of Encapsulation

```
public class Rectangle {  
    int width;  
    int height;  
  
    public Rectangle(int w, int h) {  
        this.width = w;  
        this.height = h;  
    }  
}
```

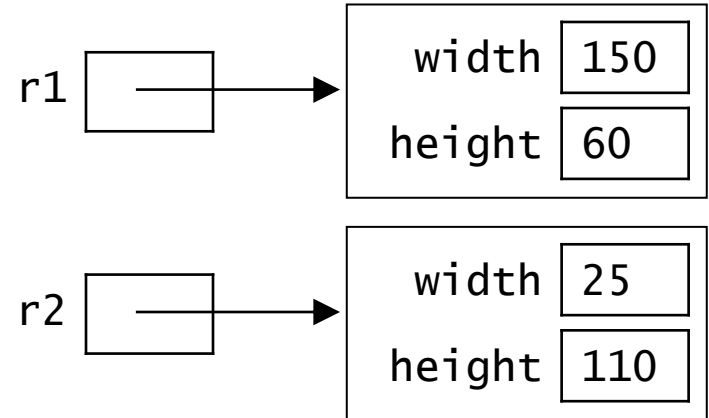


```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
  
        int area1 = r1.area();  
        System.out.println("r1's area = " + area1);  
  
        int area2 = r2.area();  
        System.out.println("r2's area = " + area2);  
  
        // grow both rectangles  
        r1.grow(50,10);  
        r2.grow(5,30);  
  
        System.out.println("r1: " + r1.getWidth() + " x " + r1.getHeight());  
        System.out.println("r2: " + r2.getWidth() + " x " + r2.getHeight());  
    }  
}
```

Blueprint Class vs. Client Program:

understanding the principle of Encapsulation

```
public class Rectangle {  
    int width;  
    int height;  
  
    public Rectangle(int w, int h) {  
        this.width = w;  
        this.height = h;  
    }  
}
```



```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
  
        int area1 = r1.area();  
        System.out.println("r1's area = " + area1);  
  
        int area2 = r2.area();  
        System.out.println("r2's area = " + area2);  
  
        // grow both rectangles  
        r1.grow(50,10);  
        r2.grow(5,30);  
  
        System.out.println("r1: " + r1);  
        System.out.println("r2: " + r2);  
    }  
}
```

Converting an Object to a String

- The `toString()` method allows objects to be displayed in a human-readable format.
 - it returns a string representation of the object
- This method is called *implicitly* when you attempt to print an object or when you perform string concatenation:

```
Rectangle r1 = new Rectangle(10, 20);  
System.out.println(r1);
```

equivalent to:

```
System.out.println(r1.toString());
```

Converting an Object to a String

- The `toString()` method allows objects to be displayed in a human-readable format.
 - it returns a string representation of the object
- This method is called implicitly when you attempt to print an object or when you perform string concatenation:

```
Rectangle r1 = new Rectangle(10, 20);  
System.out.println(r1);
```

```
// the second line above is equivalent to:  
System.out.println(r1.toString());
```

- If we don't write a `toString()` method for a class, objects of that class get a default version of this method.
 - here again, it usually makes sense to write our own version

toString() Method for Our Rectangle Class

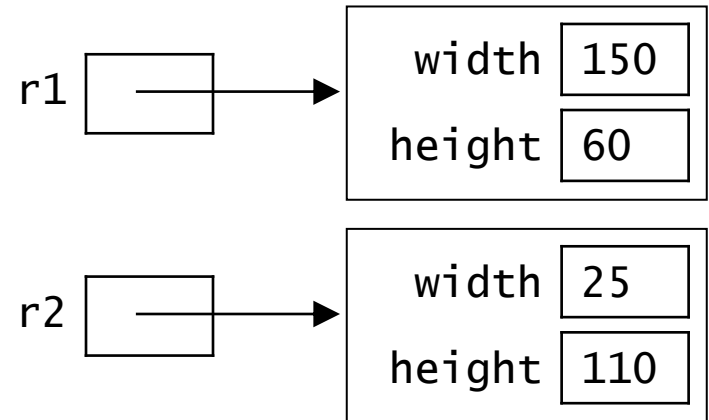
```
public String toString() {  
    return width + " x " + height;  
}
```

- Note: the method does not do any printing. It returns a String that can then be printed.

Blueprint Class vs. Client Program:

understanding the principle of Encapsulation

```
public class Rectangle {  
    int width;  
    int height;  
  
    public Rectangle(int w, int h) {  
        this.width = w;  
        this.height = h;  
    }  
}
```



```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
  
        int area1 = r1.area();  
        System.out.println("r1's area = " + area1);  
  
        int area2 = r2.area();  
        System.out.println("r2's area = " + area2);  
  
        // grow both rectangles  
        r1.grow(50,10);  
        r2.grow(5,30);  
  
        System.out.println("r1: " + r1);  
        System.out.println("r2: " + r2);  
    }  
}
```

Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        setWidth(w);  
        setHeight(h);  
    }  
    .  
    .  
    .  
    public void grow(int dw, int dh) {  
        setWidth(width+dw);  
        setHeight(height+dh);  
    }  
    public int area() {  
        return(width*height);  
    }  
    public boolean equals(Rectangle other) {  
        return (other != null && this.width == other.width  
                && this.height == other.height );  
    }  
    public String toString() {  
        return (width + " x " + height);  
    }  
}
```

A sample Client Program

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
  
        System.out.println("r1's area = " + r1.area() );  
  
        System.out.println("r2's area = " + r2.area() );  
  
        // grow both rectangles  
        r1.grow(50, 10);  
        r2.grow(5, 30);  
  
        System.out.println("r1: " + r1);  
        System.out.println("r2: " + r2);  
  
    }  
}
```

A sample Client Program

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
        System.out.println("r1's area is " + r1.getArea());  
        System.out.println("r2's area is " + r2.getArea());  
        // grow both rectangles  
        r1.grow(50, 10);  
        r2.grow(5, 30);  
        System.out.println("r1: " + r1);  
        System.out.println("r2: " + r2);  
  
        System.out.println("Number of Rectangles = " + ? );  
    }  
}
```

What if you wanted to
keep track of the total number
of rectangles created?

Sample Rectangle Class

```
public class Rectangle {
    private int width;
    private int height;
    private static int rectanglesCreated = 0;

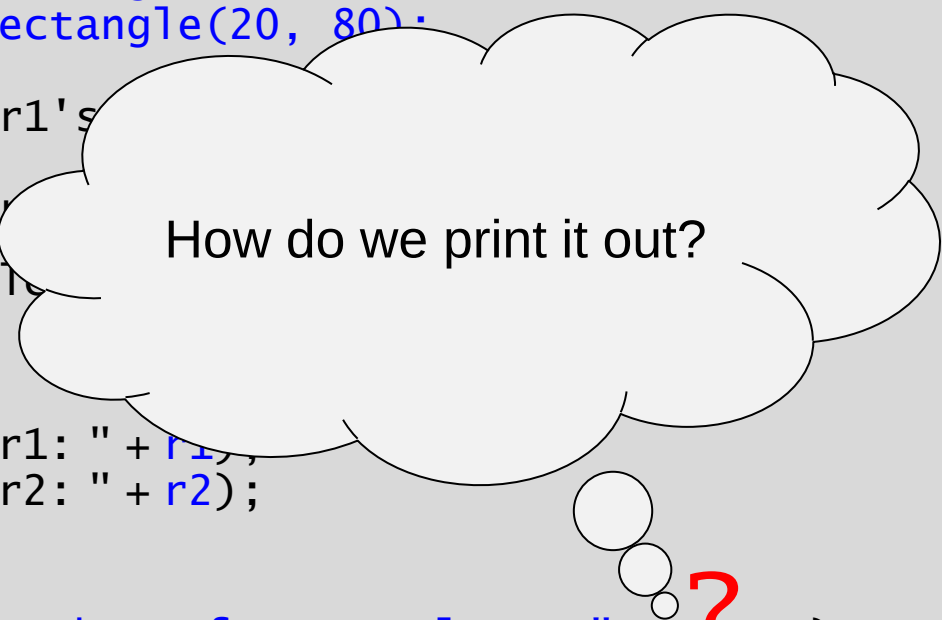
    public Rectangle(int w, int h) {
        setWidth(w);
        setHeight(h);

        rectanglesCreated++;
    }

    .
    .
    public void grow(int dw, int dh) {
        setWidth(width+dw);
        setHeight(height+dh);
    }
    public int area() {
        return(width*height);
    }
    public boolean equals(Rectangle other) {
        return (other != null && this.width == other.width
                && this.height == other.height );
    }
}
```

A sample Client Program

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
  
        System.out.println("r1's  
        System.out.println("r2's  
        // grow both rectangles  
        r1.grow(50, 10);  
        r2.grow(5, 30);  
  
        System.out.println("r1: " + r1);  
        System.out.println("r2: " + r2);  
  
        System.out.println("Number of Rectangles = " + ? );  
    }  
}
```



How do we print it out?

Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    private static int rectanglesCreated = 0;  
  
    public Rectangle(int w, int h) {  
        setWidth(w);  
        setHeight(h);  
  
        rectanglesCreated++;  
    }  
    .  
    .  
    public void grow(int dw, int dh) {  
        setWidth(width+dw);  
        setHeight(height+dh);  
    }  
    public int area() {  
        return(width*height);  
    }  
    public boolean equals(Rectangle other) {  
        return (other != null && this.width == other.width  
                && this.height == other.height);  
    }  
}
```

A sample Client Program

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
  
        System.out.println("r1's area = " + r1.area() );  
  
        System.out.println("r2's area = " + r2.area() );  
  
        // grow both rectangles  
        r1.grow(50, 10);  
        r2.grow(5, 30);  
  
        System.out.println("r1: " + r1);  
        System.out.println("r2: " + r2);  
  
        System.out.println("Number of Rectangles = " +  
                           Rectangle.rectanglesCreated );  
    }  
}
```



Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public static int rectanglesCreated = 0;  
  
    public Rectangle(int w, int h) {  
        setWidth(w);  
        setHeight(h);  
  
        rectanglesCreated++;  
    }  
    .  
    .  
    public void grow(int dw, int dh) {  
        setWidth(width+dw);  
        setHeight(height+dh);  
    }  
    public int area() {  
        return(width*height);  
    }  
    public boolean equals(Rectangle other) {  
        return (other != null && this.width == other.width  
                && this.height == other.height);  
    }  
}
```

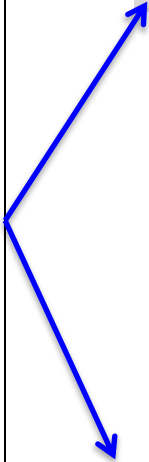
A sample Client Program

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
  
        System.out.println("r1's area = " + r1.area() );  
        System.out.println("r2's area = " + r2.area() );  
        // grow both rectangles  
        r1.grow(50, 10);  
        r2.grow(5, 30);  
  
        System.out.println("r1: " + r1);  
        System.out.println("r2: " + r2);  
  
        System.out.println("Number of Rectangles = " +  
                           Rectangle.rectanglesCreated );  
        Rectangles.rectanglesCreated = 0;  
    }  
}
```



Sample Rectangle Class

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    private static int rectanglesCreated = 0;  
  
    public Rectangle(int w, int h) {  
        setWidth(w);  
        setHeight(h);  
  
        rectanglesCreated++;  
    }  
    .  
    .  
  
    public static int numRectanglesCreated() {  
        return( rectanglesCreated );  
    }  
  
    public void grow(int dw, int dh) {  
        setWidth(width+dw);  
        setHeight(height+dh);  
    }  
  
    public int area() {  
        return(width*height);  
    }  
}
```



A blue arrow originates from the left side of the slide and splits into two branches. The upper branch points to the line `private static int rectanglesCreated = 0;`, which is highlighted with a light gray background. The lower branch points to the line `public static int numRectanglesCreated() {`.

A sample Client Program

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
  
        System.out.println("r1's area = " + r1.area() );  
  
        System.out.println("r2's area = " + r2.area() );  
  
        // grow both rectangles  
        r1.grow(50, 10);  
        r2.grow(5, 30);  
  
        System.out.println("r1: " + r1);  
        System.out.println("r2: " + r2);  
  
        System.out.println("Number of Rectangles = " +  
                           Rectangle.numRectanglesCreated() );  
    }  
}
```



A sample Client Program:

memory trace

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
        System.out.println("r1's area = " + r1.area() );  
        System.out.println("r2's area = " + r2.area() );  
        // grow both rectangles  
        r1.grow(50, 10);  
        r2.grow(5, 30);  
        System.out.println("r1: " + r1);  
        System.out.println("r2: " + r2);  
        ...  
    }  
}
```

Memory Stack

Memory Heap

Static

Rectangles
Created

0

A sample Client Program:

memory trace

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
        System.out.println("r1's area = " + r1.area() );  
        System.out.println("r2's area = " + r2.area() );  
        // grow both rectangles  
        r1.grow(50, 10);  
        r2.grow(5, 30);  
        System.out.println("r1: " + r1);  
        System.out.println("r2: " + r2);  
        ...  
    }  
}
```

Memory Stack

Memory Heap

Static

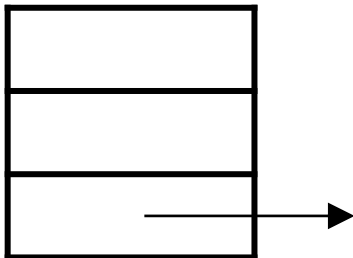
Rectangles
Created

0

r2

r1

args



A sample Client Program:

memory trace

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
        System.out.println("r1's area = " + r1.area() );  
        System.out.println("r2's area = " + r2.area() );  
        // grow both rectangles  
        r1.grow(50, 10);  
        r2.grow(5, 30);  
        System.out.println("r1: " + r1);  
        System.out.println("r2: " + r2);  
        ...  
    }  
}
```

Memory Stack

Memory Heap

Static

width	
height	

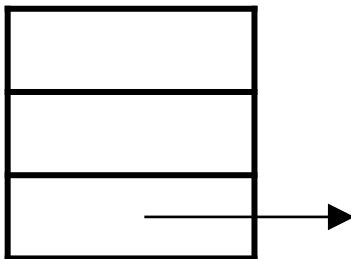
Rectangles
Created

0

r2

r1

args



A sample Client Program:

memory trace

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
        System.out.println("r1's area = " + r1.area() );  
        System.out.println("r2's area = " + r2.area() );  
        // grow both rectangles  
        r1.grow(50, 10);  
        r2.grow(5, 30);  
        System.out.println("r1: " + r1);  
        System.out.println("r2: " + r2);  
        ...  
    }  
}
```

Memory Stack

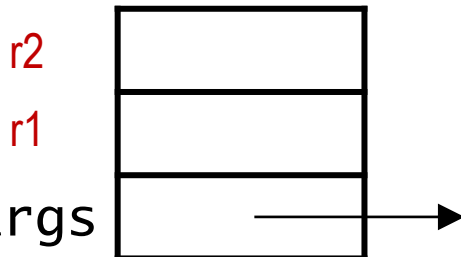
Memory Heap

Static

width	
height	

Rectangles
Created

0



A sample Client Program:

memory trace

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
        rectanglesCreated++;  
    }  
    .
```

Note: for purposes of this example, showing the simplified method that directly updates the variables.

Memory Stack

Memory Heap

Static

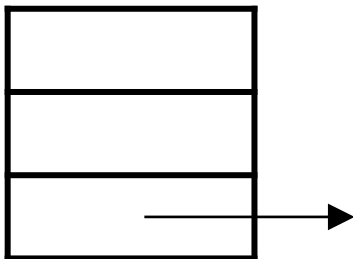
width	
height	

Rectangles Created	0
--------------------	---

r2

r1

args



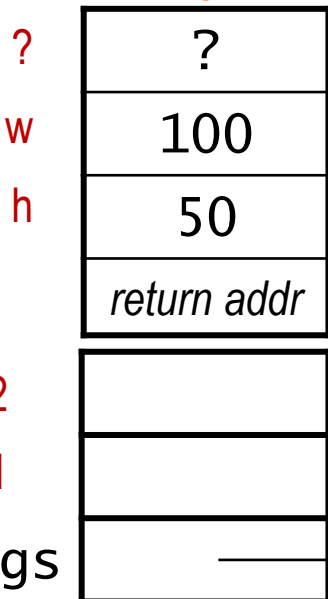
A sample Client Program:

memory trace

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
        rectanglesCreated++;  
    }  
}
```

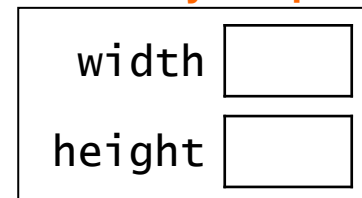
Note: for purposes of this example, showing the simplified method that directly updates the variables.

Memory Stack



constructor call

Memory Heap



Static

Rectangles Created 0

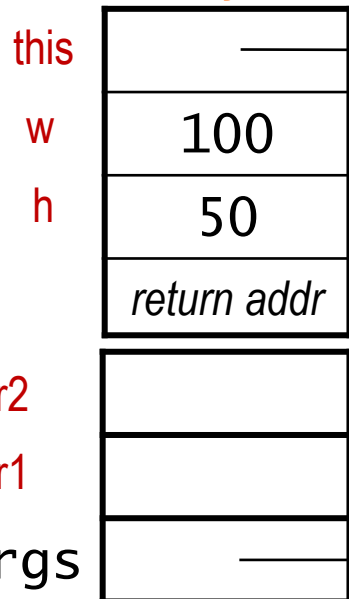
A sample Client Program:

memory trace

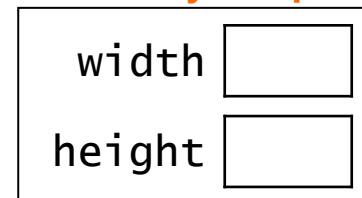
```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
        rectanglesCreated++;  
    }  
}
```

Note: for purposes of this example, showing the simplified method that directly updates the variables.

Memory Stack



Memory Heap



Static



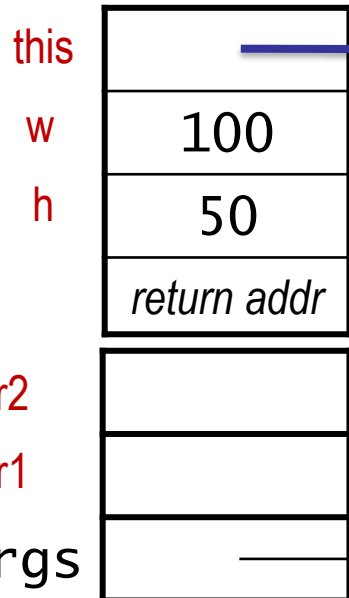
A sample Client Program:

memory trace

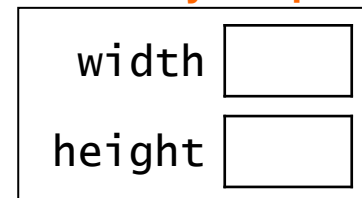
```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        this.width = w;  
        this.height = h;  
        rectanglesCreated++;  
    }  
}
```

Note: for purposes of this example, showing the simplified method that directly updates the variables.

Memory Stack



Memory Heap



Static



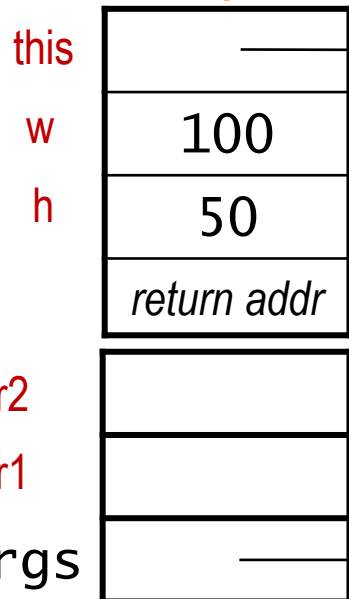
A sample Client Program:

memory trace

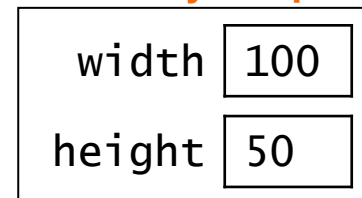
```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
        rectanglesCreated++;  
    }  
}
```

Note: for purposes of this example, showing the simplified method that directly updates the variables.

Memory Stack



Memory Heap



Static



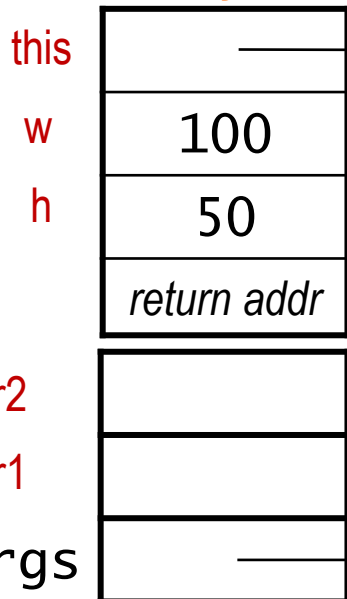
A sample Client Program:

memory trace

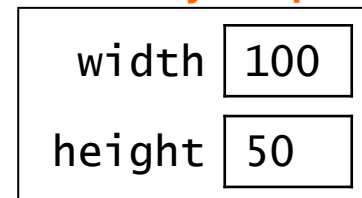
```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
        rectanglesCreated++;  
    }  
}
```

Note: for purposes of this example, showing the simplified method that directly updates the variables.

Memory Stack



Memory Heap



Static



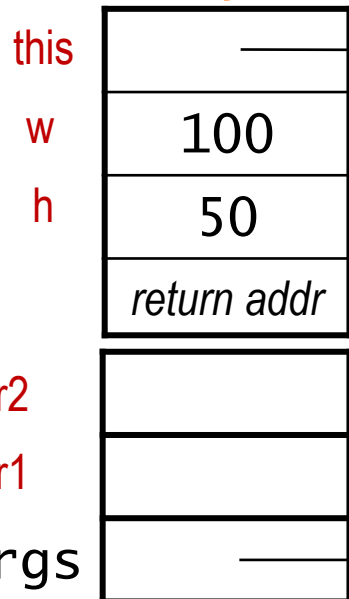
A sample Client Program:

memory trace

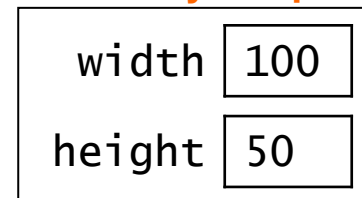
```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
        rectanglesCreated++;  
    }  
}
```

Note: for purposes of this example, showing the simplified method that directly updates the variables.

Memory Stack



Memory Heap



Static

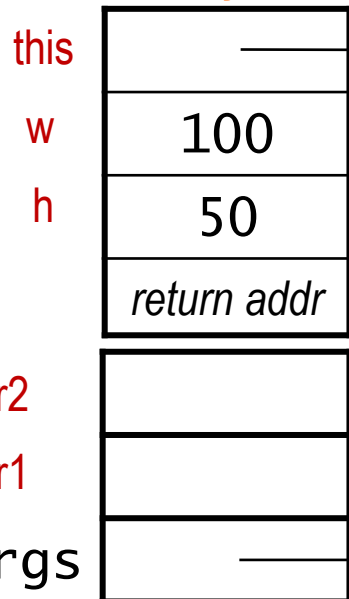


A sample Client Program:

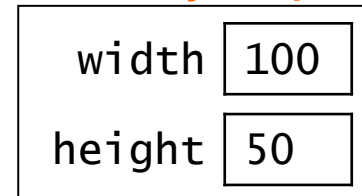
memory trace

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
        rectanglesCreated++;  
    } // returning from method  
}
```

Memory Stack



Memory Heap



Static



A sample Client Program:

memory trace

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
        System.out.println("r1's area = " + r1.area() );  
        System.out.println("r2's area = " + r2.area() );  
        // grow both rectangles  
        r1.grow(50, 10);  
        r2.grow(5, 30);  
        System.out.println("r1: " + r1);  
        System.out.println("r2: " + r2);  
        ...  
    }  
}
```

Memory Stack

Memory Heap

Static

Rectangles
Created

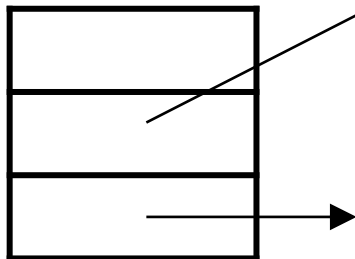
1

width	100
height	50

r2

r1

args



A sample Client Program:

memory trace

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
        System.out.println("r1's area = " + r1.area() );  
        System.out.println("r2's area = " + r2.area() );  
        // grow both rectangles  
        r1.grow(50, 10);  
        r2.grow(5, 30);  
        System.out.println("r1: " + r1);  
        System.out.println("r2: " + r2);  
        ...  
    }  
}
```

Memory Stack

Memory Heap

Static

Rectangles
Created

1

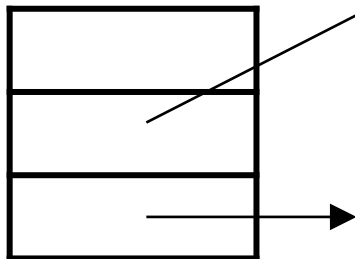
width 100
height 50

width
height

r2

r1

args



A sample Client Program:

memory trace

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
        System.out.println("r1's area = " + r1.area() );  
        System.out.println("r2's area = " + r2.area() );  
        // grow both rectangles  
        r1.grow(50, 10);  
        r2.grow(5, 30);  
        System.out.println("r1: " + r1);  
        System.out.println("r2: " + r2);  
        ...  
    }  
}
```

Memory Stack

Memory Heap

Static

Rectangles
Created

1

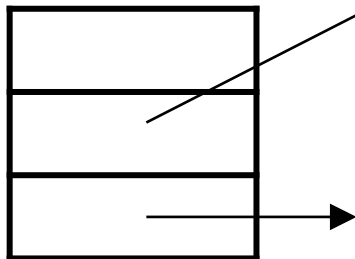
width 100
height 50

width
height

r2

r1

args



A sample Client Program:

memory trace

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
        rectanglesCreated++;  
    }  
}
```

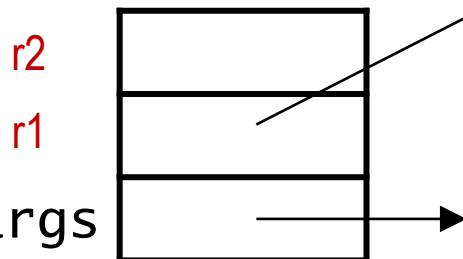
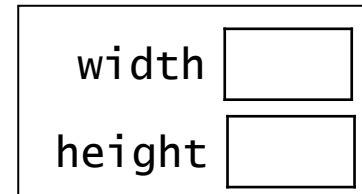
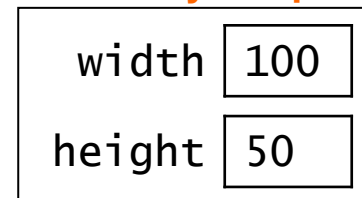
Note: for purposes of this example, showing the simplified method that directly updates the variables.

Memory Stack

Memory Heap

Static

Rectangles Created 1



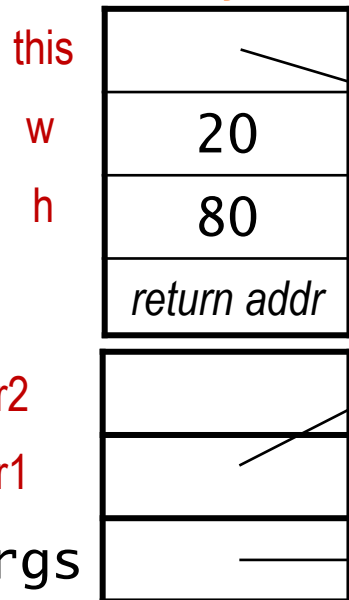
A sample Client Program:

memory trace

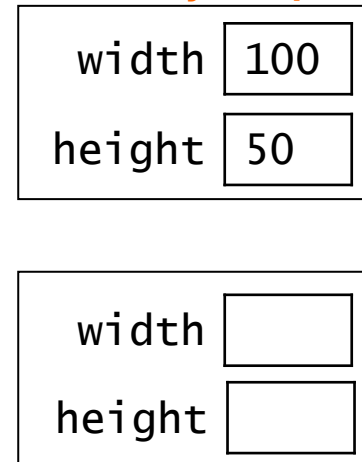
```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
        rectanglesCreated++;  
    }  
}
```

Note: for purposes of this example, showing the simplified method that directly updates the variables.

Memory Stack



Memory Heap



Static

Rectangles Created 1

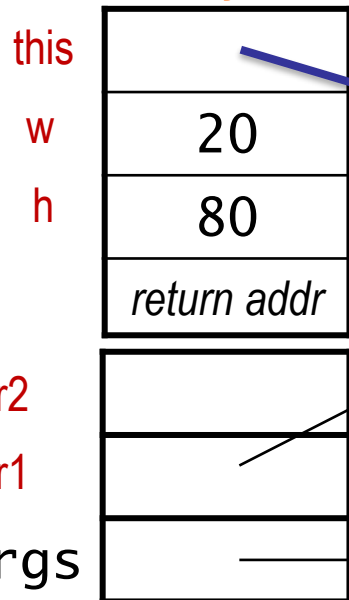
A sample Client Program:

memory trace

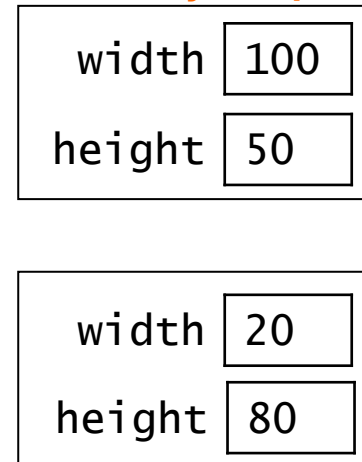
```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
        rectanglesCreated++;  
    }  
}
```

Note: for purposes of this example, showing the simplified method that directly updates the variables.

Memory Stack



Memory Heap



Static

Rectangles Created 1

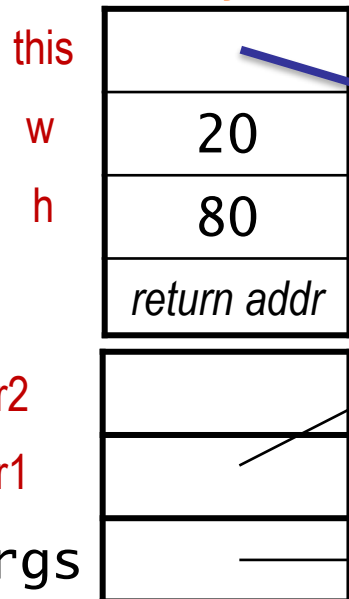
A sample Client Program:

memory trace

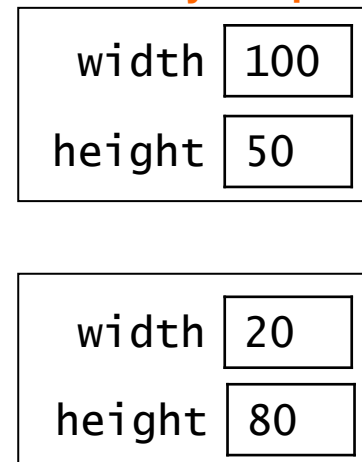
```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
        rectanglesCreated++;  
    }  
}
```

Note: for purposes of this example, showing the simplified method that directly updates the variables.

Memory Stack



Memory Heap



Static

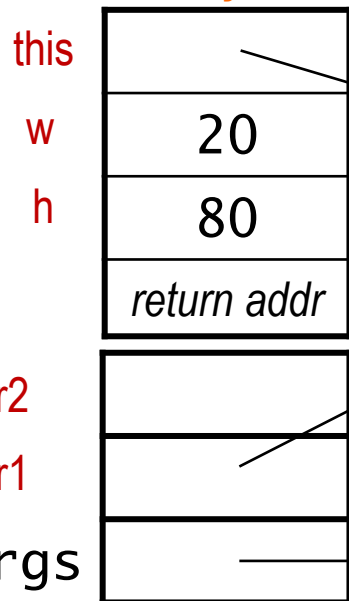
Rectangles Created 2

A sample Client Program:

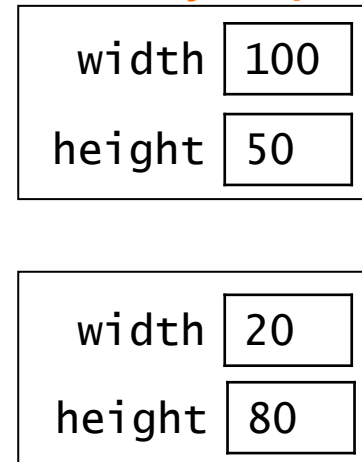
memory trace

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        width = w;  
        height = h;  
        rectanglesCreated++;  
    } // return from method  
}
```

Memory Stack



Memory Heap



Static

Rectangles Created 2

A sample Client Program:

memory trace

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
        System.out.println("r1's area = " + r1.area() );  
        System.out.println("r2's area = " + r2.area() );  
        // grow both rectangles  
        r1.grow(50, 10);  
        r2.grow(5, 30);  
        System.out.println("r1: " + r1);  
        System.out.println("r2: " + r2);  
        ...  
    }  
}
```

Memory Stack

Memory Heap

Static

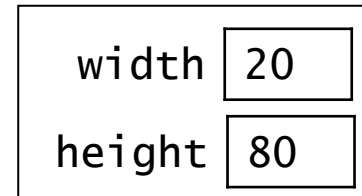
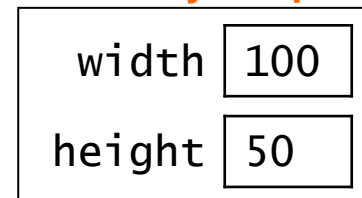
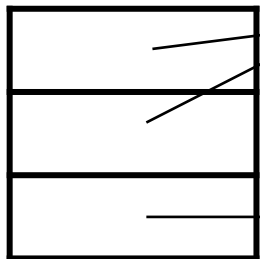
Rectangles
Created

2

r2

r1

args



A sample Client Program:

memory trace

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
        System.out.println("r1's area = " + r1.area() );  
        System.out.println("r2's area = " + r2.area() );  
        // grow both rectangles  
        r1.grow(50, 10);  
        r2.grow(5, 30);  
        System.out.println("r1: " + r1);  
        System.out.println("r2: " + r2);  
        ...  
    }  
}
```

Memory Stack

Memory Heap

Static

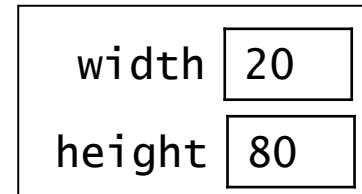
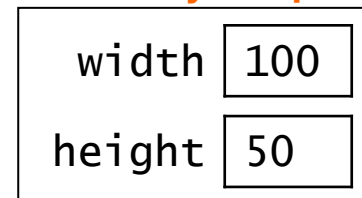
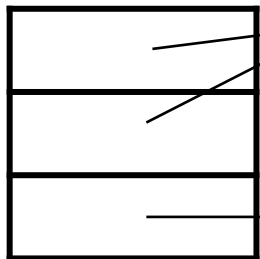
Rectangles
Created

2

r2

r1

args



A sample Client Program:

memory trace

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
        System.out.println("r1's area = " + r1.area() );  
        System.out.println("r2's area = " + r2.area() );  
        // grow both rectangles  
        r1.grow(50, 10);  
        r2.grow(5, 30);  
        System.out.println("r1: " + r1);  
        System.out.println("r2: " + r2);  
        ...  
    }  
}
```

Memory Stack

Memory Heap

Static

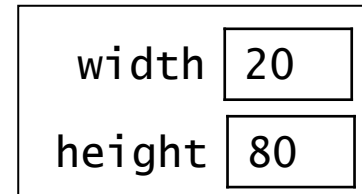
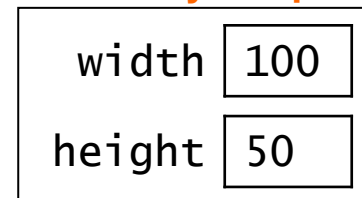
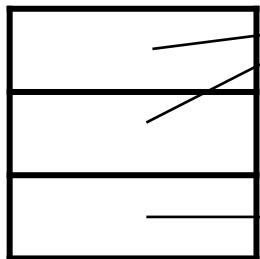
Rectangles
Created

2

r2

r1

args



A sample Client Program:

memory trace

```
public class Rectangle {  
    private int width;  
    private int height;  
    .  
    .  
    public int area() {  
        return( width*height );  
    }  
    .  
}
```

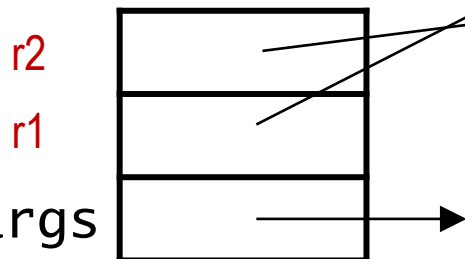
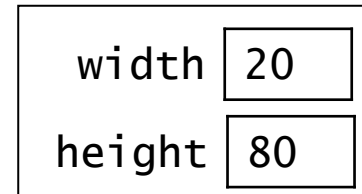
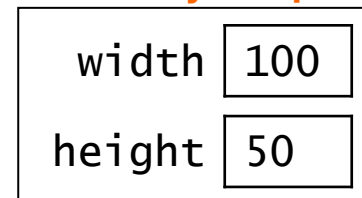
Memory Stack

Memory Heap

Static

Rectangles
Created

2



A sample Client Program:

memory trace

```
public class Rectangle {  
    private int width;  
    private int height;  
    .  
    .  
    public int area() {  
        return( width*height );  
    }  
    .  
}
```

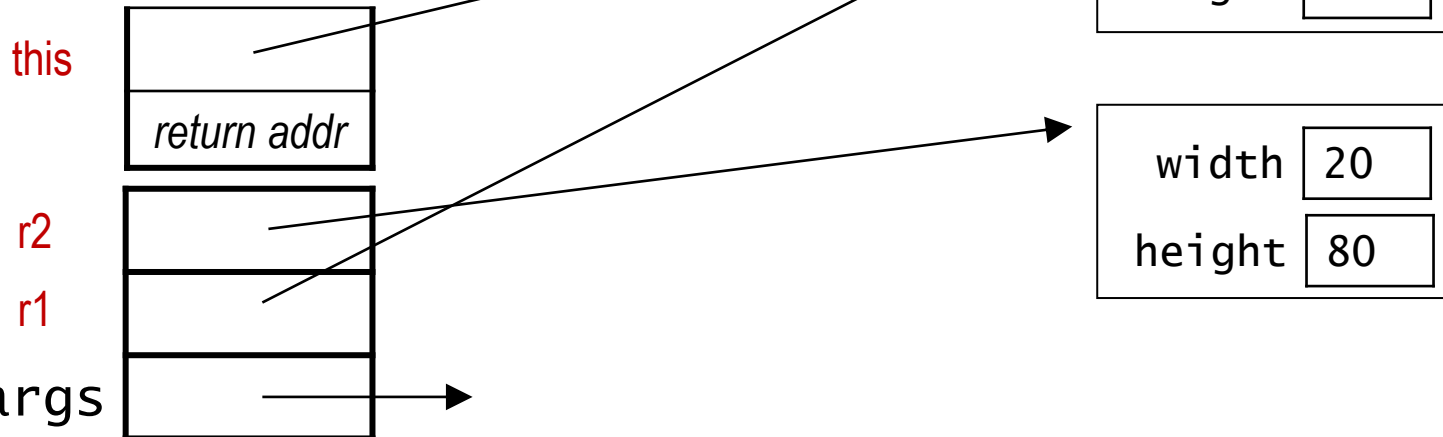
Memory Stack

Memory Heap

Static

Rectangles
Created

2



A sample Client Program:

memory trace

```
public class Rectangle {  
    private int width;  
    private int height;  
    .  
    .  
    public int area() {  
        return( width*height );  
    }  
    .  
}
```

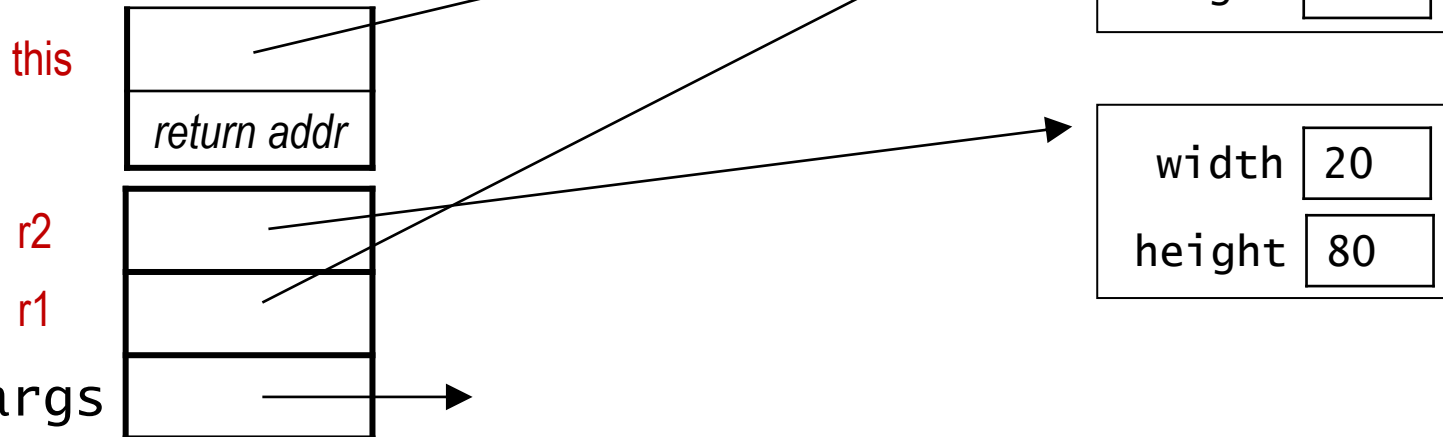
Memory Stack

Memory Heap

Static

Rectangles
Created

2



A sample Client Program:

memory trace

```
public class Rectangle {  
    private int width;  
    private int height;  
    .  
    .  
    public int area() {  
        return( this.width*this.height );  
    }  
    .  
}
```

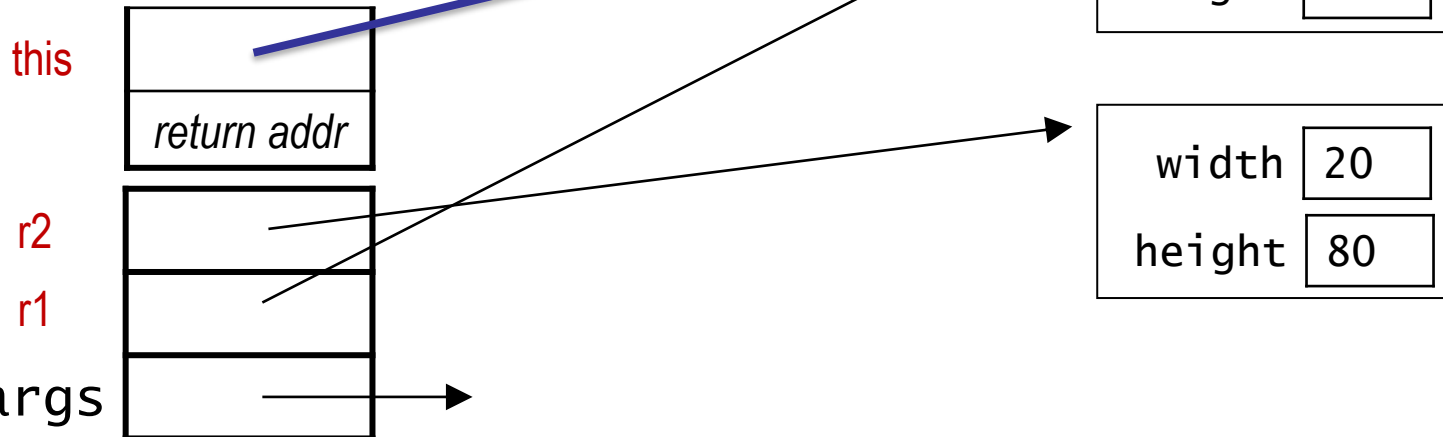
Memory Stack

Memory Heap

Static

Rectangles
Created

2



A sample Client Program:

memory trace

```
public class Rectangle {  
    private int width;  
    private int height;  
    .  
    .  
    public int area() {  
        return( this.width*this.height ); // returns 5000  
    }  
    .  
}
```

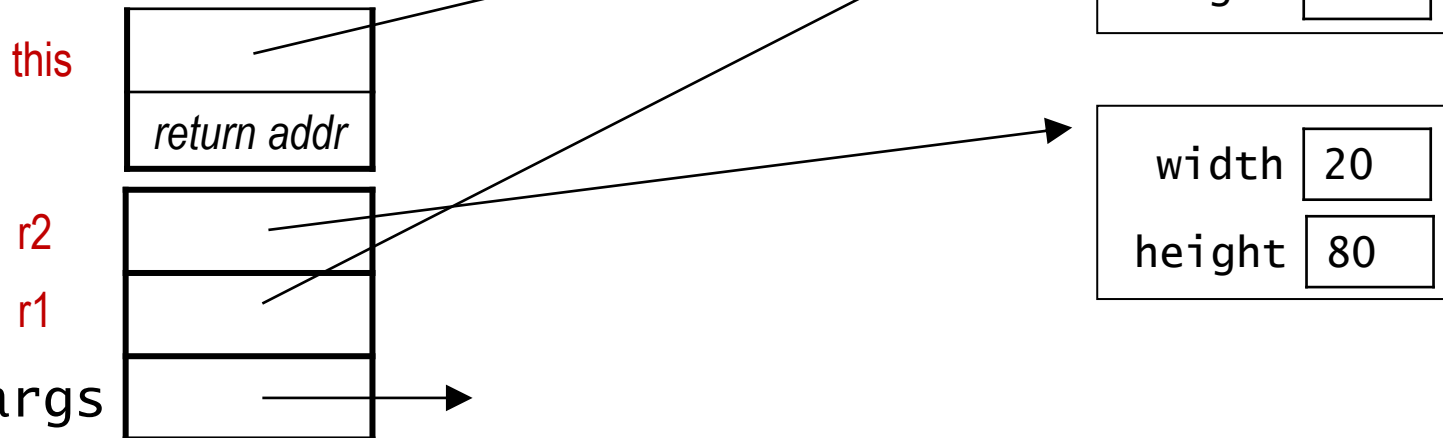
Memory Stack

Memory Heap

Static

Rectangles
Created

2



A sample Client Program:

memory trace

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
        System.out.println("r1's area = " + 5000 ); // outputs area = 5000  
        System.out.println("r2's area = " + r2.area() );  
        // grow both rectangles  
        r1.grow(50, 10);  
        r2.grow(5, 30);  
        System.out.println("r1: " + r1);  
        System.out.println("r2: " + r2);  
        ...  
    }  
}
```

Memory Stack

Memory Heap

Static

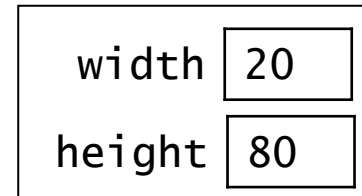
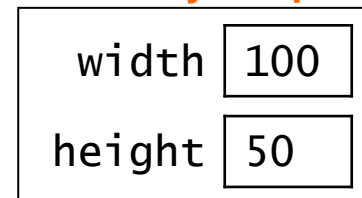
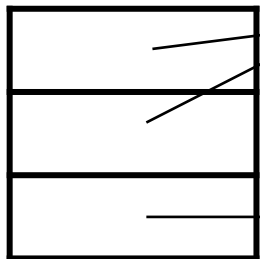
Rectangles
Created

2

r2

r1

args



A sample Client Program:

memory trace

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
        System.out.println("r1's area = " + 5000 ); // outputs area = 5000  
        System.out.println("r2's area = " + r2.area() );  
        // grow both rectangles  
        r1.grow(50, 10);  
        r2.grow(5, 30);  
        System.out.println("r1: " + r1);  
        System.out.println("r2: " + r2);  
        ...  
    }  
}
```

Memory Stack

Memory Heap

Static

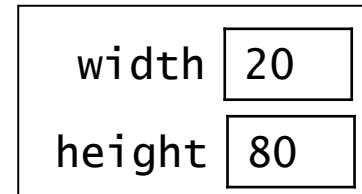
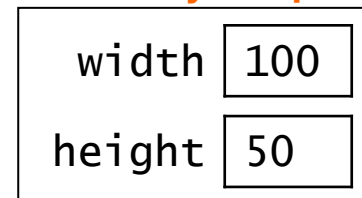
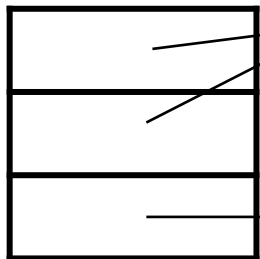
Rectangles
Created

2

r2

r1

args



A sample Client Program:

memory trace

```
public class Rectangle {  
    private int width;  
    private int height;  
    .  
    .  
    public int area() {  
        return( width*height );  
    }  
    .  
}
```

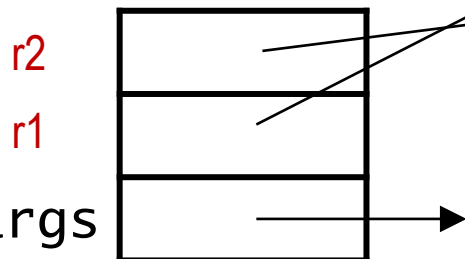
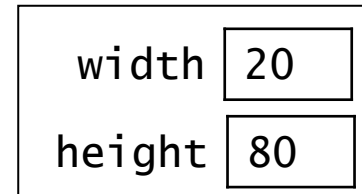
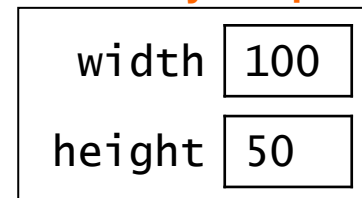
Memory Stack

Memory Heap

Static

Rectangles
Created

2



A sample Client Program:

memory trace

```
public class Rectangle {  
    private int width;  
    private int height;  
    .  
    .  
    public int area() {  
        return( width*height );  
    }  
    .  
}
```

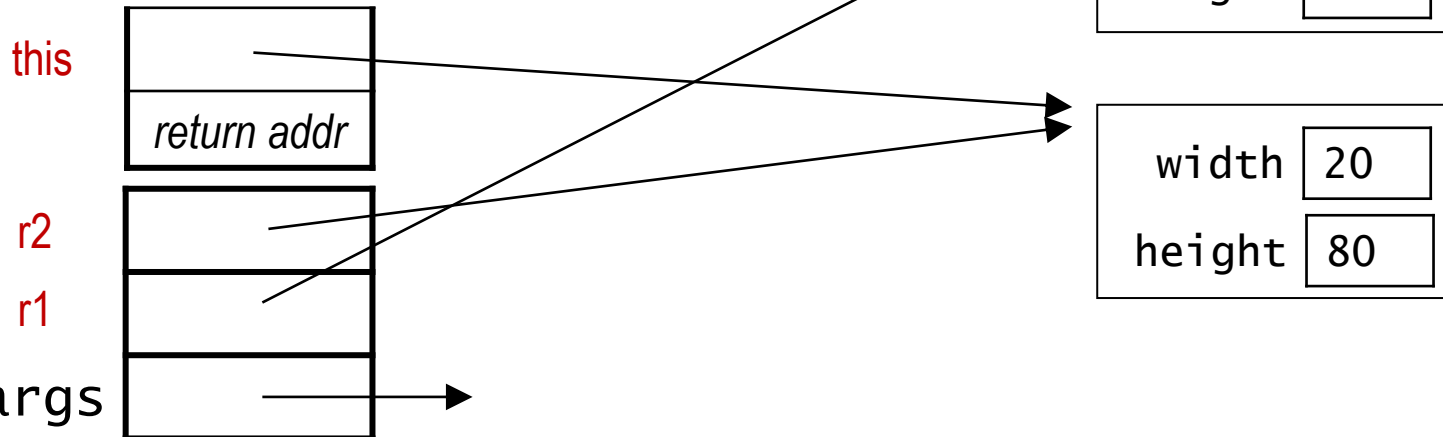
Memory Stack

Memory Heap

Static

Rectangles
Created

2



A sample Client Program:

memory trace

```
public class Rectangle {  
    private int width;  
    private int height;  
    .  
    .  
    public int area() {  
        return( width*height ); // returns 1600  
    }  
    .  
}
```

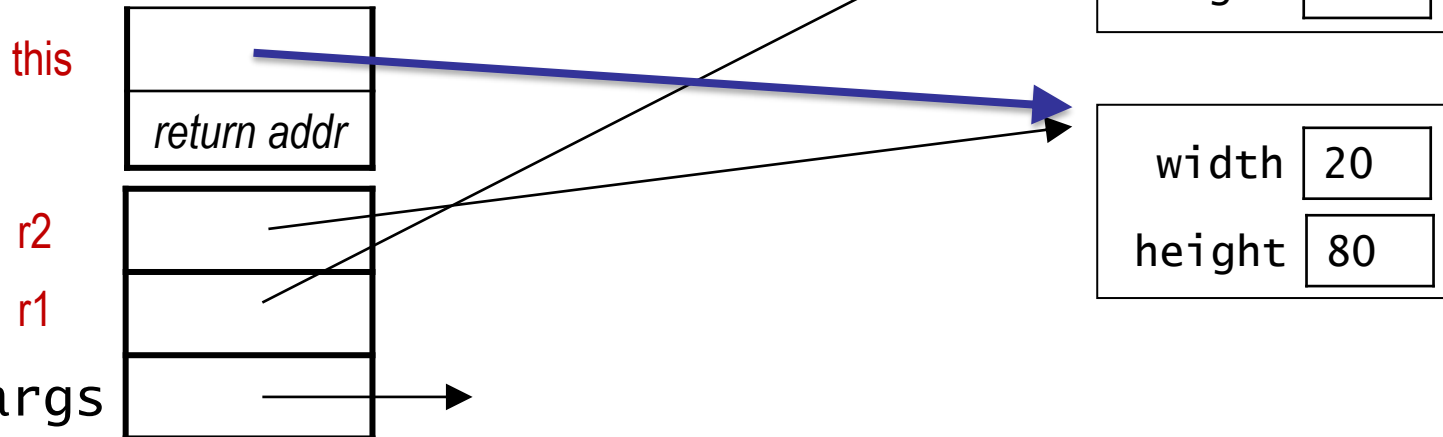
Memory Stack

Memory Heap

Static

Rectangles
Created

2



A sample Client Program:

memory trace

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
        System.out.println("r1's area = " + r1.area() );  
        System.out.println("r2's area = " + 1600 ); // outputs 1600  
        // grow both rectangles  
        r1.grow(50, 10);  
        r2.grow(5, 30);  
        System.out.println("r1: " + r1);  
        System.out.println("r2: " + r2);  
        ...  
    }  
}
```

Memory Stack

Memory Heap

Static

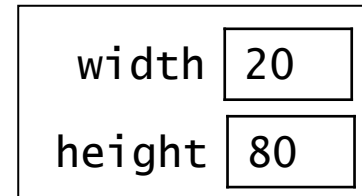
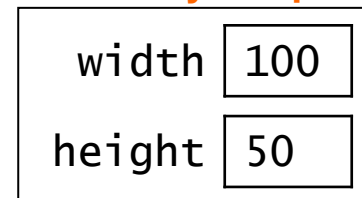
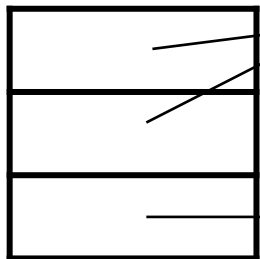
Rectangles
Created

2

r2

r1

args



A sample Client Program:

memory trace

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
        System.out.println("r1's area = " + r1.area() );  
        System.out.println("r2's area = " + 1600 ); // outputs 1600  
        // grow both rectangles  
        r1.grow(50, 10);  
        r2.grow(5, 30);  
        System.out.println("r1: " + r1);  
        System.out.println("r2: " + r2);  
        ...  
    }  
}
```

Memory Stack

Memory Heap

Static

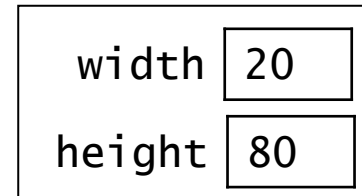
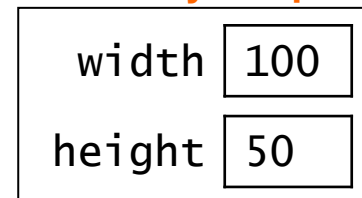
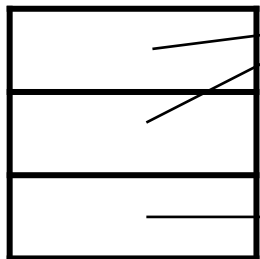
Rectangles
Created

2

r2

r1

args



A sample Client Program:

memory trace

```
public class Rectangle {  
    private int width;  
    private int height;  
    .  
    .  
    public void grow( int dw, int dh ) {  
        setWidth( width+dw );  
        setHeight( height+dh );  
    }  
}
```

Memory Stack

Memory Heap

Static

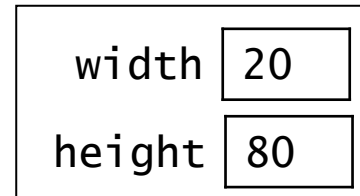
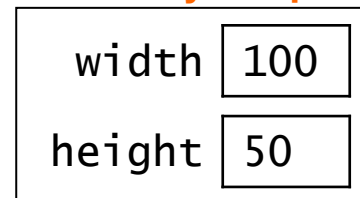
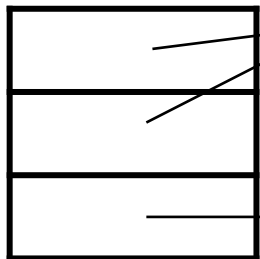
Rectangles
Created

2

r2

r1

args

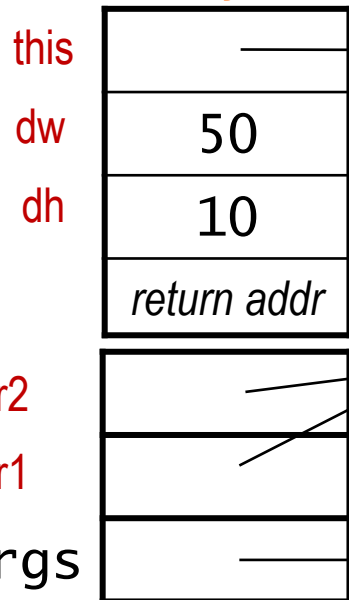


A sample Client Program:

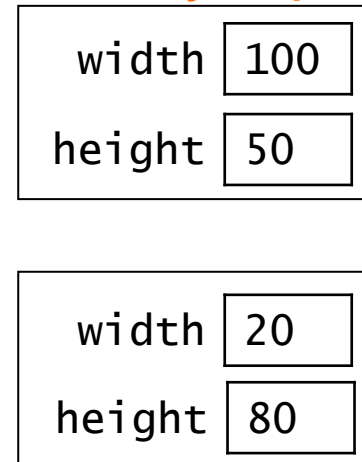
memory trace

```
public class Rectangle {  
    private int width;  
    private int height;  
    .  
    .  
    public void grow( int dw, int dh ) {  
        setWidth( width+dw );  
        setHeight( height+dh );  
    }  
}
```

Memory Stack



Memory Heap



Static

Rectangles
Created

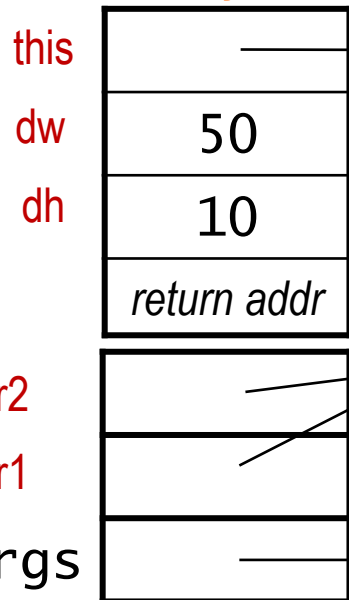
2

A sample Client Program:

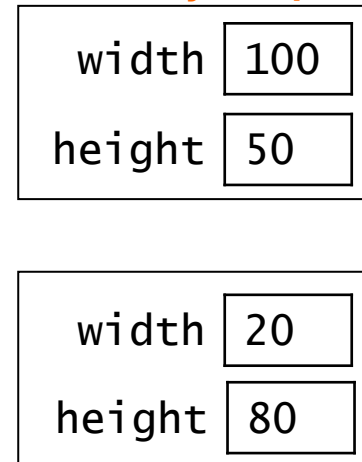
memory trace

```
public class Rectangle {  
    private int width;  
    private int height;  
    .  
    .  
    public void grow( int dw, int dh ) {  
        setWidth( width+dw );  
        setHeight( height+dh );  
    }  
}
```

Memory Stack



Memory Heap



Static

Rectangles
Created

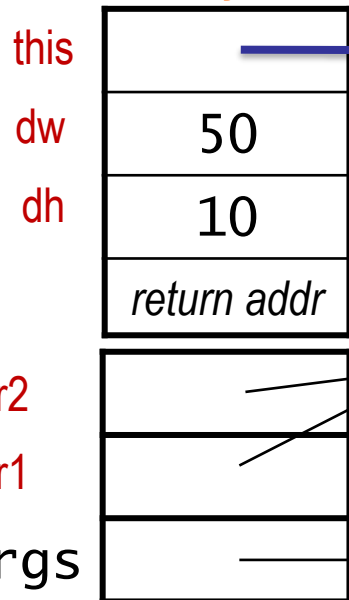
2

A sample Client Program:

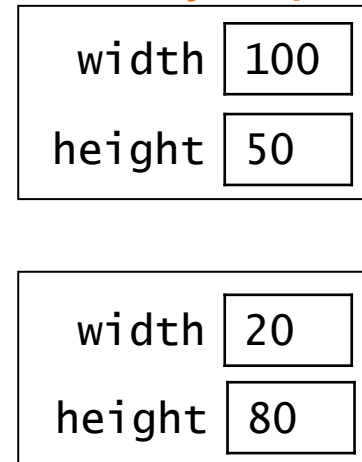
memory trace

```
public class Rectangle {  
    private int width;  
    private int height;  
    .  
    .  
    public void grow( int dw, int dh ) {  
        this.setWidth( width+dw );  
        setHeight( height+dh );  
    }  
}
```

Memory Stack



Memory Heap



Static

Rectangles
Created

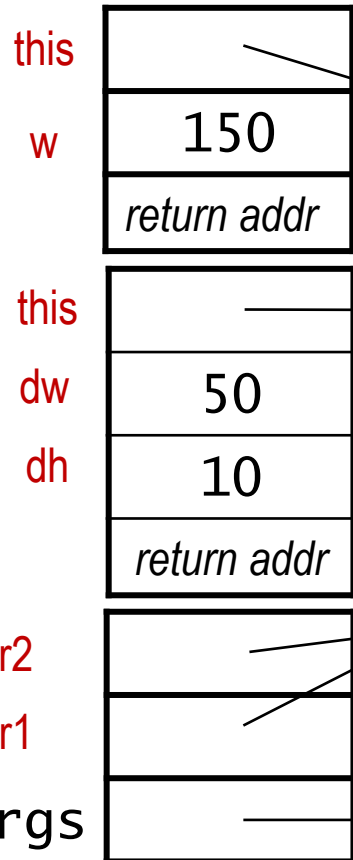
2

A sample Client Program:

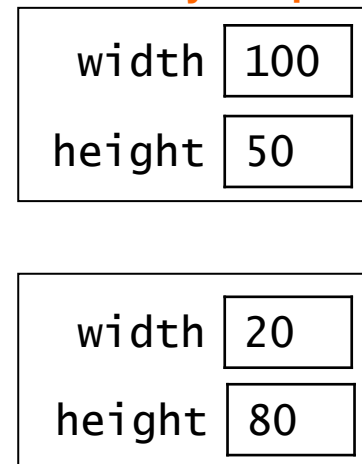
memory trace

```
public class Rectangle {  
    public void setwidth(int w) {  
        if (w <= 0) {  
            throw new IllegalArgumentException(  
        }  
        width = w;  
    }  
}
```

Memory Stack



Memory Heap



Static

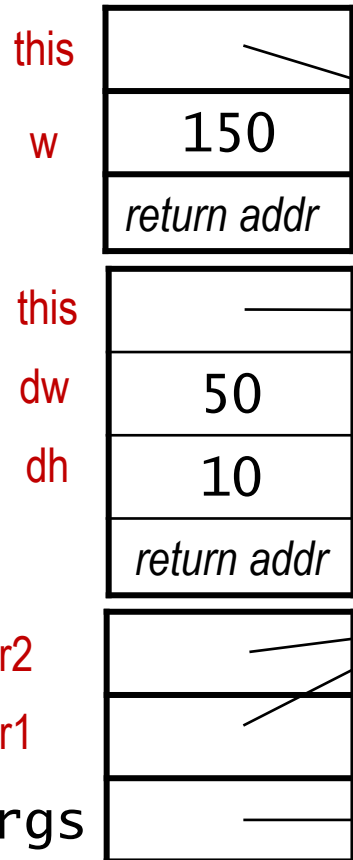
Rectangles Created 2

A sample Client Program:

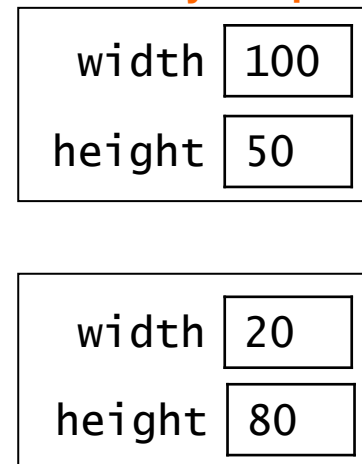
memory trace

```
public class Rectangle {  
    public void setWidth(int w) {  
        if (w <= 0) {  
            throw new IllegalArgumentException(  
                "width must be positive: " + w);  
        }  
        width = w;  
    }  
}
```

Memory Stack



Memory Heap



Static

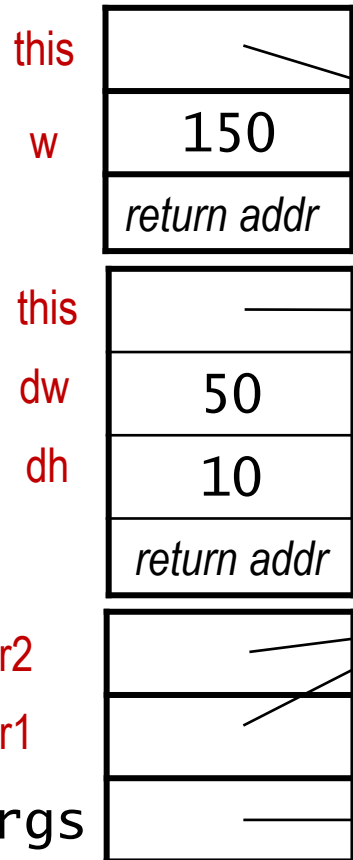
Rectangles Created 2

A sample Client Program:

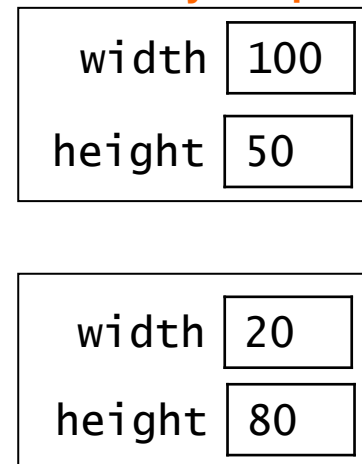
memory trace

```
public class Rectangle {  
    public void setWidth(int w) {  
        if (w <= 0) {  
            throw new IllegalArgumentException()  
        }  
        width = w;  
    }  
}
```

Memory Stack



Memory Heap



Static

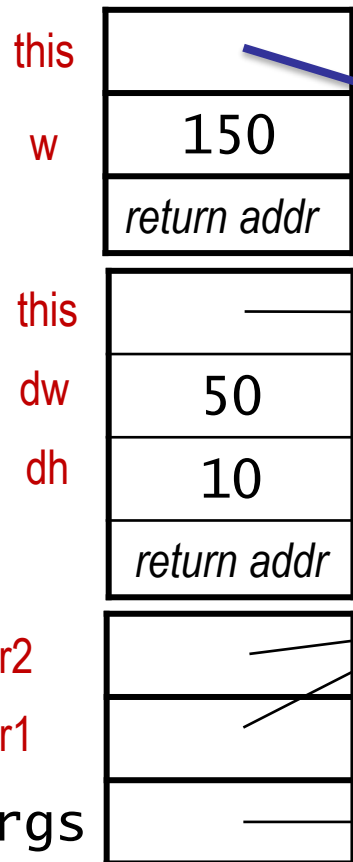
Rectangles Created 2

A sample Client Program:

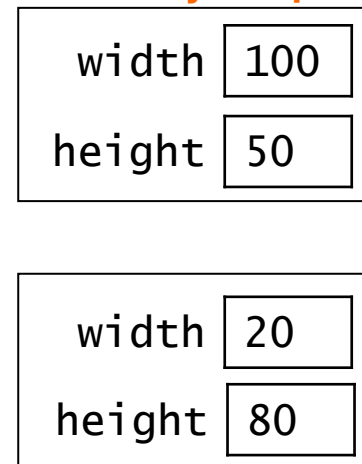
memory trace

```
public class Rectangle {  
    public void setWidth(int w) {  
        if (w <= 0) {  
            throw new IllegalArgumentException(  
                "width must be positive" );  
        }  
        this.width = w;  
    }  
}
```

Memory Stack



Memory Heap



Static

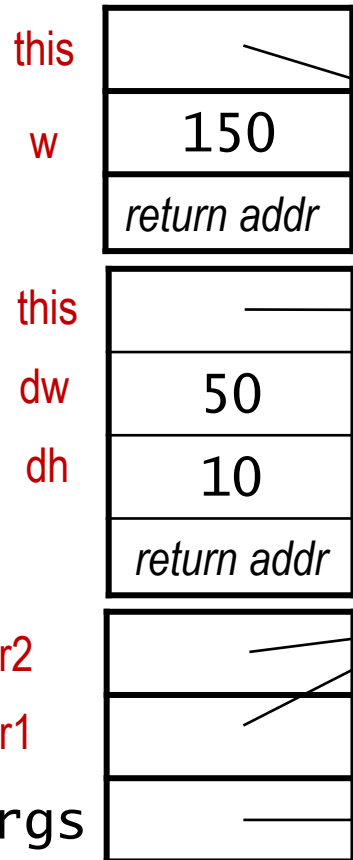
Rectangles Created 2

A sample Client Program:

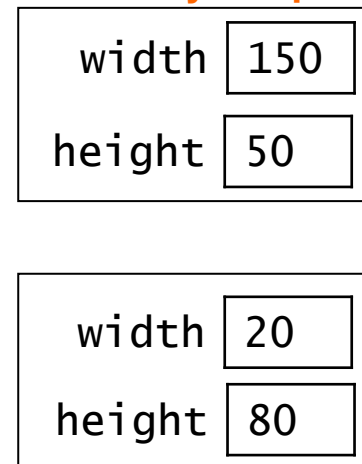
memory trace

```
public class Rectangle {  
    public void setWidth(int w) {  
        if (w <= 0) {  
            throw new IllegalArgumentException(  
                "width must be positive"  
            );  
        }  
        width = w;  
    }  
}
```

Memory Stack



Memory Heap



Static

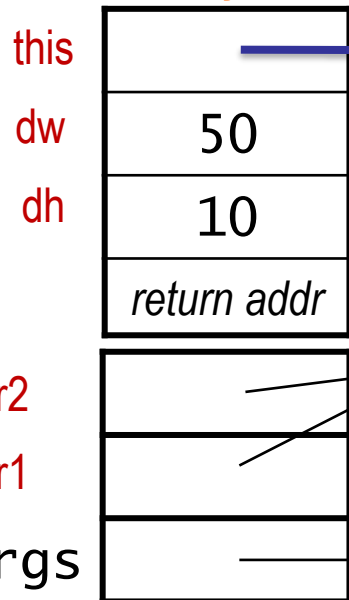
Rectangles Created 2

A sample Client Program:

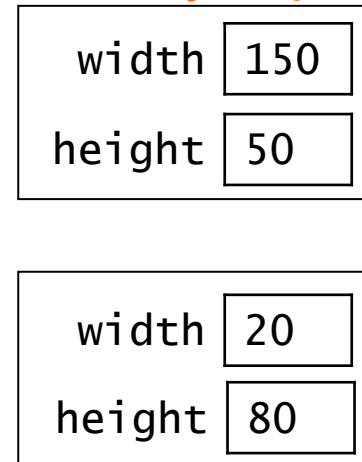
memory trace

```
public class Rectangle {  
    private int width;  
    private int height;  
    .  
    .  
    public void grow( int dw, int dh ) {  
        setWidth( width+dw );  
        setHeight( height+dh );  
    }  
}
```

Memory Stack



Memory Heap



Static

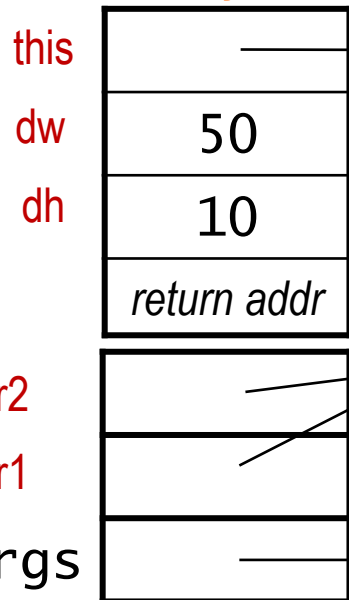
Rectangles Created 2

A sample Client Program:

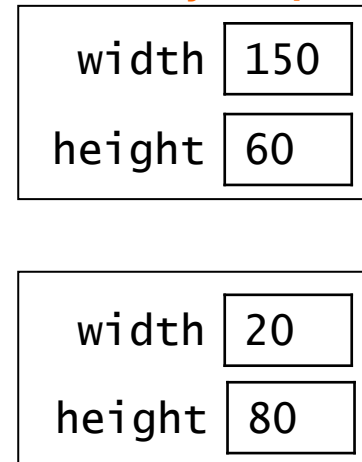
memory trace

```
public class Rectangle {  
    private int width;  
    private int height;  
    .  
    .  
    public void grow( int dw, int dh ) {  
        setWidth( width+dw );  
        setHeight( height+dh );  
    }  
}
```

Memory Stack



Memory Heap



Static

Rectangles
Created

2

A sample Client Program:

memory trace

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
        System.out.println("r1's area = " + r1.area() );  
        System.out.println("r2's area = " + 1600 ); // outputs 1600  
        // grow both rectangles  
        r1.grow(50, 10);  
        r2.grow(5, 30);  
        System.out.println("r1: " + r1);  
        System.out.println("r2: " + r2);  
        ...  
    }  
}
```

Memory Stack

Memory Heap

Static

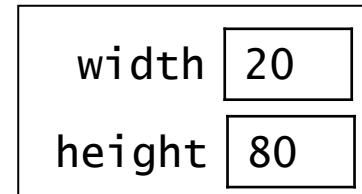
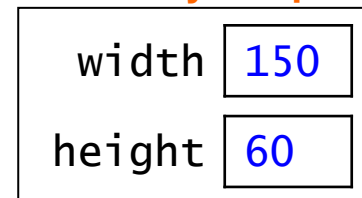
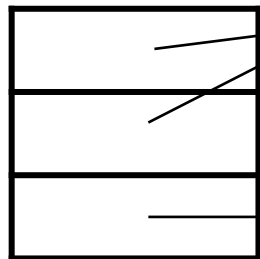
Rectangles
Created

2

r2

r1

args



A sample Client Program:

memory trace

```
public class RectangleClient {
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle(100, 50);
        Rectangle r2 = new Rectangle(20, 80);
        System.out.println("r1's area = " + r1.area() );
        System.out.println("r2's area = " + 1600 ); // outputs 1600
        // grow both rectangles
        r1.grow(50, 10);
        r2.grow(5, 30);
        System.out.println("r1: " + r1);
        System.out.println("r2: " + r2);
        ...
    }
}
```

Memory Stack

Memory Heap

Static

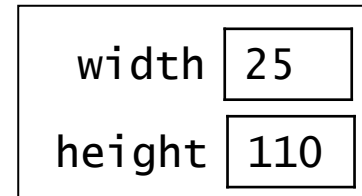
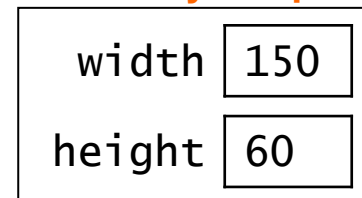
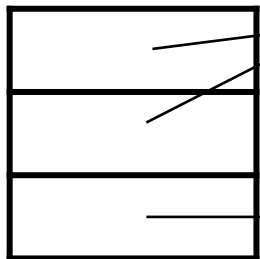
Rectangles
Created

2

r2

r1

args



A sample Client Program:

memory trace

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
        System.out.println("r1's area = " + r1.area() );  
        System.out.println("r2's area = " + 1600 ); // outputs 1600  
        // grow both rectangles  
        r1.grow(50, 10);  
        r2.grow(5, 30);  
        System.out.println("r1: " + r1);  
        System.out.println("r2: " + r2);  
        ...  
    }  
}
```

Memory Stack

Memory Heap

Static

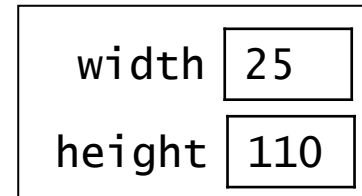
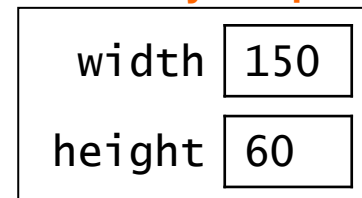
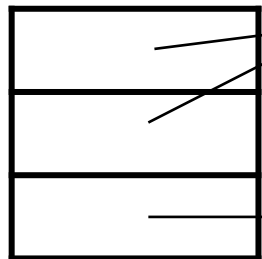
Rectangles
Created

2

r2

r1

args



A sample Client Program:

memory trace

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
        System.out.println("r1's area = " + r1.area() );  
        System.out.println("r2's area = " + 1600 ); // outputs 1600  
        // grow both rectangles  
        r1.grow(50, 10);  
        r2.grow(5, 30);  
        System.out.println("r1: " + r1);  
        System.out.println("r2: " + r2);  
        ...  
    }  
}
```

Memory Stack

Memory Heap

Static

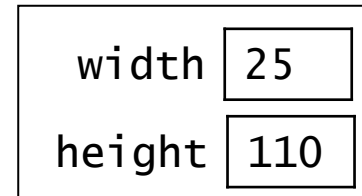
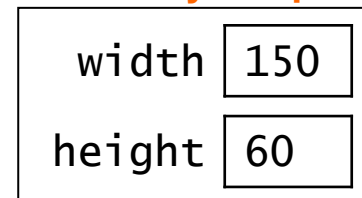
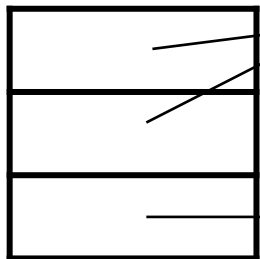
Rectangles
Created

2

r2

r1

args



A sample Client Program:

memory trace

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
        System.out.println("r1's area = " + r1.area() );  
        System.out.println("r2's area = " + 1600 ); // outputs 1600  
        // grow both rectangles  
        r1.grow(50, 10);  
        r2.grow(5, 30);  
        System.out.println("r1: " + r1.toString());  
        System.out.println("r2: " + r2);  
        ...  
    }  
}
```

Memory Stack

Memory Heap

Static

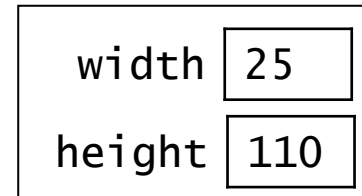
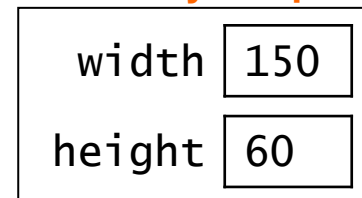
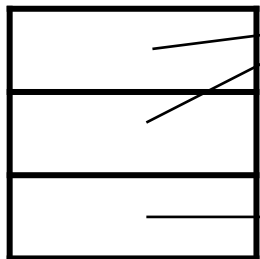
Rectangles
Created

2

r2

r1

args



A sample Client Program:

memory trace

```
public class Rectangle {  
    private int width;  
    private int height;  
    .  
    .  
    public String toString() {  
        return width + " x " + height;  
    }  
}
```

Memory Stack

Memory Heap

Static

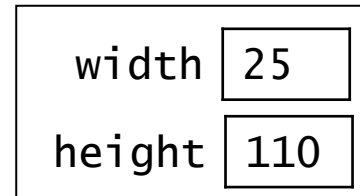
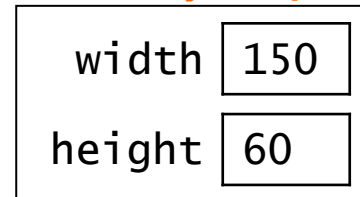
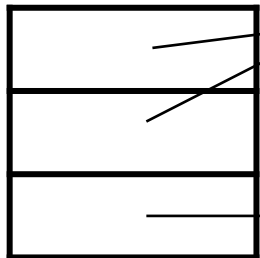
Rectangles
Created

2

r2

r1

args



A sample Client Program:

memory trace

```
public class Rectangle {  
    private int width;  
    private int height;  
    .  
    .  
    public string toString() {  
        return width + " x " + height;  
    }  
}
```

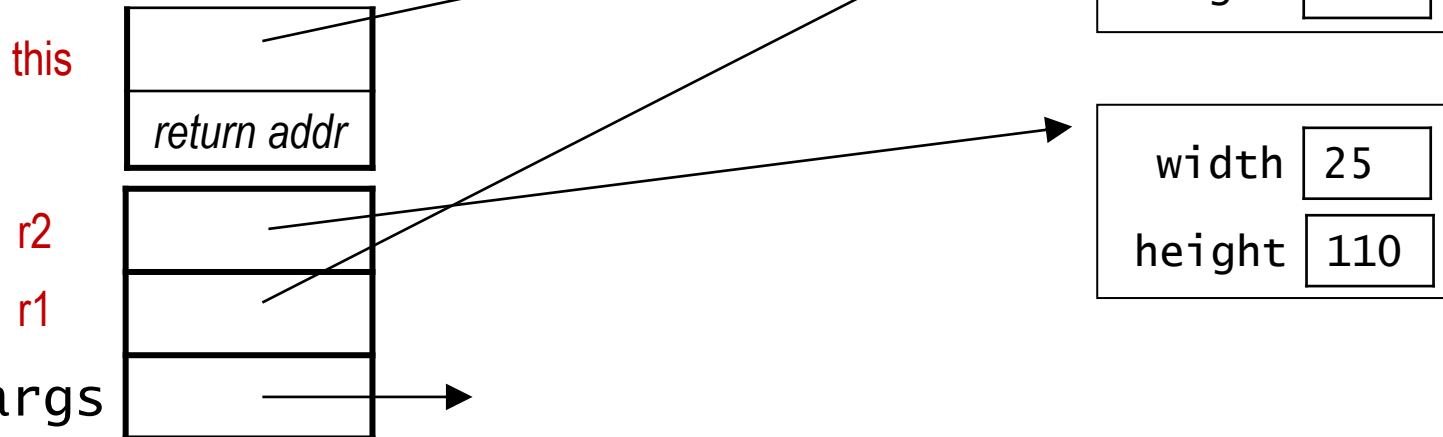
Memory Stack

Memory Heap

Static

Rectangles
Created

2



A sample Client Program:

memory trace

```
public class Rectangle {  
    private int width;  
    private int height;  
    .  
    .  
    public String toString() {  
        return width + " x " + height;  
    }  
}
```

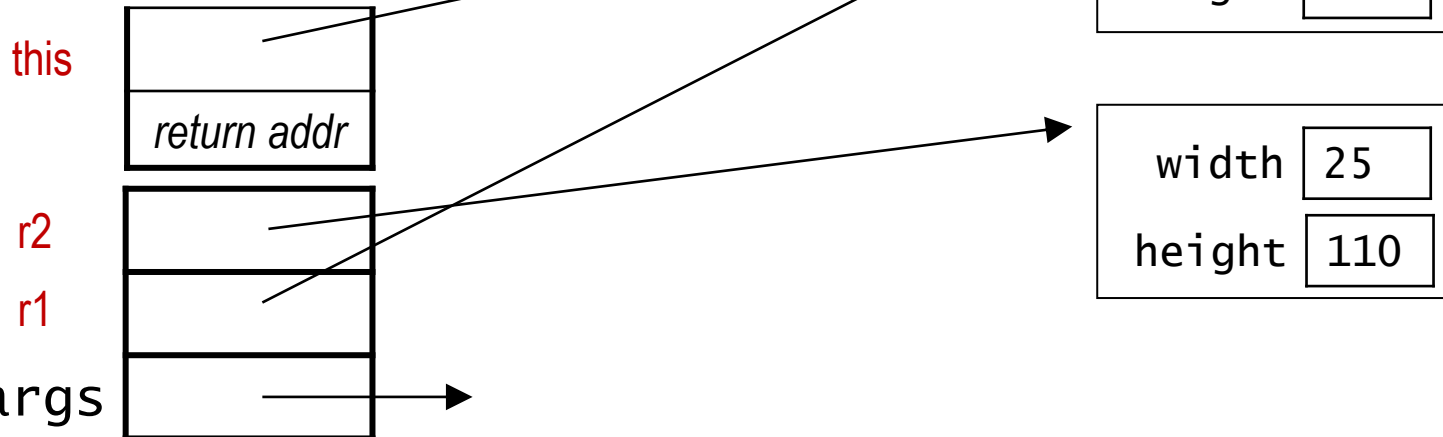
Memory Stack

Memory Heap

Static

Rectangles
Created

2



A sample Client Program:

memory trace

```
public class Rectangle {  
    private int width;  
    private int height;  
    .  
    .  
    public String toString() {  
        return this.width + " x " + this.height;  
    }  
}
```

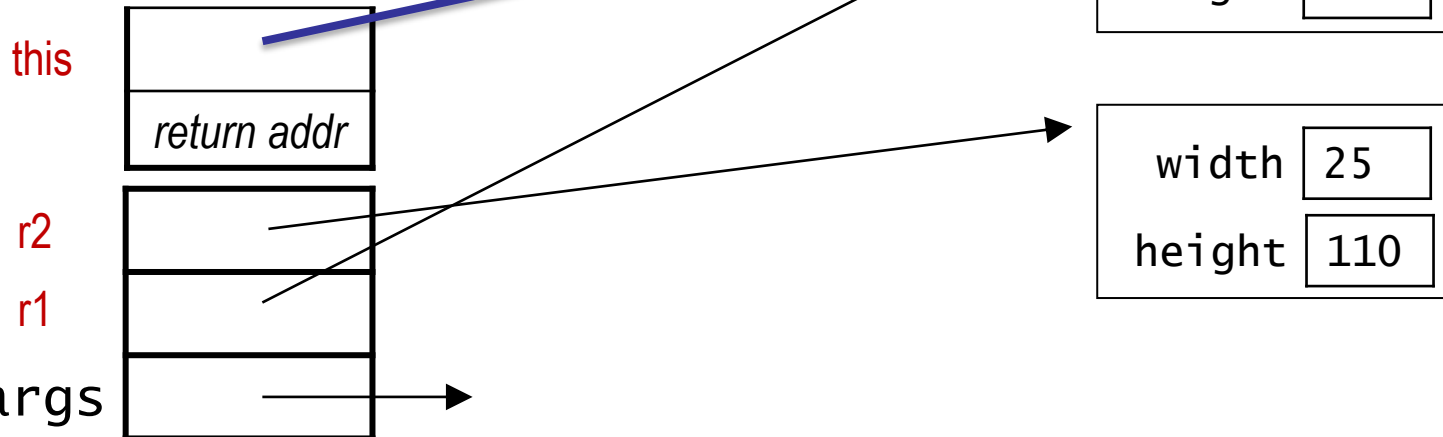
Memory Stack

Memory Heap

Static

Rectangles
Created

2



A sample Client Program:

memory trace

```
public class Rectangle {  
    private int width;  
    private int height;  
    .  
    .  
    public String toString() {  
        return width + " x " + height; // returns "150 x 60"  
    }  
}
```

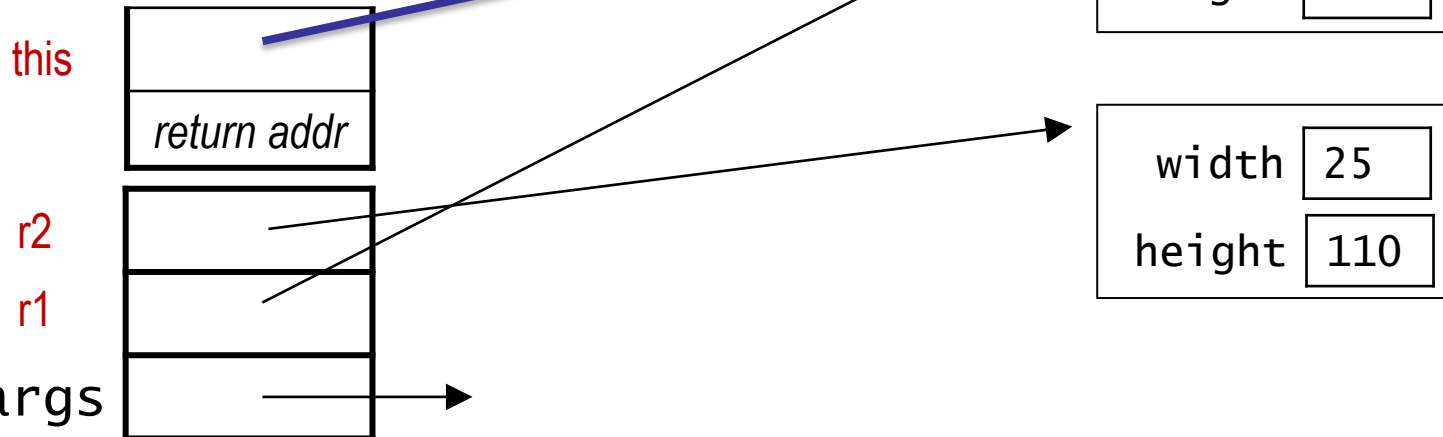
Memory Stack

Memory Heap

Static

Rectangles
Created

2



A sample Client Program:

memory trace

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
        System.out.println("r1's area = " + r1.area() );  
        System.out.println("r2's area = " + 1600 ); // outputs 1600  
        // grow both rectangles  
        r1.grow(50, 10);  
        r2.grow(5, 30);  
        System.out.println("r1: " + "150 x 60" );  
        System.out.println("r2: " + r2);  
        ...  
    }  
}
```

Memory Stack

Memory Heap

Static

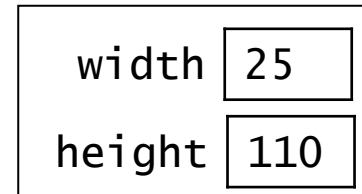
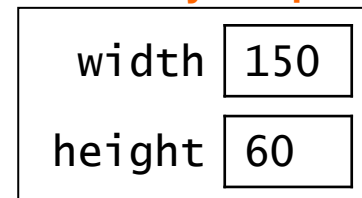
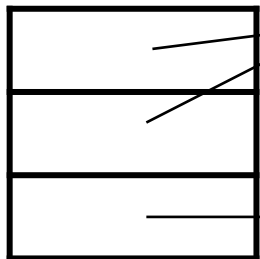
Rectangles
Created

2

r2

r1

args



A sample Client Program:

memory trace

```
public class RectangleClient {
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle(100, 50);
        Rectangle r2 = new Rectangle(20, 80);
        System.out.println("r1's area = " + r1.area() );
        System.out.println("r2's area = " + 1600 ); // outputs 1600
        // grow both rectangles
        r1.grow(50, 10);
        r2.grow(5, 30);
        System.out.println("r1: " + r1); // outputs r1: 150 x 60
        System.out.println("r2: " + r2);
        ...
    }
}
```

Memory Stack

Memory Heap

Static

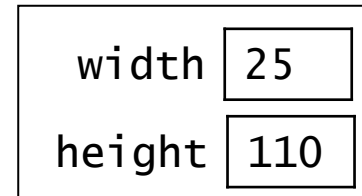
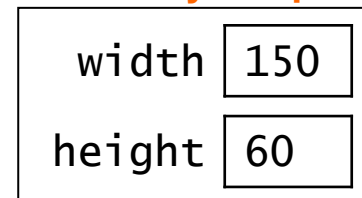
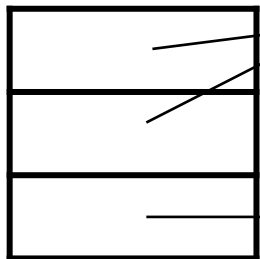
Rectangles
Created

2

r2

r1

args



A sample Client Program:

memory trace

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
        System.out.println("r1's area = " + r1.area() );  
        System.out.println("r2's area = " + 1600 ); // outputs 1600  
        // grow both rectangles  
        r1.grow(50, 10);  
        r2.grow(5, 30);  
        System.out.println("r1: " + r1);  
        System.out.println("r2: " + r2);  
        ...  
    }  
}
```

Memory Stack

Memory Heap

Static

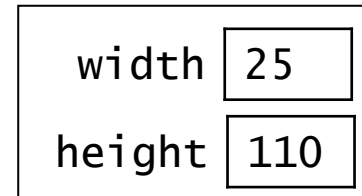
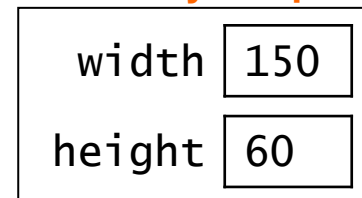
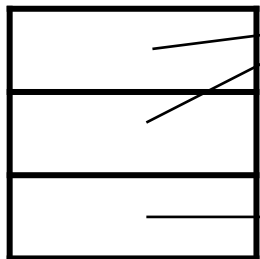
Rectangles
Created

2

r2

r1

args



A sample Client Program:

memory trace

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
        System.out.println("r1's area = " + r1.area() );  
        System.out.println("r2's area = " + 1600 ); // outputs 1600  
        // grow both rectangles  
        r1.grow(50, 10);  
        r2.grow(5, 30);  
        System.out.println("r1: " + r1);  
        System.out.println("r2: " + r2); // output r2: 25 x 110  
        ...  
    }  
}
```

Memory Stack

Memory Heap

Static

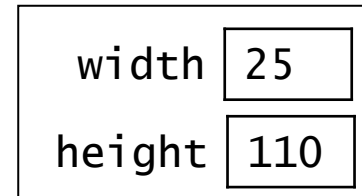
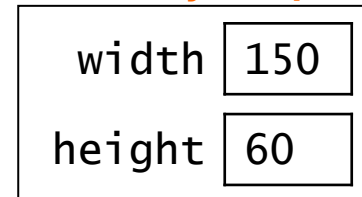
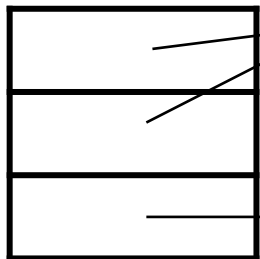
Rectangles
Created

2

r2

r1

args



A sample Client Program:

memory trace

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
        System.out.println("r1's area = " + r1.area() );  
        System.out.println("r2's area = " + 1600 ); // outputs 1600  
  
        ...  
  
        System.out.println("Number of Rectangles = " +  
                           Rectangle.numRectanglesCreated() );  
    }  
}
```

Memory Stack

Memory Heap

Static

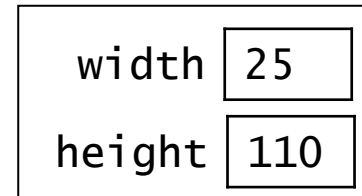
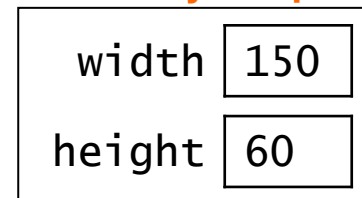
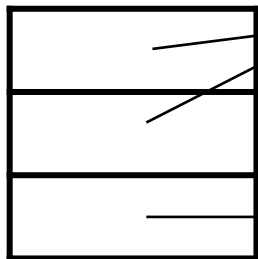
Rectangles
Created

2

r2

r1

args



A sample Client Program:

memory trace

```
public class Rectangle {  
    private int width;  
    private int height;  
    private static int rectanglesCreated;  
    .  
    .  
    public static int numRectanglesCreated() {  
        return( rectanglesCreated ); // return 2  
    }  
}
```

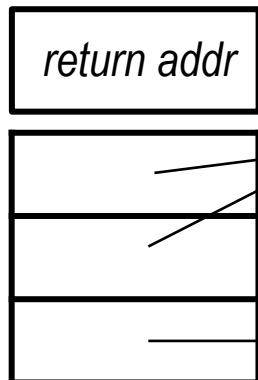
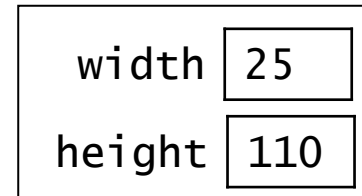
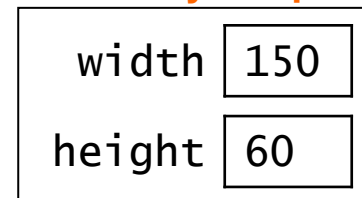
Memory Stack

Memory Heap

Static

Rectangles
Created

2



r2

r1

args

A sample Client Program:

memory trace

```
public class Rectangle {  
    private int width;  
    private int height;  
    private static int rectanglesCreated;  
    .  
    .  
    public static Rectangle[] createRectangles(int n) {  
        return new Rectangle[n];  
    }  
}
```

Note the absence of the
this reference!

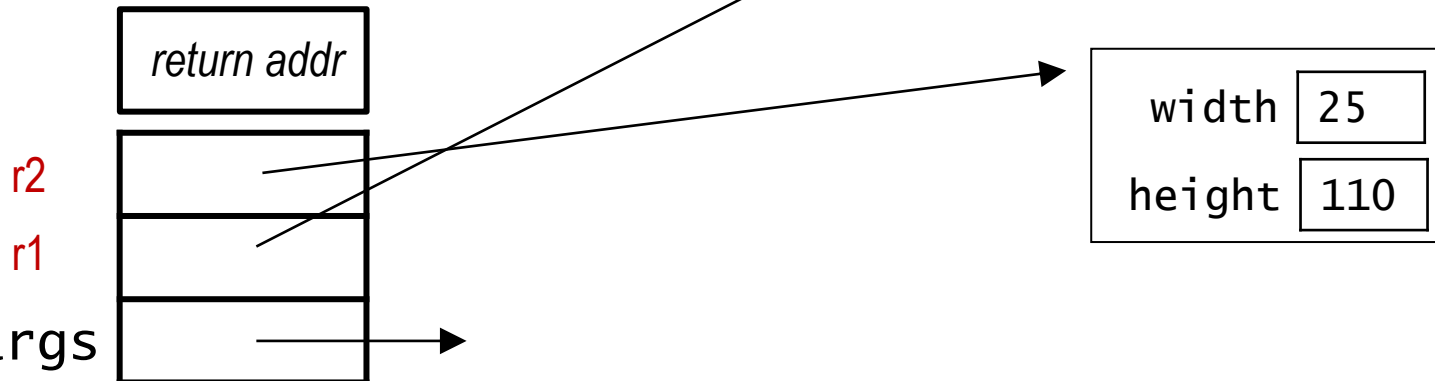
Memory Stack

Memory Heap

Static

Rectangles
Created

2



A sample Client Program:

memory trace

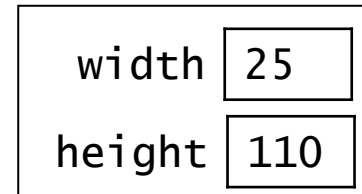
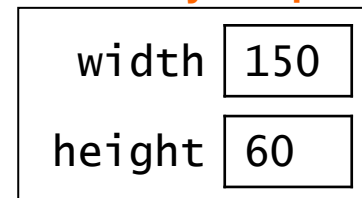
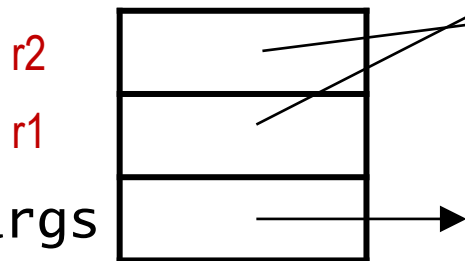
```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
        System.out.println("r1's area = " + r1.area() );  
        System.out.println("r2's area = " + 1600 ); // outputs 1600  
  
        ...  
  
        System.out.println("Number of Rectangles = " +  
                           Rectangle.numRectanglesCreated() ); // 2  
    } // end of the method
```

Memory Stack

Memory Heap

Rectangles
Created

2



A sample Client Program:

memory trace

```
public class RectangleClient {
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle(100, 50);
        Rectangle r2 = new Rectangle(20, 80);
        System.out.println("r1's area = " + r1.area() );
        System.out.println("r2's area = " + 1600 ); // outputs 1600

        ...

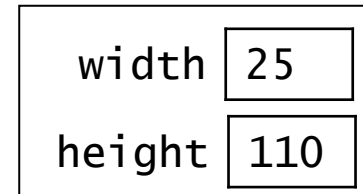
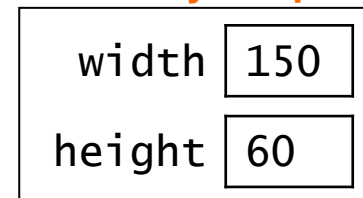
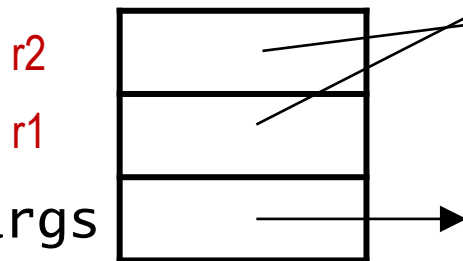
        System.out.println("Number of Rectangles = " +
                           Rectangle.numRectanglesCreated() ); // 2
    } // end of the method
```

Memory Stack

Memory Heap

Rectangles
Created

2



A sample Client Program:

memory trace

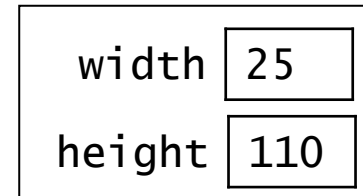
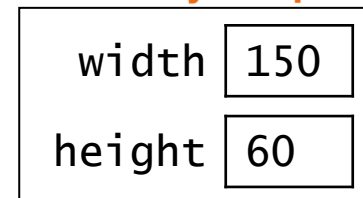
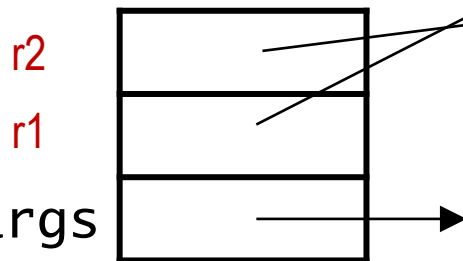
```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
        System.out.println("r1's area = " + r1.area() );  
        System.out.println("r2's area = " + 1600 ); // outputs 1600  
  
        ...  
  
        System.out.println("Number of Rectangles = " +  
                           Rectangle.numRectanglesCreated() ); // 2  
    } // end of the method
```

Memory Stack

Memory Heap

Rectangles
Created

2



A sample Client Program:

memory trace

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
        System.out.println("r1's area = " + r1.area());  
        System.out.println("r2's area = " + r2.area());  
  
        ...  
  
        System.out.println("Number of Rectangles = " +  
                           r1.numRectanglesCreated() ); // 2  
    } // end of the method
```

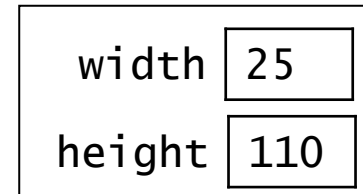
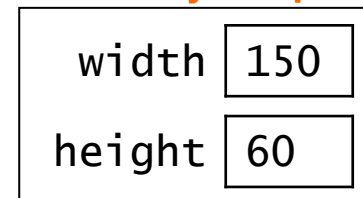
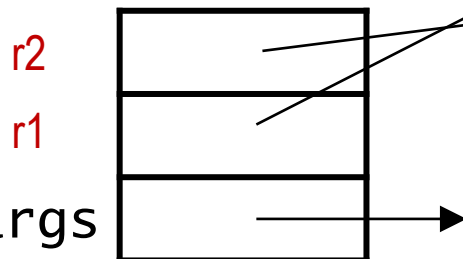
In Java, can also call a *static* method **through** an object!

Memory Stack

Memory Heap

Rectangles
Created

2



A sample Client Program:

memory trace

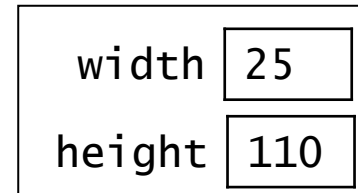
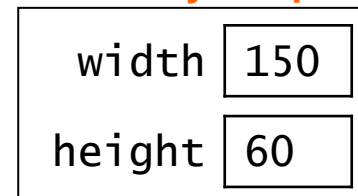
```
public class RectangleClient {
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle(100, 50);
        Rectangle r2 = new Rectangle(20, 80);
        System.out.println("r1's area = " + r1.area() );
        System.out.println("r2's area = " + 1600 ); // outputs 1600

        ...

        System.out.println("Number of Rectangles = " +
                           Rectangle.numRectanglesCreated() );
    } // end of the method
```

Memory Stack

Memory Heap



Rectangles
Created

2

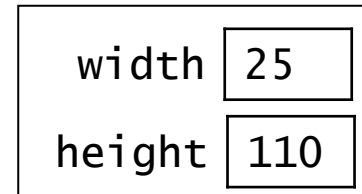
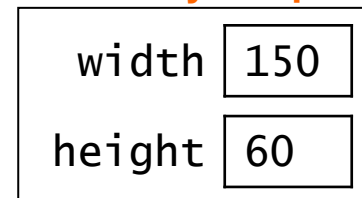
A sample Client Program:

memory trace

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
        System.out.println("r1's area = " + r1.area() );  
        System.out.println("r2's area = " + 1600 ); // outputs 1600  
  
        ...  
  
        System.out.println("Number of Rectangles = " +  
                           Rectangle.numRectanglesCreated() );  
    } // end of the method
```

Memory Stack

Memory Heap



A sample Client Program:

memory trace

```
public class RectangleClient {  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle(100, 50);  
        Rectangle r2 = new Rectangle(20, 80);  
        System.out.println("r1's area = " + r1.area() );  
        System.out.println("r2's area = " + 1600 ); // outputs 1600  
  
        ...  
  
        System.out.println("Number of Rectangles = " +  
                           Rectangle.numRectanglesCreated() );  
    } // end of the method
```

Memory Stack

Memory Heap

Classes as Custom Data Type:

a summary

```
public class TestClass {  
    public static void main(String[] args) {  
  
        int x;  
        int x = 5;  
        x = 12;  
        int j = x;  
        System.out.println(x);  
        if ( x == j ) {  
  
        }  
        if ( j == x ) {  
  
        }  
    } // end of main method  
} // end of class TestClass
```

Classes as Custom Data Type:

a summary

```
public class TestClass {  
    public static void main(String[] args) {  
  
        int x;           // no-arg constructor  
        int x = 5;       // custom constructor  
        x = 12;          // mutator/setter method  
        int j = x;       // accessor method  
        System.out.println(x); // toString method  
        if ( x == j ) {   // equals method  
  
        }  
        if ( j == x ) {   // equals method  
  
        }  
    } // end of main method  
} // end of class TestClass
```

Classes as Custom Data Type:

a summary

```
public class TestClass {  
    public static void main(String[] args) {  
  
        int x;                // Rectangle r = new Rectangle();  
        int x = 5;            // Rectangle r2 = new Rectangle(5);  
        x = 12;               // r.setWidth(12);  
        int j = x;            // r.getHeight();  
        System.out.println(x); // System.out.println(r);  
        if ( x == j ) {       // r.equals(r2);  
  
        }  
        if ( j == x ) {       // r2.equals( r );  
  
        }  
    } // end of main method  
} // end of class TestClass
```

Sample Rectangle Class (C++)

C++

```
class Rectangle {  
    private:  
        int width;  
        int height;  
  
    public:  
        Rectangle(int w, int h) {  
            setwidth(w);  
            setHeight(h);  
        }  
        void grow(int dw, int dh) {  
            setwidth(width+dw);  
            setHeight(height+dh);  
        }  
        double area() {  
            return(width*height);  
        }  
        boolean operator==(Rectangle other) {  
            return (other != null && width == other.width  
                    && height == other.height );  
        }  
}
```

```
// Destructor  
~Rectangle() {  
    // object clean-up  
}
```

Sample Rectangle Class (C++)

C++

```
class Rectangle {  
    private:  
        int width;  
        int height;  
  
    public:  
        Rectangle(int w, int h) {  
            setwidth(w);  
            setHheight(h);  
        }  
        void grow(int dw, int dh) {  
            setwidth(width+dw);  
            setHeight(height+dh);  
        }  
        double area() {  
            return(width*height);  
        }  
        boolean operator==(Rectangle other) {  
            return (other != null && width == other.width  
                    && height == other.height );  
        }  
}
```

Sample Rectangle Class (C++)

C++

```
class Rectangle {  
    private:  
        int width;  
        int height;  
  
    friend ostream &operator<<(ostream &out, Rectangle &r);  
    public:  
        Rectangle(int w, int h) {  
            setwidth(w);  
            setHheight(h);  
        }  
        void grow(int dw, int dh) {  
            this.setwidth(width+dw);  
            this.setHeight(height+dh);  
        }  
        double area() {  
            return(this.width*this.height);  
        }  
        boolean operator==(Rectangle other) {  
            return (other != null && this.width == other.width  
                    && this.height == other.height );  
        }  
}
```


Classes as Custom Data Type:

C++

```
// C++ allows operator overloading

int main( ) {

    int x;                                // Rectangle r;
    int x = 5;                            // Rectangle r2 = new Rectangle(5,10);
    x = 12;                               // r = 12;
    int j = x;                            // j = r;
    cout << x;                            // cout << r;
    if ( x == j ) {                       // r == r2);

    }
    if ( j == x ) {                       // r2 == r;

    }

} // end of main method

} // end of class TestClass
```

Classes as Custom Data Type:

C++

```
// C++ allows operator overloading

int main( ) {

    int x;                                // Rectangle r;
    int x = 5;                            // Rectangle r2 = new Rectangle(5,10);
    x = 12;                               // r.operator=(12);
    int j = x;                            // j = r;
    cout << x;                            // cout << r;
    if ( x == j ) {                       // r == r2);

    }
    if ( j == x ) {                       // r2 == r;

    }

} // end of main method

} // end of class TestClass
```

Classes as Custom Data Type:

C++

```
// C++ allows operator overloading

int main( ) {

    int x;                // Rectangle r;
    int x = 5;            // Rectangle r2 = new Rectangle(5,10);
    x = 12;               // r = 12;
    int j = x;            // j = r;
    cout << x;            // cout << r;
    if ( x == j ) {       // r == r2);

    }
    if ( j == x ) {       // r2 == r;

    }

} // end of main method

} // end of class TestClass
```

Classes as Custom Data Type:

C++

```
// C++ allows operator overloading

int main( ) {

    int x;                                // Rectangle r;
    int x = 5;                            // Rectangle r2 = new Rectangle(5,10);
    x = 12;                                // r = 12;
    int j = x;                            // operator=(j, r);
    cout << x;                            // cout << r;
    if ( x == j ) {                       // r == r2);

    }
    if ( j == x ) {                       // r2 == r;

    }

} // end of main method

} // end of class TestClass
```

Classes as Custom Data Type:

C++

```
// C++ allows operator overloading

int main( ) {

    int x;                                // Rectangle r;
    int x = 5;                            // Rectangle r2 = new Rectangle(5,10);
    x = 12;                               // r = 12;
    int j = x;                            // j = r;
    cout << x;                           // cout << r;
    if ( x == j ) {                       // r == r2);

    }
    if ( j == x ) {                       // r2 == r;

    }

} // end of main method

} // end of class TestClass
```

Classes as Custom Data Type:

C++

```
// C++ allows operator overloading

int main( ) {

    int x;                                // Rectangle r;
    int x = 5;                            // Rectangle r2 = new Rectangle(5,10);
    x = 12;                               // r = 12;
    int j = x;                            // j = r;
    cout << x;                            // operator<<(cout, r);
    if ( x == j ) {                       // r == r2

    }
    if ( j == x ) {                       // r2 == r

    }

} // end of main method

} // end of class TestClass
```

Classes as Custom Data Type:

C++

```
// C++ allows operator overloading

int main( ) {

    int x;                                // Rectangle r;
    int x = 5;                            // Rectangle r2 = new Rectangle(5,10);
    x = 12;                               // r = 12;
    int j = x;                            // j = r;
    cout << x;                            // cout << r;
    if ( x == j ) {                       // r == r2

    }

    if ( j == x ) {                       // r2 == r

    }

} // end of main method

} // end of class TestClass
```

Classes as Custom Data Type:

C++

```
// C++ allows operator overloading

int main( ) {

    int x;                                // Rectangle r;
    int x = 5;                            // Rectangle r2 = new Rectangle(5,10);
    x = 12;                               // r = 12;
    int j = x;                            // j = r;
    cout << x;                            // cout << r;
    if ( x == j ) {                       // r.operator==(r2)

    }
    if ( j == x ) {                       // r2 == r

    }

} // end of main method

} // end of class TestClass
```


Classes as Custom Data Type:

C++

```
// C++ allows operator overloading

int main( ) {

    int x;                // Rectangle r;
    int x = 5;            // Rectangle r2 = new Rectangle(5,10);
    x = 12;               // r = 12;
    int j = x;            // j = r;
    cout << x;            // cout << r;
    if ( x == j ) {       // r == r2;

    }
    if ( j == x ) {       // r2 == r
    }

    } // end of main method
} // end of class TestClass
```

Classes as Custom Data Type:

C++

```
// C++ allows operator overloading

int main( ) {

    int x;                // Rectangle r;
    int x = 5;            // Rectangle r2 = new Rectangle(5,10);
    x = 12;               // r = 12;
    int j = x;            // j = r;
    cout << x;            // cout << r;
    if ( x == j ) {       // r == r2;

    }
    if ( j == x ) {       // r2.operator==(r)
    }

    } // end of main method
} // end of class TestClass
```

Sample Rectangle Class (Python)

[Python](#)

```
class Rectangle:
    def __init__(self, init_width=15, init_height=15):
        self.x = 0
        self.y = 0
        self.width = init_width
        self.height = init_height

    def grow(self, dwidth, dheight):
        self.width += dwidth
        self.height += dheight

    def area(self):
        return self.width * self.height

    ...

    def __eq__(self, other):
        if self.width == other.width and self.height == other.height:
            return True
        else:
            return False

    def __repr__(self):
        return str(self.width) + ' x ' + str(self.height)
```

Sample Rectangle Class (Python)

[Python](#)

```
class Rectangle:
    def __init__(self, init_width, init_height):
        self.x = 0
        self.y = 0
        self.width = init_width
        self.height = init_height

    def grow(self, dwidth, dheight):
        self.width += dwidth
        self.height += dheight

    def area(self):
        return self.width * self.height

    ...

    def __eq__(self, other):
        if self.width == other.width and self.height == other.height:
            return True
        else:
            return False

    def __repr__(self):
        return str(self.width) + ' x ' + str(self.height)
```

Sample Rectangle Class (Python)

[Python](#)

```
class Rectangle:
    def __init__(self, init_width, init_height):
        self.x = 0
        self.y = 0
        self.width = init_width
        self.height = init_height

    def grow(self, dwidth, dheight):
        self.width += dwidth
        self.height += dheight

    def area(self):
        return self.width * self.height

    ...

    def __eq__(self, other):
        if self.width == other.width and self.height == other.height:
            return True
        else:
            return False

    def __repr__(self):
        return str(self.width) + ' x ' + str(self.height)
```

Classes as Custom Data Type:

Python

// Python allows *operator* overloading

```
x = 5
x = 12
j = x
print(x)
if ( x == j ) {

}

if ( j == x ) {

}

// r = Rectangle(5, 10)
// mutator methods?
// accessor methods?
// print(r)
// r == other

// other == r
```

Classes as Custom Data Type:

Python

// Python allows *operator* overloading

```
x = 5                // r = Rectangle(5, 10)
x = 12              // object data members
j = x               // can be directly accessed
print(x)           // r.repr() or r.str()
if ( x == j ) {    // r == other

}

if ( j == x ) {    // other == r

}
```

Classes as Custom Data Type:

Python

// Python allows *operator* overloading

```
x = 5
x = 12
j = x
print(x)
if ( x == j ) {
```

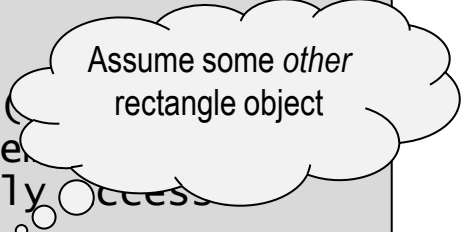
```
}
```

```
if ( j == x ) {
```

```
}
```

```
// r = Rectangle()
// object data member
// can be directly accessed
// print(r)
// r.__eq__(other)
```

```
// other.__eq__(r)
```



Assume some *other*
rectangle object

Classes as Custom Data Type:

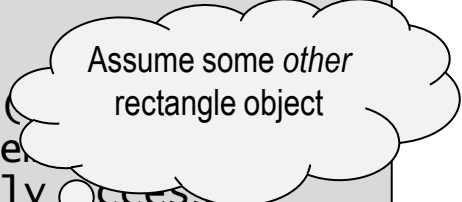
Python

// Python allows *operator* overloading

```
x = 5
x = 12
j = x
print(x)
if ( x.__eq__(j) ) {
}

if ( j.__eq__(x) ) {
}
```

```
// r = Rectangle()
// object data member
// can be directly accessed
// print(r)
// r.__eq__(other)
```



Assume some *other*
rectangle object

Your TURN!!!

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    public Rectangle(int w, int h) {  
        setWidth(w);  
        setHeight(h);  
    }  
    public Rectangle(int dim) {  
        this(dim, dim);  
    }  
    public Rectangle() {  
        this(0);  
    }  
    .  
    .  
    .  
  
    public static void main( String [] args ) {  
        Rectangle r1 = new Rectangle();  
    }  
}
```